Reading Notes 5

Qimu Luo

2024 年 3 月 27 日

目录

1 引入 **3**
1.1 机器学习定义 . . . 3
1.2 机器学习算法的类型（即书 2.5） . . . 4
1.2.1 监督学习 (Supervised Learning, SL) . . . 4
1.2.2 无监督学习 (Unsupervised Learning, UL) . . . 4
1.2.3 强化学习 (Reinforcement Learning, RL) . . . 4
1.3 传统学习或浅层学习（含书 2.6.1） . . . 5
1.3.1 特征选择 (Feature selection) . . . 5
1.3.2 特征抽取 (Feature Extraction) . . . 6
1.4 表示学习 (含书 2.6) . . . 6
1.5 深度学习 (含书 2.6.2) . . . 8

2 机器学习 (或模式识别 Pattern Recognizing, PR) **9**
2.1 基本概念 . . . 9
2.2 三个基本要素 . . . 10
2.2.1 模型 . . . 10
2.2.2 学习准则 . . . 11
2.2.3 优化算法 . . . 14
2.3 简单案例：线性回归 (Linear Regression) . . . 17
2.3.1 基础概念 . . . 17
2.3.2 参数学习 . . . 17
2.4 偏差-方差分解 (Bias-Variance Decomposition) . . . 22
2.4.1 偏差、方差公式推导 . . . 22
2.4.2 图像分析 . . . 23
2.4.3 解决方案 . . . 24
2.5 评价指标 . . . 25
2.5.1 准确和错误率率 . . . 25
2.5.2 查准率和查全率 . . . 25
2.5.3 宏平均和微平均 . . . 26
2.5.4 其他方法 . . . 27

2.6 理论和定理 . . . 27
2.6.1 PAC 学习理论 (Probably Approximately Correct) . . . 27
2.6.2 没有免费午餐定理 (No Free Lunch Theorem, NFL) . . . 28
2.6.3 丑小鸭定理 (Ugly Ducking Theorem) . . . 28
2.6.4 奥卡姆剃刀 (Occam's Razor) . . . 29
2.6.5 归纳偏置 (Inductive Bias) . . . 29

3 线性模型 (Linear Model) 29
3.1 引入 . . . 29
3.2 线性判别函数和决策边界 . . . 30
3.2.1 两类分类 (Binary Classification) . . . 30
3.2.2 多类分类 (Multi-class Classification) . . . 31
3.3 Logistic 回归 (Logistic Regression, LR) . . . 32
3.3.1 模型 . . . 32
3.3.2 参数学习 . . . 34
3.4 Softmax 回归 (Soft Regression) . . . 34
3.4.1 模型 . . . 34
3.4.2 参数学习 . . . 35
3.5 感知器 (perceptron) . . . 36
3.5.1 参数学习 . . . 36
3.5.2 感知器的收敛性 . . . 38
3.5.3 参数平均感知器 . . . 38
3.5.4 扩展到多分类 . . . 39
3.6 支持向量机 (Support Vector Machine, SVM) . . . 41
3.6.1 寻找目标函数 . . . 41
3.6.2 参数学习 . . . 42
3.6.3 软间隔 (Soft Margin) . . . 46
3.6.4 核函数 (Kernel Function) . . . 47
3.7 三种二分类模型损失函数对比 . . . 48
3.7.1 公式改写 . . . 48
3.7.2 图像分析 . . . 49
3.8 总结 . . . 49

4 前馈神经网络 (Feedforward Neural Network, FNN) 50
4.1 神经元 (Artificial Neuron) . . . 50
4.1.1 Sigmoid 型函数 . . . 52
4.1.2 ReLU 函数 (Rectified Linear Unit，修正线性单元) . . . 54
4.1.3 Swish 函数 . . . 56
4.1.4 GELU 函数 (Gaussian Error Linear Unit, 高斯误差线性单元) . . . 57
4.1.5 Maxout 单元 . . . 58
4.2 网络结构 . . . 58
4.3 前馈神经网络 . . . 59

4.3.1 结构表示以及信息传递方式 . . . 60
4.3.2 通用近似定理 (Universal Approximation Theorem) . . . 61
4.3.3 机器学习应用 . . . 62
4.3.4 参数学习 . . . 63
4.4 反向传播算法 (Backpropagation, BP) . . . 63
4.5 自动梯度计算 . . . 66
4.5.1 数值微分 (Numerical Differentiation) . . . 66
4.5.2 符号微分 (Symbolic Differentiation) . . . 67
4.5.3 自动微分 (Automatic Differentiation, AD) . . . 67
4.6 优化问题 . . . 69
4.6.1 非凸优化 . . . 69
4.6.2 梯度消失问题 (Vanishing Gradient) . . . 70

5 卷积神经网络 71
5.1 引入 . . . 71
5.2 卷积 (Convolution) . . . 71
5.2.1 一维卷积–信号处理 . . . 71
5.2.2 二维卷积–图像处理 . . . 73
5.2.3 互相关 (Cross-Correlation) . . . 73
5.2.4 卷积变种 . . . 74
5.2.5 卷积数学性质 . . . 74
5.3 卷积神经网络 . . . 75
5.3.1 用卷积代替全连接 . . . 75
5.3.2 卷积层 (Convolution layer–图像提取) . . . 76
5.3.3 汇聚层 (Pooling layer–图像压缩) . . . 78
5.3.4 整体结构 . . . 79
5.4 参数学习 . . . 79
5.5 卷积网络反向传播算法 . . . 80
5.6 几种典型的卷积 . . . 80
5.6.1 LeNet-5 . . . 80
5.6.2 AlexNet . . . 81
5.6.3 Inception 网络 . . . 81
5.6.4 残差网络 (Residual Network) . . . 82

1 引入

1.1 机器学习定义

1. 机器学习是指从有限的观测数据中学习或者猜测到一般规律，并将这些规律运用到未观测的样本上的方法。(Learn rule from known, apply it into unknown.)

2. 传统机器学习的一般模式：原始数据 $\longrightarrow$ 特征处理 (预处理、特征提取、特征转换) $\longrightarrow$ 预测 (浅层学习) $\longrightarrow$ 结果

1.2 机器学习算法的类型（即书 2.5）

机器学习按模型分类，可分为线性和非线性模型；按学习准则的不同，可分为统计方法和非统计方法；但一般按照训练样本提供的信息以及反馈方式的不同可分为监督学习、无监督学习、强化学习。

1.2.1 监督学习 (Supervised Learning, SL)

如果机器学习的目标是建模样本特征 $\mathbf{x}$ 和标签 y 之间的关系，且训练集中每个样本都有对应的标签，即称为监督学习。据标签类型的不同又分为以下几类：

1. 回归 (Regression) 问题中的标签是连续值，输出 $f(\mathbf{x}, \theta)$ 也是连续值。

2. 分类 (Classification) 问题中的标签 y 是离散的类别，学习到的模型也称为分类器 (Classifier)。根据类别数量分为两类分类 (Binary Classification) 和多类分类问题 (Multi-class Classification)。

3. 结构化学习 (Structure Learning) 的输出是结构化的对象，如序列，树或者图。由于输出空间较大，因此定义一个联合空间，将 x 和 y 映射为空间中的联合特征向量 $\phi(\mathbf{x}, \mathbf{y})$，预测模型可以写为：

$$\hat{\mathbf{y}} = \underset{y \in Gen(x)}{argmax}\, f(\phi(\mathbf{x}, \mathbf{y}), \theta) \tag{1}$$

其中 $Gen(x)$ 表示输入 $\mathbf{x}$ 所有可能的输出目标的集合，计算 argmax 即使函数达到最大值的 y 的取值 (Why max-函数代表样本和标签的匹配程度，在训练时，穷举所有可能的输出后，希望正确的结果的 $F(\mathbf{x}, \hat{\mathbf{y}})$ 是最大的) 的过程称为解码 (decoding)，一般通过动态规划的方法来计算。

应用例：结构化学习和深度神经网络 (DNN) 解决手写数字识别:

$$F(\mathbf{x}, \mathbf{y}) = -CE(N(X), y)$$

(需结合后面交叉熵)

1.2.2 无监督学习 (Unsupervised Learning, UL)

指从不包含目标标签的训练样本中自动学习到一些有效信息。典型的如聚类、密度估计、特征学习、降维等。

1.2.3 强化学习 (Reinforcement Learning, RL)

是一类通过交互来学习的机器学习算法。智能体根据环境状态做出一个动作，并得到及时或者延时的奖励，在交互过程中不断调整学习策略，以最大化期望总回报。三者的比较如图 1所示。jjj 监督学习 (任务驱动型)，需要每个样本都有标签，而无监督学习 (数据驱动型) 不需要标签。但由于人工标注的成本很高，因此还有弱监督学习 (Weak Supervised Learning) 还有半监督学习 (Semi-Supervised Learning) 的方法，以从大规模的无标注数据中充分挖掘有

	监督学习	无监督学习	强化学习
训练样本	训练集 $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$	训练集 $\{\mathbf{x}^{n}\}_{n=1}^{N}$	智能体和环境交互的轨迹 τ 和累积奖励 G_τ
优化目标	$y = f(\mathbf{x})$ 或 $p(y\|\mathbf{x})$	$p(\mathbf{x})$ 或带隐变量 $\mathbf{z}$ 的 $p(\mathbf{x}\|\mathbf{z})$	期望总回报 $\mathbb{E}_\tau[G_\tau]$
学习准则	期望风险最小化 最大似然估计	最大似然估计 最小重构错误	策略评估 策略改进

图 1: Comparison between 3 types of machine learning

用信息，减少需要标注的样本。强化学习 (错误驱动型) 更偏重于智能体与环境的交互，不需要显示地以“输入/输出对”的方式给出训练样本，是在线学习。在试错（exploration）与开发（exploitation）之间折中权衡，开发已有的经验来获取收益 (award)，同时也要进行试探，使得未来可以获得更好的动作选择空间。

1.3 传统学习或浅层学习（含书 2.6.1）

传统的机器学习即浅层学习，主要关注如何学习一个**预测模型**，而不参与数据的特征学习。因此需要人工对数据进行一系列特征处理，当机器学习被应用到实际生活中时，要面对各式各样的数据形式，比如声音、图像、文本。人们需要对特征进行合理的表示，使其对计算机合理化。例如图像类数据可以被视为一个连续的向量，文本数据可以被视为离散符号。因此在接收到原始数据之后，对特征处理的大体流程如下：

1. 数据预处理：对数据去除噪声即一些干扰项比如文本的停用词。
2. 特征提取：从原始数据中提取有效特征。例如在图像分类中，提取边缘、尺度不变特征变换等特征等。
3. 特征转换：对特征进行如降维或者升维方面的加工。

降维是将高维度的数据保留下最重要的一些特征，去除噪声和不重要的特征，从而实现提升数据处理速度的目的。其主要途径是特征抽取和特征选择，两者达到的效果都是试图减少特征数据集中的属性 (特征) 的数目。但是两者所采用的方式方法不同。

1.3.1 特征选择 (Feature selection)

即仍然在原始世界，对现有的数据进行筛选，“取其精华，去其糟粕”。其方法是从原始特征数据集中选择出子集，是一种包含的关系，使得基于这个特征子集训练出来的模型的准确率最高。**没有更改**原始的特征空间。特征选择的直接方法是子集搜索 (subset search)：假设原始特征数为 d，则共有 2^d 个候选子集，目标即选择一个最优的子集。如果直接测试每个子集的准确率来实现目标效率太低。更合理的两种方法：前向搜索 (forward search) 指从空集开始，每轮添加该轮最优特征；反向搜索 (backward search)，指从原始特征开始，每次删除最无用的特征。子集搜索又分为几种方法：

- 过滤式 (Filter)：对数据集进行特征选择，然后再训练学习器。其不依赖具体模型，每次增加最有信息量的特征或者删除最没信息量的特征。形象地说，即对每一维的特征“打

分”，即给每一维的特征赋予权重，那么权重就代表着该维特征的重要性，然后依据权重排序。

- 包裹式 (Wrapper)：从初始特征集合中不断的选择特征子集，训练学习器，用后续机器学习模型的准确率来评价一个特征子集，每次增加对后续最有用的特征，反之删除，直到选择出最佳的子集。此过程将机器学习模型包裹到特征选择的内部，将子集的选择看作是一个搜索寻优问题，生成不同的组合，对组合进行评价，再与其他组合进行比较，这样就将子集的选择看作是一个优化问题。

- ℓ_1 正则化 (Embedded)：通过加入 ℓ_1 范数 (把各个参数的绝对值相加) 来防止过拟合，由于 ℓ_1 正则化会导致稀疏的特征，在模型既定的情况下学习出对提高模型准确性最好的属性。这个方法能够在确定模型的过程中，挑选出那些对模型的训练有重要意义的属性。例：假设线性回归模型判断病症。(补充：稀疏特征是那些在数据集中不连续出现的特征，并且大多数值为零；密集特征是那些在数据集中经常或有规律地出现的特征，并且大多数值都是非零的。)

1.3.2 特征抽取 (Feature Extraction)

其方法主要是通过属性间的关系，如组合不同的属性得到新的属性，其**改变了**原来的特征空间，构造一个新的特征空间。以线性投影为例：

$$\mathbf{x}' = P\mathbf{x} \tag{2}$$

- $\mathbf{x}$：原始特征向量 $\mathbf{x} \in \mathbb{R}^d$
- $P \in \mathbb{R}^{k\times b}$：映射矩阵
- $\mathbf{x}'$：经过线性投影后得到在新空间的特征向量。

特征提取又可以分为监督和无监督的方法。

- 线性判别分析分析 (LDA-Linear Discriminant Analysis)：监督学习的一种。LDA 算法目标：最大化类间均值，最小化类内方差。即将数据投影在低维度上，并且投影后同种类别数据的投影点尽可能的接近，不同类别数据的投影点的中心点尽可能的远。

- 主成分分析 (PCA-Principal Components Analysis): 无监督学习的一种。算法的大体思想：在减少需要分析的指标同时，尽量减少原指标包含信息的损失，以达到对所收集数据进行全面分析的目的。由于各变量之间存在一定的相关关系，因此可以考虑将关系紧密的变量变成尽可能少的新变量，使这些新变量两两不相关，那么就可以用较少的综合指标分别代表存在于各个变量中的各类信息。

传统机器学习模型主要关注最后一步即构建预测函数，前三步需要人工干预完成，因此很多模式识别变成特征工程问题 (Feature Engineering)。

1.4 表示学习 (含书 2.6)

1. 定义：为了提高机器学习的效率，需要把特征进行有效的表示。一种可以自动学习出这种有效的特征并且提高最终学习的性能的算法就叫做表示学习。

2. 关键：解决语义鸿沟的问题。即解决输入数据的底层特征和高层语义之间的差异性的问题。例如给出关于“车”的一些图，不同图片车的形状颜色都有差异，而人都能抽象出共同的高层语义“车”，而机器对输入数据的理解是建立在底层特征上的，因此面临的每张图片的像素差异都会很大，这给机器分类识别造成困难。因此一个完全基于特层特征的模型会对机器预测模型的能力要求过高。因此表示学习追求实现一个“好的表示”的目标，其大体包括以下三个方面：

   - 多：表示能力强。即同样大小的向量可以承载更多的信息。
   - 简单：好的表示可以使得后续学习任务变得简单，那么就需要包括更高层的语义信息。
   - 一般性：希望学习到的表示具有一定的普适性，可以迁移到类似的任务中。

   传统机器学习中的两种表示：局部表示 (Local representation) 和分布式表示 (Distributed reprentation)

   - 局部表示 (离散表示/符号表示)，通常以 one-hot 向量的形式，即一种易于让机器理解的二进制的表示，只有激活点维的值为 1，其余都是 0。(handwritten digits desired outputs)
   - 分布表示：通过矩阵或神经网络降维 (chatgpt) 将语义分散存储到向量的各个维度中，实现降维稠密向量。

   以颜色表示为例，此处除了基本的几种颜色，对颜色进行了进一步的细分如“中国红”,“天蓝色”,“咖啡色”,“琥珀色”等。

   - 局部表示：假设所有颜色的名字构成一个词表 V, 大小为 $\|V\|$, 即可以用一个有 $\|V\|$ 维的 one-hot 向量来表示一种颜色。第 i 种颜色的 one-hot 向量中，第 i 维的值为 1，其余为 0。然而由于其维数高且不能拓展因此每多一种颜色就需要增加一个维度；另外，不能展示不同颜色之间的相似度。
   - 分布表示：用 RGB 值来表示颜色，不同颜色对应到 R(Red), G(Green), B(Blue) 三维空间中的一个点，此时仅需一个三维的稠密向量就可以表示所有的颜色，也易于增添新的颜色，计算不同颜色之间的相似度，因此表示能力得到增强。

颜色	局部表示	分布式表示
琥珀色	$[1,0,0,0]^T$	$[1.00,0.75,0.00]^T$
天蓝色	$[0,1,0,0]^T$	$[0.00,0.5,1.00]^T$
中国红	$[0,0,1,0]^T$	$[0.67,0.22,0.12]^T$
咖啡色	$[0,0,0,1]^T$	$[0.44,0.31,0.22]^T$

   - 嵌入 (embedding): 可用神经网络将高维的局部表示空间 $\mathbb{R}^{|V|}$ 映射到一个低维的分布式表示空间 $\mathbb{R}^d$，$d \ll |V|$。在这个空间中，特征不是坐标轴上的点而是分散在整个低维的空间中，这个将一个度量空间中的一些对象映射到**低维**度量空间并且保持不同对象之间的拓扑关系不变的过程即嵌入, 如自然语言中词的分布式表示也叫词嵌入。

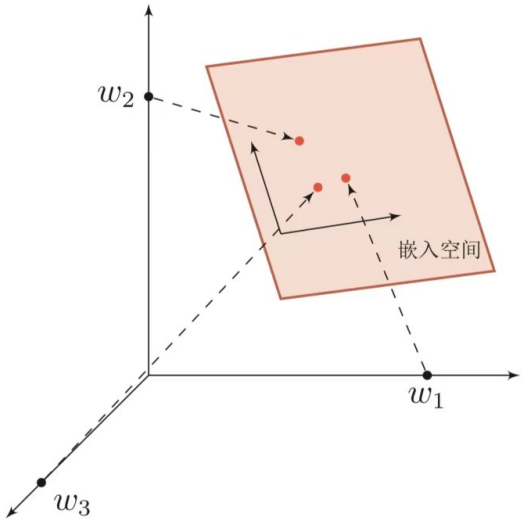


图 2: one-hot and embedding

3. 总结：要学习到一种好的表示 (通常为分布式表示)，需要从底层特征开始，经过多步非线性转换 (连续多次线性转换等价于一次线性转换) 才能得到。一个深层结构的特点是可以增加特征的重用性，从而指数级地增加表示能力。因此，表示学习的关键是构建有一定**深度**的多层次特征表示。

1.5 深度学习 (含书 2.6.2)

1. 定义：所谓“深度”，即对模型进行非线性转换的次数。深度学习处理数据即通过多层的特征转换，把原始数据转换为更高层、更抽象的表示，从而避免了特征工程。深度学习 (Deep Learning) 的主要目的即从数据中自动学习到有效的特征表示。

2. 处理数据的流程：原始数据 $\longrightarrow$ 表示学习 (底层特征、中层特征、高层特征) $\longrightarrow$ 预测 $\longrightarrow$ 结果

3. 关键: 解决贡献度分配问题 (Credit Assignment Problem, CAP)。即一个系统中不同组件 (component) 对最终系统输出的结果的贡献或者影响。(例子：围棋，每步棋对结果成败的贡献) 从某种意义上，深度学习可以视为一种强化学习 (RL), 每个内部组件不能得到监督信息，需要整个模型的最终监督信息 (奖励) 得到，并且有一定的延时性。目前，其采用的主要模型是神经网络模型由于其可用误差反向传播算法，可以比较好地解决贡献度分配问题。(超过一层都可以被视为 DL 模型)

4. 端到端学习 (End-to-End Learning): 传统机器学习需要将一个任务的输入输出人为切割为很多个子模块或者多个阶段，每个子模块分开学习。这种方式不仅仅会面临每个模块需要单独进行优化的问题，而且可能会导致错误传播，即前一步的错误会对后续的模型造成较大的影响，增加了机器学习在实际应用的难度；而端到端学习不需在学习过程分模块或者分阶段训练，直接优化总体的任务目标，不需要明确不同模块功能或者人为干预训练数据为“输入-输出”对的形式，无需其他信息，但是核心都是要解决“贡献度分配”的问题。(神经网络、深度学习也是)

2 机器学习 (或模式识别 Pattern Recognizing, PR)

2.1 基本概念

以挑选芒果为例：

1. 特征 (feature): 颜色、大小、形状、产地、品牌等。
2. 特征向量 (feature vector): 用一个 d 维向量 $x = [x_1, x_2, ..., x_d]^T$ 表示一个芒果的所有特征构成的向量，称为特征向量，一维表示一个特征。(非数值型需要转换)
3. 标签 (label)：连续值 (对芒果的甜度、水分以及成熟度的综合打分) 或者离散值 (好、坏两类标签)。
4. 样本 (sample/instance)：一个标记好特征以及标签的芒果可以看作是一个样本。
5. 数据集 (data set)：一组样本构成的集合。
   - 训练集 (training set/training sample): 训练模型
   - 测试集（test set/test sample): 检验模型的好坏

假设训练集由 N 个样本组成，每个样本独立同分布 (Identically and Independently Distributed, IID) 即独立从相同的数据分布中抽取出来, 记为

$$\mathcal{D} = \left\{(X^{(1)}, y^{(1)}), (X^{(2)}, y^{(2)}), ..., (X^{(N)}, y^{(N)})\right\} \tag{3}$$

对给定训练集 $\mathcal{D}$, 希望能让计算机找到一个函数 $f(\mathbf{x}, \theta)$ 来建立每个样本特性向量 $\mathbf{x}$ 和标签 y 之间的映射。对 $\mathbf{x}$ 来说，可以通过决策函数来预测其标签的值：

$$\hat{y} = f(\mathbf{x}, \theta) \tag{4}$$

或者标签的条件概率

$$p(y|\mathbf{x}) = f_y(\mathbf{x}, \theta) \tag{5}$$

其中 θ 即可学习的参数。而机器学习或者训练 (learning/training) 的过程，即通过一种学习算法 $\mathcal{A}$ (learning algorithm) 在训练集上找到一组理想的参数 θ^*，使得函数 $f(\mathbf{x}, \theta^*)$ 可以近似真实的映射关系，函数 $f(\mathbf{x}, \theta)$ 即模型 (model)。之后，可以用函数 $f(\mathbf{x},\theta^*)$ 来

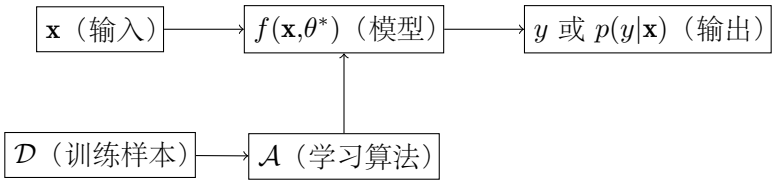


图 3: 机器学习过程示例

预测芒果的好坏。为了评价的公正性，需要合理评估模型的准确性，独立同分布地抽取一组新样本作为测试集 $\mathcal{D}'$ ，并在测试集所有样本中测试其准确率 (Accuracy)。

$$Acc(f(\mathbf{x}, \theta^*)) = \frac{1}{|\mathcal{D}|'} \sum_{(\mathbf{x},y)\in\mathcal{D}'} I(f(\mathbf{x}, \theta^*) = y) \tag{6}$$

$|\mathcal{D}|'$ 指测试集的大小，$I(\cdot)$ 指指示函数。

- 在计算机算法中，指的是当输入为 True 的时候，输出为 1，输入为 False 的时候，输出为 0。对于此函数，$f(\mathbf{x}, \theta^*) = y$ 时输出为 1，$f(\mathbf{x}, \theta^*) \neq y$ 时输出为 0。

2.2 三个基本要素

2.2.1 模型

一个机器学习需要确定输入空间或者特征空间 $\mathcal{X}$ 和输出空间 $\mathcal{Y}$。不同机器学习任务的主要区别在 $\mathcal{Y}$。比如：

- 两类分类问题：$\mathcal{Y} = \{+1, -1\}$
- $\mathbf{C}$ 类分类问题：$\mathcal{Y} = \{1, 2, ..., \mathbf{C}\}$
- 回归问题：$\mathcal{Y} = \mathbb{R}$

输入空间 $\mathcal{X}$ 和输出空间 $\mathcal{Y}$ 构成了一个样本空间。对于样本空间中的样本 $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$, 假定存在一个未知的真实映射函数 $g : \mathcal{X} \longrightarrow \mathcal{Y}$ 使得

$$y = g(\mathbf{x}) \tag{7}$$

或者真实条件概率分布：

$$p_r(y|\mathbf{x}) \tag{8}$$

机器学习的目标即找到模型来近似真实映射函数 $g(\mathbf{x})$ 或者条件概率分布 $p_r(y|\mathbf{x})$。由于不确定真实的具体形式，只能根据经验来假设函数集合 $\mathcal{F}$, 即假设空间 (hypothesis space)，通过观测其在训练集 $\mathcal{D}$ 上的特性，从中选择一个理想的假设使得 $f^* \in \mathcal{F}$。$\mathcal{F}$ 通常是一个参数化的函数族

$$\mathcal{F} = \{f(\mathbf{x}, \theta)|\theta \in \mathbb{R}^m\} \tag{9}$$

$f(\mathbf{x}, \theta)$ 为 $\mathcal{F}$ 中的模型, θ 为一组可学习参数，m 为参数的数量。常见的 $\mathcal{F}$ 分为线性和非线性，对应的模型 f 称为线性模型和非线性模型。

- 线性模型

  $$f(\mathbf{x}, \theta) = \mathbf{w}^T\mathbf{x} + b \tag{10}$$

  线性模型的 $\mathcal{F}$ 为一个参数化的函数族，θ 包含了权重向量 $\mathbf{w}$ 和偏置 b

- 非线性模型: 广义的非线性模型可写成多个非线性基函数 $\phi(\mathbf{x})$ 的线性组合：

  $$f(\mathbf{x}, \theta) = \mathbf{w}^T\phi(\mathbf{x}) + b \tag{11}$$

  - θ: 包含了权重向量 $\mathbf{w}$ 和偏置 b
  - $\phi(\mathbf{x}) = [\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), ..., \phi_K(\mathbf{x})]^T$ 为 K 个非线性基函数组成的向量

  如果 $\phi(\mathbf{x})$ 本身为可学习的基函数，比如：

  $$\phi_K(\mathbf{x}) = h(\mathbf{w}_k^T\phi'(\mathbf{x}) + b_k), \forall 1 \leq k \leq K \tag{12}$$

  - $h(\cdot)$；非线性函数
  - $\phi'(\mathbf{x})$：另一组基函数
  - $\mathbf{w}_k$ 和 b_k：可学习的参数
  - $f(\mathbf{x}, \theta)$：等价于神经网络模型 (h-activate function)

2.2.2 学习准则

每个样本 $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$ 是按照某个随机分布 $p_r(y|\mathbf{x})$ 独立地随机产生的，因此这里要求 $p_r(y|\mathbf{x})$ 固定，不随时间的变化，否则将无法学习。模型越好，说明模型 $f(\mathbf{x}, \theta^*)$ 应该在所有 $(\mathbf{x}, y)$ 的可能取值上都与真实映射函数 $y = g(\mathbf{x})$ 一致，即误差应该尽可能小：

$$|f(\mathbf{x}, \theta^*) - y| < \epsilon, \forall (\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y} \tag{13}$$

或者与真实条件分布概率一致：

$$|f(\mathbf{x}, \theta^*) - p(y|\mathbf{x})| < \epsilon, \forall (\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y} \tag{14}$$

- ϵ：一个很小的正数
- $f(\mathbf{x}, \theta^*)$：模型预测的条件概率分布中，y 对应的概率

可以用期望风险 (Expected Risk) $\mathcal{R}(\theta)$ 来衡量模型的好坏：

$$\mathcal{R}(\theta) = \mathbb{E}_{(\mathbf{x},y)\sim p_r(\mathbf{x},y)}[\mathcal{L}(y, f(\mathbf{x}, \theta))] \tag{15}$$

- $p_r(\mathbf{x}, y)$：真实的数据分布
- $\mathcal{L}(y, f(\mathbf{x}, \theta))$：损失函数，一个非负实数函数，量化预测模型与真实标签 (真实映射函数) 之间的差异。

损失函数的几种类别：

1. **0-1 损失函数 (0-1 Loss Function)** 最为直观的一种：

$$\mathcal{L}(y, f(\mathbf{x}, \theta)) = I(y \neq f(\mathbf{x}, \theta)) = \begin{cases} 0 & if y = f(\mathbf{x}, \theta) \\ 1 & if y \neq f(\mathbf{x}, \theta) \end{cases} \tag{16}$$

   虽然 **0-1** 能客观评价模型好坏，但数学性质不是很好：不连续且导数为 0，难以优化，因此常用连续且可微的函数替代。

2. **平方损失函数 (Quadratic Loss Function)**：经常用在预测标签为实数值的任务中：

$$\mathcal{L}(y, f(\mathbf{x}, \theta)) = \frac{1}{2}(y - f(\mathbf{x}, \theta))^2 \tag{17}$$

   (e.g. 神经网络手写数字识别) 平方损失函数一般不适合用于分类问题。

3. **交叉熵损失函数 (Cross-Entropy Loss Function)**：一般用于分类问题。

   - **模型**：假设样本的标签 $y \in \{1, ...C\}$ 为离散的类别，模型 $f(\mathbf{x}, \theta) \in [0, 1]^C$ 的输出为类别标签的条件概率分布即

$$p(y = c|\mathbf{x}, \theta) = f_c(\mathbf{x}, \theta) \tag{18}$$

     并满足

$$f_c(\mathbf{x}, \theta) \in [0, 1], \sum_{c=1}^{C} = f_c(\mathbf{x}, \theta) = 1 \tag{19}$$

- **真实**: 假设用一个 C 维的 one-hot 向量 y 来表示样本标签。假设样本的标签为 k，那么第 k 维的值是 1，其余是 0。标签向量 $\mathbf{y}$ 可以视为样本标签的真实概率分布，即第 c 维 $(y_c, 1 \leq c \leq C)$ 是类别为 c 的真实概率。假设样本的类别为 k 则它属于第 k 类的概率为 1，其他类的概率为 0。

对于两个概率分布，用交叉熵来衡量它们的差异。即标签的真实分布 $\mathbf{y}$ 和模型预测分布 $f(\mathbf{x}, \theta)$ 之间的交叉熵为：

$$\mathcal{L}(y, f(\mathbf{x}, \theta)) = -\sum_{c=1}^{C} y_c \log f_c(\mathbf{x}, \theta) \tag{20}$$

(补充知识：自信息和熵)

- 自信息：
  假设对一个随机变量 X (取值集合为 $\mathcal{X}$, 概率分布为 $p(x)$，$x \in \mathcal{X}$) 进行编码，自信息 $I(x)$ 是变量 $X = x$ 的信息量或者换编码长度，定义为：

  $$I(x) = -\log\left(p(x)\right) \tag{21}$$

  自信息即一个随机事件所包含的信息量，因此事件发生概率越小，事件发生时所包含的信息量越大，自信息就越大。假设自信息和随机事件 E 的概率存在这样的函数关系：

  $$I(E) = f(P(E))$$

  如果对于事件 E 是由两个独立事件 A 和 B 构成的，那么易得：

  $$I(E) = I(A \cap B) = I(A) + I(B)$$

  $$P(E) = P(A \cap B) = P(A) \cdot P(B)$$

  因此，容易得到：

  $$f(P(E)) = f(P(A)) + f(P(B)) = f(P(A)) \cdot f(P(B))$$

  而满足 $f(x \cdot y) = f(x) + f(y)$ 形式的函数便是对数函数 $f(x) = k\log(x)$ 由于 $I(x) \geq 0$ 且事件的概率 $P(x) \in [0, 1]$ 而且在熵的定义中，对数的底可以是 2、e、10，因此 $\log(x) \leq 0$ 因此 $k < 0$。

- 熵: 在信息论中，熵用来衡量一个随机事件的不确定性，定义为随机变量 X 的平均编码长度：

  $$H(X) = \mathbb{E}_X[I(x)] = \mathbb{E}_X[-\log\left(p(x)\right)] = -\sum_{x \in \mathcal{X}} p(x) \log p(x) \tag{22}$$

  (最后一步：$\sum_i p(x_i) I(x_i)$) 其中当 $p(x_i) = 0$ 时，定义 $0 \log 0 = 0$。熵越高，自信息的数学期望就越高，随机变量的信息就越多，反之亦然。因此对于一个确定的信息，其熵为 0，信息量也为 0。而如果概率分布为一个均匀分布，则熵最大。

- 交叉熵 (Cross Entropy): 对应分布为 $p(x)$ 的随机变量，熵 $H(p)$ 表示其最优编码长度。因此交叉熵是按照概率分布 q 的最优编码对真实分布为 p 的信息进行编码的长度:

$$H(p,q) = \mathcal{E}[-\log q(x)] = -\sum_x p(x)\log q(x) \tag{23}$$

  在给定 p 的情况下，p，q 越接近，交叉熵越小，反之越大。

以一个三类分类问题看等式 (20) 的应用：一个样本的标签向量 (真实)$\mathbf{y} = [0,0,1]^T$，模型预测的标签分布为 $f(\mathbf{x},\theta) = [0.3,0.3,0.4]^T$, 它们的交叉熵：

$$\mathcal{L}(\theta) = -(0\times\log(0.3) + 0\times\log(0.3) + 1\times\log(0.4) = -\log 0.4 \tag{24}$$

由于 $\mathbf{y}$ 为 one-hot 向量 (仅有一类正确)，则：

$$\mathcal{L}(y, f(\mathbf{x},\theta)) = -\log f_y(x,\theta) \tag{25}$$

其中，$f_y(x,\theta)$ 可以看做真实类别 y 的似然函数。因此交叉熵损失函数也是负对数似然损失函数 (Negative Log-likelihood Function)。补充知识：

- 似然性即某些参数为特定值的可能性，可以被视作条件概率的“逆反”。二者关注的变量不同。概率：已知事物的参数 $\longrightarrow$ 预测结果；似然性：已知某些观测结果 $\longrightarrow$ 对事物性质的参数进行估值。

  $$\mathcal{L}(B/A) = P(A/B = b)$$

  以投掷硬币为例：投掷一次硬币，正面朝上的概率用 p_H 来代表，它就是我们这个例子的参数，而我们用事件 A 表示投掷三次硬币，两次正面向上。因此如果 $p_H = 0.5$:

  $$P(A/p_H = 0.5) = 3\times p_H^2\times(1-p_H) = 0.375$$

  如果 $p_H = 0.6$

  $$P(A/p_H = 0.6) = 0.472$$

  所以当我们投掷了三次硬币并且看到了两次正面，可以猜测 p_H 比较可能是 0.6 而非 0.5。若单独看 0.375 这个数字或 0.432 这个数字是没有意义的，因为似然性并不是概率，并不是一定介于 0 到 1 之间，似然性是用在把各种可能的 p_H 值放在一起比较，来得知哪个的可能性比较高。而似然函数除了用来计算似然性外，则是用来了解当参数改变时，似然性怎么变化，用来寻找最大可能性的 p_H 值会是多少。因此对同一个似然函数，其所代表的模型中，某项参数值具有多种可能，但如果存在一个参数值，使得概似函数值达到最大的话，那么这个值就是该项参数最为“合理”的参数值。

- 最大似然估计：似然函数取得最大值表示相应的参数能够使得统计模型最为合理。做法是：首先选取似然函数，整理之后求最大值点。实际应用中一般会取似然函数的对数作为求最大值的函数，这样求出的最大值点和直接求最大值点得到的结果是相同的。但是似然函数的最大值点不一定唯一，也不一定存在。

4. Hinge 损失函数对于两类分类问题，假设 y 和 $f(x,\theta)=\{-1,+1\}$，则 Hinge 损失函数：

$$\mathcal{L}(y,f(x,\theta))=max(0,1-yf(x,\theta)) \tag{26}$$

分类正确，损失为 0；分类错误，损失为 2

损失函数用来评估模型单次预测的好坏，而风险函数则量平均意义下模型的好坏。下面将展示**风险最小化准则**：一个好的模型应该有比较小的期望错误，但由于缺乏真实的数据分布和映射函数，无法计算期望风险 $\mathcal{R}(\theta;\mathbf{x},y)$。然而对给定训练集 $\mathcal{D}$ 可以计算其经验风险 (Empirical Risk), 即在训练集上的平均损失。

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta)=\frac{1}{N}\sum_{n=1}^{N}\mathcal{L}(y^{(n)},f(\mathbf{x}^{(n)},\theta)) \tag{27}$$

因此，一个切实的学习准则: 找到一组参数 θ^* 使得经验风险最小：

$$\theta^*=\underset{\theta}{argmin}\,\mathcal{R}_{\mathcal{D}}^{emp}(\theta) \tag{28}$$

即经验风险 (Empirical Risk Minimization) 最小化准则 (argmin- 使后面式子达到最小的时候的 θ 的取值)

- 过拟合：当训练集大小趋于无穷大时，经验风险应该越接近于期望风险，然而通常情况下，训练样本有限且包含一定的噪声数据，不能很好地反应全部数据的真实分布。经验风险最小化原则容易导致模型在训练集上错误率低，然而在未知数据上错误率高，即过拟合 (Overfitting)。更精确的解释：给定一个假设空间 $\mathcal{F}$, 一个假设 f 属于 $\mathcal{F}$, 如果存在其他的假设 f' 也属于 $\mathcal{F}$，使得在训练集上的损失前者比后者小，然而在整个空间相反，那么假设 f 过度拟合了训练数据。过度拟合问题除了训练数据少和噪声的原因还有模型能力强，因此可以在经验风险最小化的基础上引入参数的正则化 (regulation)，来限制模型能力即结构风险最小化准则 (Structure Risk Minimization, SRM) 准则。

  $$\theta^*=\underset{\theta}{argmin}\,\mathcal{R}_{\mathcal{D}}^{struct}(\theta)=\underset{\theta}{argmin}\,\mathcal{R}_{\mathcal{D}}^{emp}(\theta)+\frac{1}{2}\lambda\left\|\theta\right\|^2=\underset{\theta}{argmin}\,\frac{1}{N}\sum_{n=1}^{N}\mathcal{L}(y^{(n)},f(x^{(n)},\theta))+\frac{1}{2}\lambda\left\|\theta\right\|^2 \tag{29}$$

  其中 $\|\theta\|$ 是 ℓ_2 范数的正则化项，可以用来减少参数空间，防止过度拟合；λ 用来控制正则化的强度。因此机器学习不仅仅是拟合训练集上的数据，还要使泛化错误最低，可以看作是从一个有限、高维、有噪声的数据上得到更一般规律的泛化问题。

- 欠拟合 (underfitting)：模型不能很好地拟合训练数据，在训练集上的错误率较高，一般是由于模型能力导致的。

如图所示 (4)

2.2.3 优化算法

机器学习的训练过程就是最优化问题的求解过程。优化可以分为参数优化和超参数优化：

- 参数：可以通过优化算法进行学习

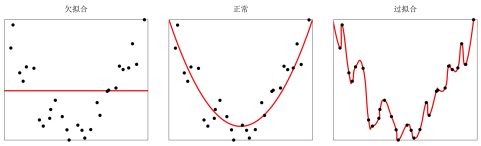


图 4: Overfitting and underfitting

- 超参数：用来定义模型结构或者优化策略，如梯度下降的步长，神经网络的层数，聚类算法的类别的个数，支持向量机的核函数等，通常是按照人的经验设定，或者通过搜索方法对一组超参数组合进行不断试错调整。

1. 梯度下降法：通过迭代的方式来计算训练集上风险函数的最小值。

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial \mathcal{R}_{\mathcal{D}}(\theta)}{\partial \theta} \tag{30}$$

即

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{1}{N} \sum_{n=1}^{N} \frac{\partial \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta))}{\partial \theta} \tag{31}$$

θ_t 为第 t 此迭代时的参数值，α 为搜素步长 (or learning rate)。

2. 提前停止：针对梯度下降法的防止过拟合的办法 (除了加正则化项)。由于过拟合的原因，在训练集上收敛的参数不一定在测试集上最优。因此通常还需要一个验证集 (validation set) 来测试模型是否在验证集上最优，每次迭代，把新得到的模型 $f(\mathbf{x}, \theta)$ 在测试集上测试，并且计算错误率，错误率不再下降时，停止迭代，即提前停止 (early stop)。如图 (5) 所示：

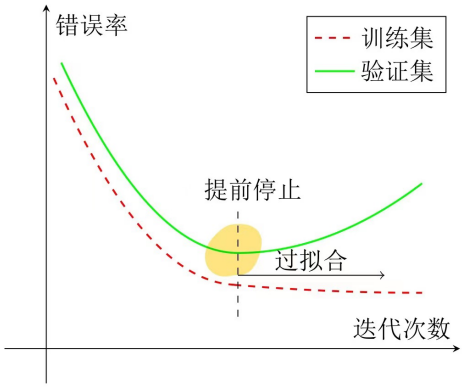


图 5: Early Stop

3. 随机梯度下降法 (SGD) 公式 (30) 的方式为批量梯度下降法 (Batch Gradient Descent, BGD) 即从真实数据分布中采集 N 个样本，每次迭代计算每个样本上损失函数的梯度并求和，当 N 很大时，以经验风险的梯度来近似期望风险的梯度，然而如果训练集很大，那么计算很复杂且成本很高。为了优化，可以每次迭代只采集一个样本，计算其损失函数的梯度并随时更新，当经过足够次数的迭代，随机梯度下降可以收敛到局部最优

算法 2.1: 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，验证集 $\mathcal{V}$，学习率 α

随机初始化 θ;
repeat
  对训练集 $\mathcal{D}$ 中的样本随机重排序;
  for $n = 1 \cdots N$ **do**
    从训练集 $\mathcal{D}$ 中选取样本 $(\mathbf{x}^{(n)}, y^{(n)})$;
    // 更新参数
    $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; x^{(n)}, y^{(n)})}{\partial \theta}$;
  end
until 模型 $f(\mathbf{x}, \theta)$ 在验证集 $\mathcal{V}$ 上的错误率不再下降;
输出: θ

图 6: SGD

解。算法如图所示 (6) BGD 和 SGD 最大区别：前者迭代所有样本的平均损失函数，后者迭代单个样本的损失函数。SGD 相当于在 BGD 的基础上引入了随机噪声，当目标函数非凸时，反而可以使其逃离局部最优点。(局部最优全局最优如图 (7) 所示，loss 曲面的鞍点如图 8所示) (正因为梯度下降法容易收敛到局部最优，所以大家几乎从来不用梯

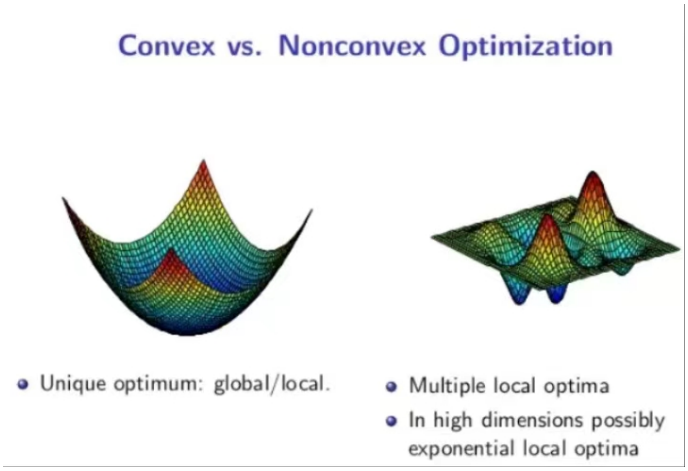


图 7: convex optimization vs non-convex

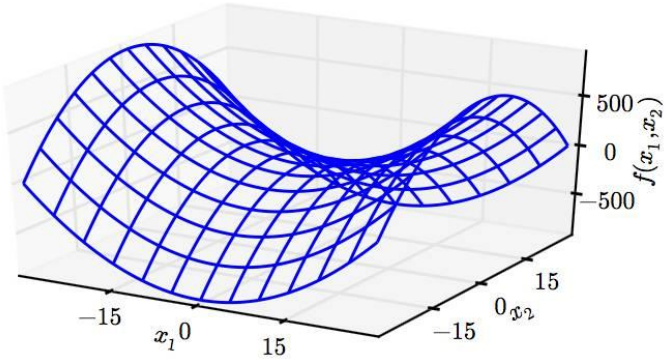


图 8: Loss surface (saddle)

度下降做非凸优化，包括训练神经网络；正因为随机梯度下降法容易逃离鞍点和泛化不

好的 minima（主要是 sharp minima），所以随机梯度下降（SGD）和它的变种（比如 Momentun、Adam）才是训练神经网络最流行的方法。鞍点（saddle points）和泛化不好的最优点（bad/sharp minima）在深度学习里的确是广泛存在的。但这是神经网络复杂的 loss landscape 带来的问题，而不是优化器带来的问题。反而，优化器是来解决问题的。正因为 saddle points 和 bad minima 太多了，所以才需要随机优化。）

4. 小批量梯度下降法 (Mini-Batch Gradient Descent): SGD 无法利用计算机的并行计算的能力，小批量梯度下降法是批量梯度下降和随机梯度下降的折中，即每次随机选取一小部分训练样本计算梯度并更新参数。假设第 t 次迭代时，随机选取一个包含 k 个样本的子集 $\mathcal{I}_t$，计算此子集上的样本损失函数的梯度并随时进行更新：

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{1}{k} \sum_{(\mathbf{x},y)\in\mathcal{I}_t} \frac{\partial \mathcal{L}(y, f(\mathbf{x},\theta))}{\partial \theta} \tag{32}$$

此方法收敛快，且计算开销小。

2.3 简单案例：线性回归 (Linear Regression)

2.3.1 基础概念

线性回归是一种对自变量和因变量之间的关系进行建模的回归分析。自变量数量为 1，则是简单回归，大于 1 是多元回归。本小节通过此模型来了解机器学习的一般过程和不同的学习准则即经验风险最小化，结构风险最小化，最大似然估计，最大后验估计之间的关系。从机器学习的角度：

- 自变量：样本的特性向量 $x \in \mathbb{R}^d$, 每一维对应一个自变量
- 因变量：标签 $y \in \mathbb{R}$ 是连续值
- 假设空间 $\mathcal{F}$ 是一组参数化的线性函数：

$$f(\mathbf{x};\mathbf{w},b) = \mathbf{w}^T\mathbf{x} + b \tag{33}$$

- 简化写法：函数 $f(\mathbf{x};\mathbf{w},b) \in \mathbb{R}$ 也称为线性模型。简化写法：

$$f(\mathbf{x};\hat{\mathbf{w}}) = \hat{\mathbf{w}}^T\hat{\mathbf{x}} \tag{34}$$

$\hat{\mathbf{w}}$ 为增广权重向量；$\hat{\mathbf{x}}$ 为增广特征向量。如图所示 (9)：因此线性回归的模型可以简写为 $f(\mathbf{x};\mathbf{w}) = \mathbf{w}^T\mathbf{x}$。

2.3.2 参数学习

对给定一个训练集 $\mathcal{D}$ 希望能学习一个最优的线性回归的模型参数 $\mathbf{w}$。下面将介绍四种不同的参数估计方法：经验风险最小化、结构风险最小化、最大似然估计、最大后验估计。前二者是从函数关系角度即样本的特征向量和标签间存在某种未知的函数关系 $y = g(x)$。后二者是站在概率的角度，即条件概率 $p(y|\mathbf{x})$ 服从某个未知分布。

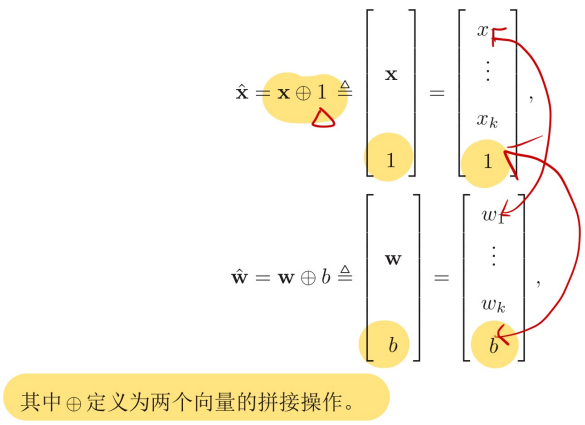


图 9: 增广特征向量和增广权重向量

1. 经验风险最小化

由于线性回归的标签 y 和模型输出都为连续的实数值，因而适合用**平方损失函数**来衡量真实标签和预测标签的差异。据经验风险最小化的原则，训练集 $\mathcal{D}$ 上的经验风险定义为：

$$\mathcal{R}(\mathbf{w}) = \sum_{n=1}^{N} \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, w)) = \frac{1}{2} \sum_{n=1}^{N} (y^{(n)} - \mathbf{w}^T \mathbf{x}^{(n)})^2 \tag{35}$$

表示成向量的形式

$$\mathcal{R}(\mathbf{w}) = \frac{1}{2} \left\| \mathbf{y} - \mathbf{X}^T \mathbf{w} \right\|^2 \tag{36}$$

其中，$\mathbf{y} \in \mathbb{R}^N$ 是由每个样本的真实标签 $y(1), ..., y(N)$ 组成的列向量，$\mathbf{X} \in \mathbb{R}^{(d+1) \times N}$ 是所有输入 x 组成的矩阵。

$$X = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & ... & x_1^{(N)} \\ ... & ... & ... & ... \\ x_d^{(1)} & x_d^{(2)} & ... & x_d^{(N)} \\ 1 & 1 & 1 & 1 \end{bmatrix} \tag{37}$$

风险函数 $\mathcal{R}(\mathbf{w})$ 是关于 w 的凸函数，其对 w 的偏导数：

$$\frac{\partial \mathcal{R}(\mathbf{w})}{\partial w} = \frac{1}{2} \frac{\partial \left\| \mathbf{y} - \mathbf{X}^T \mathbf{w} \right\|^2}{\partial \mathbf{w}} = -\mathbf{X}(\mathbf{y} - \mathbf{X}^T \mathbf{w}) \tag{38}$$

令 $\frac{\partial \mathcal{R}(\mathbf{w})}{\partial w} = 0$，得到的最优的参数 w^* 为：

$$w^* = (\mathbf{X}\mathbf{X}^T)^{-1} \mathbf{X}\mathbf{y} \tag{39}$$

即

$$w^* = (\sum_{n=1}^{N} x^{(n)} (x^{(n)})^T)^{-1} (\sum_{n=1}^{N} x^{(n)} y^{(n)}) \tag{40}$$

此方法即最小二乘法估计 (Least Square Estimation, LSE)，示例如图 (10)。在最小二乘法中，$\mathbf{X}\mathbf{X}^T \in \mathbb{R}^{(d+1) \times (d+1)}$ 必须存在逆矩阵 $\longrightarrow \mathbf{X}\mathbf{X}^T$ 满秩 $(d+1)$，即每一行特征线性不相关。(否则 $det(\mathbf{X}\mathbf{X}^T) = 0$ 不满秩。) 一种常见的不可逆的情况即 $N < d+1$，则 $\mathbf{X}\mathbf{X}^T$

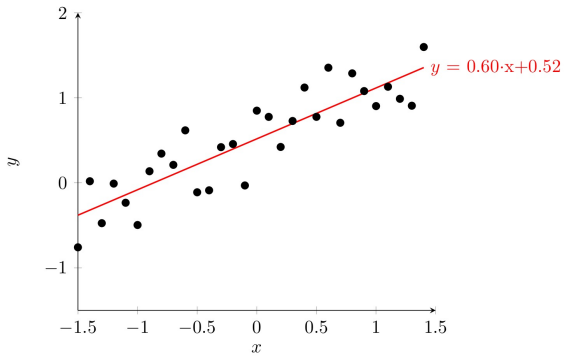


图 10: example of LSE

秩为 N(证)，则会存在很多解 w^* 使得 $\mathcal{R}(\mathbf{w}^*) = 0$。因此当其不可逆时可用主成分分析法消除不同变量之间的相关性，然后采用最小二乘法或者梯度下降法求解。$w_0 = 0$，初始化进而通过此公式进行迭代：

$$w' = w + \alpha \mathbf{X}(\mathbf{y} - \mathbf{X}^T\mathbf{w}) \tag{41}$$

这种方法也称为最小均方误差 (Least Mean Square) 算法。

2. 结构风险最小化
最小二乘法估计要求各个特征独立，保证 $\mathbf{X}\mathbf{X}^T$ 可逆，但即使其可逆，特征之间较大的共线性 (multicollinearity) 会使得 $\mathbf{X}\mathbf{X}^T$ 的逆在数值上无法准确计算，数据集上的小的扰动会导致 $\mathbf{X}\mathbf{X}^T$ 的逆发生大的改变使得最小二乘法的估计变得不稳定。因此，岭回归 (Ridge Regression) 的方法被引入，即给 $\mathbf{X}\mathbf{X}^T$ 的对角线元素加上常数 λI 使得 $(\mathbf{X}\mathbf{X}^T + \lambda I)$ 的秩不为 0。最优参数 w^*：

$$w^* = (\mathbf{X}\mathbf{X}^T + \lambda I)^{-1}\mathbf{X}\mathbf{y} \tag{42}$$

λ 为预先设定的超参数，I 为单位矩阵。
岭回归的解 w^* 可被视为结构风险最小化准则下的最小二乘法估计 (证)。

$$\mathcal{R}(\mathbf{w}) = \frac{1}{2}\left\|\mathbf{y} - \mathbf{X}^T\mathbf{w}\right\|^2 + \frac{1}{2}\lambda\left\|\mathbf{w}\right\|^2 \tag{43}$$

$\lambda > 0$ 为正则化系数。

除此之外，线性回归还可以通过建模条件概率 $p(y|\mathbf{x})$ 的角度进行参数估计：

3. 最大似然估计
假设标签 y 为一个随机变量，其服从均值为 $f(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T\mathbf{x}$ 为中心，方差为 σ^2 的高斯分布：

$$p(y|\mathbf{x}, \mathbf{w}, \sigma) = \mathcal{N}(y|\mathbf{w}^T\mathbf{x}, \sigma^2) \tag{44}$$

即

$$p(y|\mathbf{x}, \mathbf{w}, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} exp(-\frac{(y - \mathbf{w}^T\mathbf{x})^2}{2\sigma^2}) \tag{45}$$

参数 $\mathbf{w}$ 在训练集上的似然函数 (likelihood-概率为固定参数 w 随机变量 x 的分布情况，而似然是描述已知随机变量 x, 不同的参数对其分布的影响) 为:

$$p(\mathbf{y}|X,\mathbf{w},\sigma)=\prod_{n=1}^{N}\mathcal{N}(y^{(n)}|\mathbf{w}^T\mathbf{x}^{(n)},\sigma^2) \tag{46}$$

其中 $\mathbf{y}$ 为所有样本标签组成的向量，$\mathbf{X}$ 为所有样本特征向量组成的矩阵。为了方便计算，对似然函数取对数得到对数似然函数 (log likelihood):

$$\log p(\mathbf{y}|X,\mathbf{w},\sigma)=\sum_{n=1}^{N}\log\mathcal{N}(y^{(n)}|\mathbf{w}^T\mathbf{x}^{(n)},\sigma^2) \tag{47}$$

最大似然估计 (Maximum Likelihood Estimate,MLE) 是找到一组参数 w 使得似然函数 $p(y|\mathbf{x},\mathbf{w},\sigma)$ 最大，等价于对数似然函数 $\log p(\mathbf{y}|X,\mathbf{w},\sigma)$ 最大。令 $\frac{\partial\log p(\mathbf{y}|X,\mathbf{w},\sigma)}{\partial\mathbf{w}}=0$ 得到 (证):

$$\mathbf{w}^{ML}=(\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{X}\mathbf{y} \tag{48}$$

即最大似然估计的解和最小二乘估计的解相同。

4. 最大后验估计

预备知识：贝叶斯公式 (结果 $\longrightarrow$ 原因):

$$P(A|B)=\frac{P(AB)}{P(B)}=\frac{P(B|A)P(A)}{P(B)} \tag{49}$$

- $P(A|B)$: 后验概率，相当于 $P(B|A)P(A)$ 在 $P(B)$ 的权重
- $P(A)$ 和 $P(B)$：先验概率 (经验/统计)，一般直接可以得到，后者有时通过全概率公式计算得到。
- $P(B|A)$：代表一种可能性

假设参数 w 为一个随机向量，并服从一个先验分布 $p(w|v)$。为简化，$p(w|v)$ 为各向同性的高斯分布:

$$p(w|v)=\mathcal{N}(\mathbf{w}|\mathbf{0},v^2I) \tag{50}$$

其中，v^2 为每一维上的方差。据贝叶斯公式得到参数 w 的后验分布概率 (posterior distribution-已知真实分布的结果推知参数取值的概率) 为:

$$p(\mathbf{w}|X,\mathbf{y},v,\sigma)=\frac{p(\mathbf{w},\mathbf{y}|X,v,\sigma)}{\sum_w p(\mathbf{w},\mathbf{y}|X,v,\sigma)} \tag{51}$$

由于分母相当于真实标签的条件概率分布 (2.2.2 " 要求 $rr(y|\mathbf{x})$ 固定的")，是一个固定值，且是与参数 w 无关的常量，因此 w 的后验概率分布可以看作正比于分子即:

$$p(\mathbf{w}|X,\mathbf{y},v,\sigma)\propto p(\mathbf{y}|X,\mathbf{w},\sigma)p(\mathbf{w}|v) \tag{52}$$

- $p(\mathbf{y}|X,\mathbf{w},\sigma)$：$w$ 的似然函数见公式 46
- $p(\mathbf{w}|v)$：w 的先验

这种统计参数 $\mathbf{w}$ 的后验概率分布的方法即贝叶斯估计 (Bayesian Estimation)，是一种统计推断问题，采用贝叶斯估计的线性回归即贝叶斯线性回归 (Bayesian Linear Regression)。(补充知识：连续随机变量 X 的概率分布一般用概率密度函数 (Probability Density Funciton, PDF) p(x) 来描述，p(x) 可积，并满足：

$$\int_{-\infty}^{+\infty} p(x)dx = 1, (p(x) \geq 0)$$

对给定概率密度函数 p(x)，可以计算随机变量落入某一个区间的概率，即 p(x) 反映了随机变量取落入 x 的非常小的邻近区间中的概率大小。) 此种方法只能得到参数在一个**区间**上的分布，我们目的是得到一个最优的参数值 (点估计)，即需要采用最大后验估计 (Maximum A Posteriori Estimation, MAP)。即最优参数为后验分布 $p(\mathbf{y}|X, \mathbf{w}, \sigma)$ 中概率密度最高的参数 w:

$$w^{MAP} = \underset{w}{argmax}\, p(\mathbf{y}|X, \mathbf{w}, \sigma)p(\mathbf{w}|v) \tag{53}$$

由于似然函数为 46中定义的高斯密度函数，那么后验分布的对数 (证)：

$$\log p(\mathbf{w}|X, \mathbf{y}, v, \sigma) \propto \log p(\mathbf{y}|X, \mathbf{w}, \sigma) + \log p(\mathbf{w}|v) \tag{54}$$

$$\propto -\frac{1}{2\sigma^2}\sum_{n=1}^{N}(y^{(n)} - \mathbf{w}^T\mathbf{x}^{(n)})^2 - \frac{1}{2v^2}\mathbf{w}^T\mathbf{w} \tag{55}$$

$$= -\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left\|\mathbf{y} - \mathbf{w}^T\mathbf{X}\right\|^2 - \frac{1}{2v^2}\mathbf{w}^T\mathbf{w} \tag{56}$$

即最大后验概率等价于平方损失结构的最小化，正则化系数 $\lambda = \frac{\sigma^2}{2v^2}$。

总结：最大似然估计和最大后验估计可以看作是频率学派和贝叶斯学派对需要估计的参数的不同解释：频率学派将模型参数 θ 看作是固定的常数，而贝叶斯学派将参数 θ 看作是随机变量。当 $v \longrightarrow \infty$ 时：

- 先验分布退化均匀分布，称为无信息先验 (证?)
- 最大后验估计退化为最大似然估计 (证)

(补充知识：**均匀分布**)
若 a, b 为有限数，那么 [a,b] 上的均匀分布 (Uniform Distribution) 的概率密度函数定义为：

$$p(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & x < a, x > b \end{cases} \tag{57}$$

	无先验	引入先验
平方误差（损失）	经验风险最小化	结构风险最小化
概率	最大似然估计 $\mathbf{w}^{ML} = (\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{X}\mathbf{y}$	最大后验估计 $\mathbf{w}^* = (\mathbf{X}\mathbf{X}^T + \lambda I)^{-1}\mathbf{X}\mathbf{y}$

2.4 偏差-方差分解 (Bias-Variance Decomposition)

机器学习需要防止欠拟合或者过拟合，即在模型的能力和复杂度之间进行平衡：拟合能力强，复杂度高，则可能导致过拟合；限制模型复杂度，降低其拟合能力，又可能欠拟合。偏差方差分解将其数值化，提高了一个较好的指导工具。

2.4.1 偏差、方差公式推导

以回归问题为例，假设样本的真实分布为 $p_r(\mathbf{x}, y)$，那么采用平方损失函数，模型 $f(x)$ 的期望错误：

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x},y)\sim p_r(\mathbf{x},y)}[(y - f(\mathbf{x}))^2] \tag{58}$$

则最优模型：

$$f^*(\mathbf{x}) = \mathbb{E}_{y\sim p_r(y|\mathbf{x})}[y] \tag{59}$$

$p_r(y|\mathbf{x})$ 为样本的真实条件分布。(证–补充知识:)

- 条件期望：一个实数随机变量的相对于一个条件概率分布的期望值。如果 X 和 Y 离散，那么 X 在给定 $Y = y$ 条件时的条件期望即 x 在 Y 值域的函数：

$$E(X|Y = y) = \sum_{x\in\mathcal{X}} P(X = x|Y = y) = \sum_{x\in\mathcal{X}} x\frac{P(X = x, Y = y)}{P(Y = y)} \tag{60}$$

  辨析：

  - $E(X)$：是一个数，对全体随机变量的加权平均。
  - $E(X|Y)$：随机变量关于 Y 的函数无固定值。
  - $E(X|Y = y)$：y 的函数 $f(y)$ 对给定 y 有唯一确定的值与之对应，局限在 $x \in \{x : Y(x) = y\}$ 时 $X(x)$ 取值局部的加权平均。

  先局部平均再整体平均的思想：例–全年级均分的两种算法：

  - $E(X)$：直接全体学生分数的平均
  - $E(E(X|Y))$：先每个班级算平均分 (对每个班级，相当于给定了 $Y = y$)，再计算各个班级的均分。(局部 $\longrightarrow$ 整体)

  对应：目标是找到一个最优模型 $f^*(\mathbf{x})$ 使得 $\mathcal{R}(f)$ 最小，可以先计算在给定 x 的条件下 y 的分布的期望错误，再把各个模型的期望错误平均：

$$\min_f \qquad \mathcal{R}(f) = \mathbb{E}_{\mathbf{x}\sim p(\mathbf{x})}[\mathbb{E}_{y\sim p_r(y|\mathbf{x})}[(y - f(\mathbf{x}))^2]] \tag{61}$$

$f^*(x)$ 作为使用平方损失函数作为优化目标的最优模型，其损失为：

$$\varepsilon = \mathbb{E}_{(\mathbf{x},y)\sim p_r(\mathbf{x},y)}[(y - f^*(\mathbf{x}))^2] \tag{62}$$

然而，ε 是由于样本分布以及噪声引起，无法通过优化减少。为寻求优化目标，探究模型和最优模型之间的差距，因此进一步分解期望错误：

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x},y)\sim p_r(\mathbf{x},y)}[(y - f^*(\mathbf{x}) + f^*(\mathbf{x}) - f(\mathbf{x}))^2] \tag{63}$$

由于 $\mathbb{E}_{\mathbf{x}}\mathbb{E}_y[y - f^*(x)] = 0$（61 推导过程），因此：

$$\mathcal{R}(f) = \mathbb{E}_{\mathbf{x}\sim p_r(\mathbf{x})}[(f(\mathbf{x}) - f^*(\mathbf{x}))^2] + \varepsilon \tag{64}$$

第一项即当前模型和最优模型之间的差距，是机器学习算法可以优化的目标。在实际训练一个模型 $f(x)$ 时，训练集 $\mathcal{D}$ 是从真实分布上独立同分布地采样出的有限样本集合，不同的训练集会得到不同模型，一个机器学习的算法 (模型 + 优化算法) 的能力可以用不同训练集上的平均性能来评价。令 $f_{\mathcal{D}}(x)$ 表示在训练集 $\mathcal{D}$ 上学到的模型，那么对于单个样本 $\mathbf{x}$，不同训练集 $\mathcal{D}$ 和最优的模型 $f^*(x)$ 的期望差距为：

$$\mathbb{E}_{\mathcal{D}}[(f_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}))^2] \tag{65}$$

为了探究一个模型的能力以及在不同训练集之间的差异，通过引入模型在不同训练集上的平均值即 $\mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})]$ 分解期望差距：

$$\mathbb{E}_{\mathcal{D}}[(f_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}))^2] = \mathbb{E}_{\mathcal{D}}[(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] + \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}))^2] \tag{66}$$

通过推导可以得到：

$$\mathbb{E}_{\mathcal{D}}[(f_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}))^2] = (\mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}))^2 + \mathbb{E}_{\mathcal{D}}[(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})])^2] \tag{67}$$

- 第一项为偏差 (bias) 的平方：一个模型在不同训练集上的平均性能和最优性能之间的差异。偏差可以用来衡量一个模型的**拟合能力**。
- 第二项方差 (variance)：指一个模型在不同训练集上的差异，可以用来衡量一个模型是否**过拟合**。

$$\mathcal{R} = (bias)^2 + variance + \varepsilon \tag{68}$$

其中：

$$(bias)^2 = \mathbb{E}_{\mathbf{x}}[(\mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}))^2] \tag{69}$$

$$variance = \mathbb{E}_{\mathbf{x}}[\mathbb{E}_{\mathcal{D}}[(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})])^2]] \tag{70}$$

也就是说，最小化期望错误等价于最小化偏差和方差之和。

2.4.2 图像分析

图 11展示了偏差和方差的四种组合情况：每个图的中心点代表最优模型 $f^*(\mathbf{x})$ ，蓝点为不同训练集 $\mathcal{D}$ 上得到的训练集 $f_{\mathcal{D}}(\mathbf{x})$ 。

- (a)：理想情况，方差和偏差都较小
- (b)：蓝点距离中心点远 $\longrightarrow$ 高偏差 $\longrightarrow$ 拟合能力不足，模型简单；蓝点分布密集 $\longrightarrow$ 低方差 $\longrightarrow$ 不同训练集差异小，泛化能力好，可能欠拟合。
- (c)：蓝点大致分布在中心点附近 $\longrightarrow$ 低偏差 $\longrightarrow$ 拟合能力好，模型复杂；蓝点分布较为稀疏 $\longrightarrow$ 高方差 $\longrightarrow$ 不同训练集差异大，泛化能力差，可能过拟合 (数据少)。
- (d)：最差的一种情况，高偏差高方差。

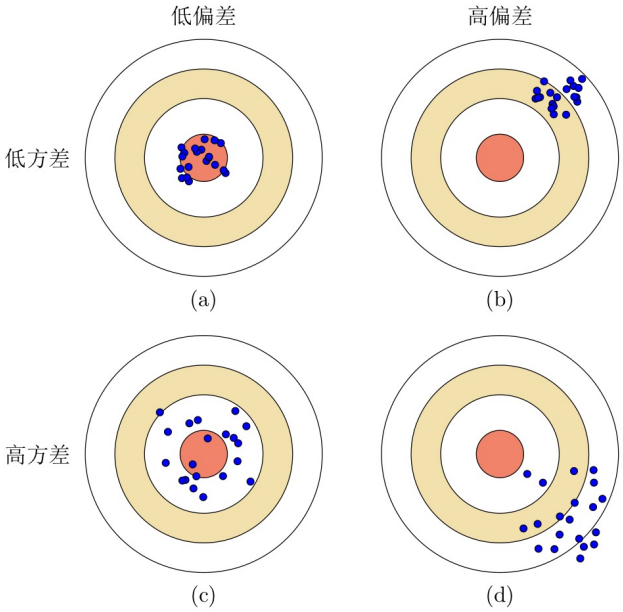


图 11: 偏差方差四种组合示例

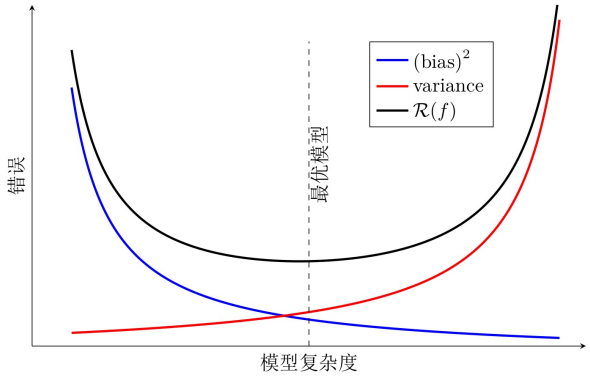


图 12: 期望错误、偏差和方差随复杂度的变化

图 12展示了模型的期望错误、偏差和方差随复杂度的变化情况：

- 随着模型复杂程度的增加，模型拟合能力变强，偏差减少而方差增大，从而导致过拟合；
- 然而模型复杂程度降低时，模型拟合能力变差，偏差增大而方差变小，从而导致欠拟合；
- 因此模型的复杂程度需要达到一个较为合理的值，使得偏差和方差都较小，使得期望错误最小达到最优模型，然而，最优模型并不一定是偏差曲线和方差曲线的交点。

2.4.3 解决方案

- 方差一般会随着样本的增加而减少，因此当样本数量较多时，可以选择能力较强的模型来减少偏差。然而对于大多数任务，训练集数量有限，最优偏差和最优方差无法兼顾。

- 当模型过于复杂，以结构错误最小化为例，可以通过调整正则化系数 λ 控制其复杂程度。然而 λ 变大复杂度上升，方差变小但是偏差上升，过于大时，总的期望错误反而会上升，因此 λ 需要在方差和偏差之间平衡。
- 因此一个模型在训练集上错误率较高时 (b 高)：增加数据特征，提高模型复杂度，减小正则化系数。
- 模型在训练集上错误较低，验证集上错误较高 (v 高)：降低复杂度，加大正则化系数，引入先验等方法。
- 另一种有效降低方差的方法：集成模型——通过多个高方差的平均来降低方差。

2.5 评价指标

前半部分主要集中于学习和训练模型，这节介绍针对训练集对模型的评估方法。对于分类问题，常见的标准有准确率、查准率、召回率、F 值等。

2.5.1 准确和错误率率

对给定测试集 $\mathcal{T} = \left\{(\mathbf{x}^{(1)}, y^{(1)}), ..., (\mathbf{x}^{(N)}, y^{(N)})\right\}$，假设标签 $y^{(n)} \in \{1, ...C\}$ 用模型对测试集每一个样本进行预测，结果为 $\mathbf{Y} = \left\{\hat{y}^{(1)}, ..., \hat{y}^{(N)}\right\}$。

- 准确率 (Accuracy)

$$\mathcal{ACC} = \frac{1}{N}\sum_{n=1}^{N} I(y^{(n)} = \hat{y}^{(n)}) \tag{71}$$

- 错误率 (Error rate)

$$\varepsilon = 1 - \mathcal{ACC} = \frac{1}{N}\sum_{n=1}^{N} I(y^{(n)} \neq \hat{y}^{(n)}) \tag{72}$$

2.5.2 查准率和查全率

准确率是对所有类别整体性能的平均，然而如果需要估计每个类的性能，就需要查准率和查全率。对于类别 c，模型在测试集上的结果可以分为以下四种：

1. 真正例 (True Positive, TP)：一个样本的真实类别为 c 且模型正确预测为类别 c：

$$TP_c = \sum_{n=1}^{N} I(y^{(n)} = \hat{y}^{(n)} = c) \tag{73}$$

2. 假负例 (False Negative, FN)：一个样本的真实类别为 c，模型错误预测为其他类：

$$FN_c = \sum_{n=1}^{N} I(y^{(n)} = c \wedge \hat{y}^{(n)} \neq c) \tag{74}$$

3. 假正例 (False Positive, FP)：一个样本的真实类别为其他类，模型错误预测为 c 类：

$$FP_c = \sum_{n=1}^{N} I(y^{(n)} \neq c \wedge \hat{y}^{(n)} = c) \tag{75}$$

4. 真负例 (True Negative, TN)：一个样本的真实类别为其他类，模型也预测为其他类，即 TN_c。对于类别 c 来说，这种情况一般不需要关注。

这几种情况的关系如图 13混淆矩阵所示：

		预测类别	
		$\hat{y} = c$	$\hat{y} \neq c$
真实类别	$y = c$	TP_c	FN_c
	$y \neq c$	FP_c	TN_c

图 13: 类别 c 预测结果的混淆矩阵

1. 查准率 (Precision)：精确率或者精度，类别 c 的查准率为所有**预测**为类别 c 的样本中，预测正确的比例：

$$\mathcal{P}_c = \frac{TP_c}{TP_c + FP_c} \tag{76}$$

站在模型的角度，关注模型的精确程度，希望现有的模型不出错。“不许错，宁愿漏检，不让现有模型出错，疑罪从无”

2. 查全率 (Recall)：召回率，类别 c 的查全率是所有**真实标签**为类别 c 的样本中，预测正确的比例：

$$\mathcal{R}_c = \frac{TP_c}{TP_c + FN_c} \tag{77}$$

站在真实的角度。“错杀一千，不放过一个，排查安全隐患，宁愿误检，不能漏检”

3. F 值 (F measure)：一个综合指标，为查准率和查全率的调和平均。

$$\mathcal{F}_c = \frac{(1 + \beta^2) \times \mathcal{P}_c \times \mathcal{R}_c}{\beta^2 \times \mathcal{P}_c + \mathcal{R}_c} \tag{78}$$

β a 用于平衡查全率和炸准率的重要性，一般取值为 1，即 $F1$ 值。($\beta < 1$，P 比 R 重要；$\beta > 1$，R 比 P 重要)

2.5.3 宏平均和微平均

为了计算分类算法在所有类别上的总体准确率、召回率、$F1$ 值，经常使用两种平均方法，即宏平均和微平均。

1. 宏平均：每一类的性能指标的平均值

$$\mathcal{P}_{macro} = \frac{1}{C} \sum_{c=1}^{C} \mathcal{P}_c \tag{79}$$

$$\mathcal{R}_{macro} = \frac{1}{C} \sum_{c=1}^{C} \mathcal{R}_c \tag{80}$$

$$F1_{macro} = \frac{2 \times \mathcal{P}_{macro} \times \mathcal{R}_{macro}}{\mathcal{P}_{macro} + \mathcal{R}_{macro}} \tag{81}$$

2. 微平均：每一个样本性能指标的算术平均。对于单个样本，其准确率和召回率相同 (1 或 0)。因此其查准率，召回率和 $F1$ 值都相同。当不同类别样本数量不均衡，用宏平均比微平均更合理，由于其更关注小类别上的指标。

应用例如图 14

对于一个三类分类问题，数据集的真实标签和模型的预测标签如下：

真实标签	1	1	2	2	2	3	3	3	3
预测标签	1	2	2	2	3	3	3	1	2

分别计算模型的查准率、查全率、F1 值以及它们的宏平均和微平均。

图 14: 评估标准应用例

2.5.4 其他方法

1. 实际应用过程还可以通过调整分类模型的阈值来进行更全面的评价，如 AUC (Area Under Curve)、ROC (Receiver Operating Characteristic) 曲线、PR (Precision-Recall) 曲线等。此外很多任务还有专门的评估方式，如 TopN 准确率。

2. 交叉验证 (Cross Validation) 是一种较好的衡量机器学习模型的统计分析方法：把原始数据分成 K 组不重复子集，每次选 $K-1$ 组作为训练集，剩下 1 组作为验证集，这样可以进行 K 次实验得到 K 个模型，这 K 个模型在各自验证集上的错误率的平均作为分类器的评价。此方法可有效避免划分训练集和测试集的随机性对结果造成的影响。

2.6 理论和定理

此节将介绍机器学习的一些定理，对理解机器学习的内在性特征很有帮助。

2.6.1 PAC 学习理论 (Probably Approximately Correct)

在机器学习中，只运用经验或者多次试验来选择合适的模型，样本数量积极学习算法的收敛速度往往不可靠且成本高，因此希望有一套理论即计算学习理论 (Computational Learning Therory) 来分析问题难度，计算模型能力，为学习算法提供理论保证，其中最基础的就是可能近似正确理论 (PAC)。

- 泛化错误 (Generalization Error)：即期望错误和经验错误之间的差异，可以衡量一个机器学习模型 f 是否很好地泛化到未知数据：

$$\mathcal{G}_{\mathcal{D}}(f) = \mathcal{R}(f) - \mathcal{R}_{\mathcal{D}}^{emp}(f) \tag{82}$$

据大数定律 (如图 15) 当训练集大小趋向无穷大，泛化错误趋向 0，即经验风险趋向期

大数定律（Law Of Large Numbers）是指n个样本$X_1, \cdots, X_n$是独立同分布的，即$\mathbb{E}[X_1] = \cdots = \mathbb{E}[X_n] = \mu$，那么其均值

$$\bar{X}_n = \frac{1}{n}(X_1 + \cdots + X_n), \tag{D.42}$$

收敛于期望值μ。

$$\bar{X}_n \to \mu \quad \text{for} \quad n \to \infty \tag{D.43}$$

图 15: 大数定律

望风险。

$$\lim_{|\mathcal{D}|\to\infty} \mathcal{R}(f) - \mathcal{R}_{\mathcal{D}}^{emp}(f) = 0 \tag{83}$$

由于不知道真实数据分布 $p_r(\mathbf{x}, y)$ 和真实目标函数 $g(x)$，因此希望学习到一个期望错误为 0 的函数不切实际，需要降低对学习算法能力的期望以求其可以学到一个近似正确的假设，即 PAC 学习。

- PAC 学习：
  - 近似正确 (Approximately Correct)：一个假设 f 近似正确，即它的泛化错误小于一个界限 $\mathcal{G}_{\mathcal{D}}(f) \leq \varepsilon$ 且 $0 < \varepsilon < \frac{1}{2}$，否则模型不能用来做正确的预测。
  - “可能”：即一个学习算法 $\mathcal{A}$ 有“可能”以 $1 - \delta$ 的概率学到一个近似正确的假设。其中 $0 < \delta < \frac{1}{2}$。

  那么一个 PAC 学习可学习的算法即该学习算法能在多项式时间内从合理数量的训练数据中学习到一个近似正确的 $f(x)$。

$$P((\mathcal{R}(f) - \mathcal{R}_{\mathcal{D}}^{emp}(f)) \leq \varepsilon) \geq 1 - \delta \tag{84}$$

其中 ε，δ 是和样本数量 n, 假设空间 $\mathcal{F}$ 相关的变量。如果固定 ε，δ，可以反过来计算样本的复杂度：

$$n(\varepsilon, \delta) \geq \frac{1}{2\varepsilon^2}(\ln|\mathcal{F}| + \ln\frac{2}{\delta}) \tag{85}$$

由此看出，如果希望模型的假设空间越大，泛化错误越小，其需要的样本数量就越多。

因此 PAC 学习理论可以帮助分析一个机器学习方法在什么**条件**下可以学到一个**近似正确**的分类器。

2.6.2 没有免费午餐定理 (No Free Lunch Theorem, NFL)

是在最优化理论提出的。对于基于迭代的最优化算法，不存在某种算法对所有问题 (有限的搜索空间内) 都有效。即如果一个算法对某些问题有效，那么它一定在另外问题上比纯随机搜索算法更差。即不能脱离具体问题谈论算法的优劣，任何算法都有局限性，需要具体问题具体分析。

2.6.3 丑小鸭定理 (Ugly Ducking Theorem)

“丑小鸭和白天鹅的区别和两只白天鹅之间的区别一样大”——世界上不存在相似性的客观标准，一切相似性标准都是主观的。丑小鸭和白天鹅从体型和基因角度所得的相似程度的结果是不同的。

2.6.4 奥卡姆剃刀 (Occam's Razor)

"如无必要，勿增实体"——和机器学习的正则化的算法类似：简单的模型泛化能力更好。在两个性能相近的模型中，应该选择一个更简单的模型。奥卡姆剃刀的一种形式化就是最小描述长度 (Minimum Description Length, MDL) 原则，即对一个数据集 $\mathcal{D}$，最好的模型 $f \in \mathcal{F}$ 会使得数据集的压缩效果最好，即编码长度最小。通过贝叶斯学习的观点来解释数据集 $\mathcal{D}$ 的后验概率 (参照公式 54)：

$$\max_f \log p(f|\mathcal{D}) = \max_f \log p(\mathcal{D}|f) + \log p(f) = \min_f -\log p(\mathcal{D}|f) - \log p(f) \tag{86}$$

- $-\log p(f)$：模型 f 的编码长度
- $-\log p(\mathcal{D}|f)$：模型 f 在该模型数据集 $\mathcal{D}$ 的编码长度。

因此模型 f 不仅需要可以编码数据集 $\mathcal{D}$ 还要使得其自身尽可能简单。

2.6.5 归纳偏置 (Inductive Bias)

在机器学习中，很多学习算法对学习问题做的一些假设即归纳偏置。比如在最近邻分类器中，假设在特征空间中，一个小的局部区域中大部分样本属于同一类；在朴素贝叶斯分类器中，假设每个特征的条件概率相互独立。(归纳偏置在贝叶斯学习中也常称为先验。)

3 线性模型 (Linear Model)

3.1 引入

线性模型是机器学习中应用最广泛的模型，是通过样本特征的线性组合来进行预测的模型，即：

$$f(\mathbf{x}, \mathbf{w}) = w_1 x_1 + w_2 x_2 + ... + w_d x_d + b = \mathbf{w}^T \mathbf{x} + b \tag{87}$$

在上一节提到的线性回归中，直接用 $f(\mathbf{x}, \mathbf{w})$ 来预测输出目标 y。然而在分类问题中，由于输出目标 y 是离散的标签，无法直接通过值域为实数的 $f(\mathbf{x}, \mathbf{w})$ 来预测因此需要引入一个非线性的决策函数 (decision function) $g(\cdot)$ 来预测输出目标：

$$y = g(f(\mathbf{x}, \mathbf{w})) \tag{88}$$

其中，$f(\mathbf{x}, \mathbf{w})$ 也称为判别函数 (discriminant function)。对于两类问题，$g(\cdot)$ 可以是符号函数 (sign function):

$$g(f(\mathbf{x}, \mathbf{w})) = sgn(f(\mathbf{x}, \mathbf{w})) \triangleq \begin{cases} +1 & f(\mathbf{x}, \mathbf{w}) > 0 \\ -1 & f(\mathbf{x}, \mathbf{w}) < 0 \end{cases} \tag{89}$$

对于 $f(\mathbf{x}, \mathbf{w}) = 0$ 不进行预测。结构如图 16所示：

本章根据损失函数的不同，将分别介绍四种不同的线性模型：logistic 回归、softmax 回归、感知器和支持向量机。

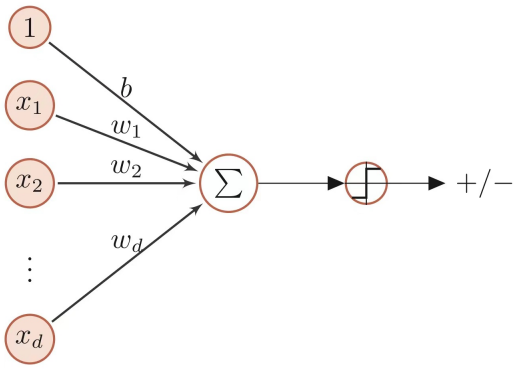


图 16: 两类分类的线性模型

3.2 线性判别函数和决策边界

由公式 88可知，一个线性分类模型 (Linear Classification Model) 或线性分类器 (Linear Classifier) 是有一个或多个线性判别函数 $f(\mathbf{x}, \mathbf{w})$ 和非线性决策函数 $g(\cdot)$ 组成。对于分类问题，先考虑两类分类，进一步拓展到多类分类。

3.2.1 两类分类 (Binary Classification)

需要的因素：

- 两类分类的类别标签 $\mathbf{y} = \{+1, -1\}$
- 一个线性判别函数 $f(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T\mathbf{x} + b$ 。
- 一个分割超平面 (hyperplane)：特征空间 $\mathbb{R}^d$ 中所有满足 $f(\mathbf{x}, \mathbf{w}) = 0$ 的点组成，称为决策边界 (decision boundary) 或决策平面 (decision surface)。决策边界将空间一分为二，一个区域对应一个类别。(d 维空间中的超平面是 $d-1$ 维的)

所谓线性分类模型即决策边界是线性超平面。在特征空间中，决策平面与权重向量正交 (证–如果向量空间中两个向量的内积为 0，则它们正交 (Orthogonal))。特征空间中每个样本点到决策平面的有向距离 (signed distance) 为 (证–点到平面的距离)

$$\gamma = \frac{f(\mathbf{x}, \mathbf{w})}{\|\mathbf{w}\|} \tag{90}$$

γ 可视为点 $\mathbf{x}$ 在 $\mathbf{w}$ 方向上的投影。(补充:)

- 有向直线由无向直线和正方向组成，例如数轴就是一条有向直线。
- 二维空间中，有向距离方向的判定即左手定则：四指指向有向直线的正方向，与之垂直的拇指即有向距离的正方向。对直线 $Ax + By + C = 0$ 来说，即 $(B, -A)$ 方向。有向距离即点到直线的距离去掉绝对值。
- 三维空间中，对由非共线向量 $\mathbf{i}$ 和 $\mathbf{j}$ 张成的平面，有向距离的正方向即 $\mathbf{i} \times \mathbf{j}$ 的方向。

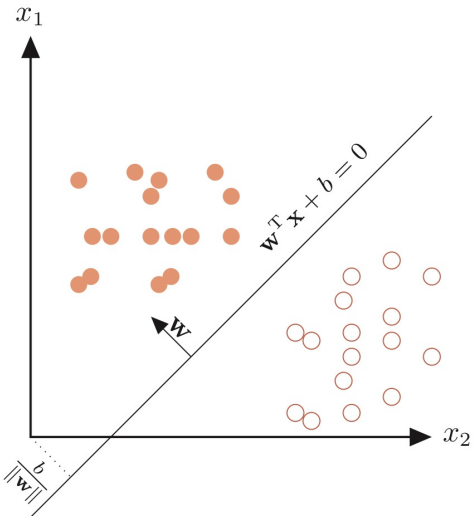


图 17: 线性模型二分类决策边界示例

图给出了一个二维数据的线性决策边界示例 17，其中样本的特征向量为 $\mathbf{x}=[x_1,x_2]$，权重向量 $\mathbf{w}=[w_1,w_2]$。对给定 N 个样本的训练集 $\mathcal{D}=\left\{(\mathbf{x}^{(n)},y^{(n)})\right\}_{n=1}^{N}$，其中 $y^{(n)}\in\{+1,-1\}$ 线性模型试图学习到参数 w^* 使得对于每个训练集内的样本 $(\mathbf{x}^{(n)},y^{(n)})$ 尽量满足：

$$y^{(n)}=\begin{cases}+1 & f(\mathbf{x}^{(n)},\mathbf{w}^*)>0\\ -1 & f(\mathbf{x}^{(n)},\mathbf{w}^*)<0\end{cases} \tag{91}$$

合并上述两个公式，即参数 ω^* 尽量满足：

$$y^{(n)}f(\mathbf{x}^{(n)},\mathbf{w}^*)>0,\forall n\in[1,N] \tag{92}$$

由此可引出 **两类线性可分**的定义：对于训练集 $\mathcal{D}=\left\{\mathbf{x}^{(n)},y^{(n)}\right\}_{n=1}^{N}$，如果存在权重向量 ω^*，对所有样本都满足 $yf(\mathbf{x}^{(n)},\mathbf{w}^*)>0$，那么训练集 $\mathcal{D}$ 是线性可分的。
而为了学习参数 $\mathbf{w}$，需要定义合适的损失函数以及优化方法。对于两类分类问题即使用 0-1 损失函数：

$$\mathcal{L}_{01}(y,f(\mathbf{x},\mathbf{w}))=I(yf(\mathbf{x},\mathbf{w})>0) \tag{93}$$

但其关于 $\mathbf{w}$ 的导数为 0，无法优化 $\mathbf{w}$。

3.2.2 多类分类 (Multi-class Classification)

多类分类即类别数 $C>2$。多类分类一般需要多个线性判别函数，但设计判别函数的方式有很多种。假设一个多类分类问题的类别为 $\{1,2,...,C\}$，常用的方式有三种：

1. “一对其余 (OVR)”：把多个分类问题 $\longrightarrow C$ 个“一对其余”的两类分类问题，共需要 C 个判别函数，第 c 个判别函数 f_c 是将类别 c 的样本和不属于类别 c 的样本分开。

2. “一对一 (OVO)”：把多个分类问题 $\longrightarrow \frac{C(C-1)}{2}$ (即两两配对，组合数 C_n^2) 个“一对一”两类分类问题。那么共需要 $\frac{C(C-1)}{2}$ 个判别函数，其中第 (i,j) 个判别函数的作用即把类 i 和类 j 分开。

3. “argmax”：改进的“一对其余”方式，共需要 C 个判别函数：

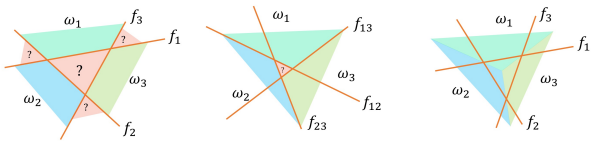
$$f_c(\mathbf{x}, \mathbf{w}_c) = \mathbf{w}_c^T \mathbf{x} + b_c, c = [1, ..., C] \tag{94}$$

如果存在类别 c 对于其他所有类别 $\tilde{c}$ ($\tilde{c} \neq c$) 都满足 $f_c(\mathbf{x}, \mathbf{w}_c) > f_{\tilde{c}}(\mathbf{x}, \mathbf{w}_{\tilde{c}})$，那么 $\mathbf{x}$ 属于类别 c。即：

$$y = \underset{c=1}{\overset{C}{argmax}}\, f_c(\mathbf{x}, \mathbf{w}_c) \tag{95}$$

另外，在线性分类中，如果决策区域是非凸的，即若点 $\mathbf{x}_1$ 和 $\mathbf{x}_2$ 被分为类 c，那么 $\rho\mathbf{x}_1 + (1-\rho)\mathbf{x}_2, \rho \in (0, 1)$ 也会被分为类 c。

三种分类方式如图 18所示。分析：



(a) “一对其余”方式　(b) “一对一”方式　(c) “argmax”方式

图 18: 三个多类分类方式

- (a)：相当于二分类问题进行了三次，把判别函数想象成三个人，每个人的任务即识别自己的那部分和别人的，问号区域即冲突区。无法确定属于“哪个人”。
- (b)：同样是三个人来分类，不过这种情况“每个人”需要来区分某区域在两个区域中“更像”哪一类，问号区域即冲突区，三个人有不同答案。
- (c)：前两类都存在难以界定的区域，然而“argmax”规避了这个问题。相邻两类的决策边界实际上由

$$f_i(\mathbf{x}, \mathbf{w}_i) - f_j(\mathbf{x}, \mathbf{w}_j) = 0 \tag{96}$$

决定，其法向量为权重向量 $\mathbf{w}_i - \mathbf{w}_j$ (权重向量和决策平面正交)。在这种方式中的“三个人”的任务变成了判定对于某块区域 $f_1(\mathbf{x}, \mathbf{w}_2), f_2(\mathbf{x}, \mathbf{w}_2), f_3(\mathbf{x}, \mathbf{w}_3)$ 哪个的可能性最大，因为存在排序，总在寻求一个“最大值”，因此一定可以界定每块区域，不存在冲突区域。

由此引出 **多类线性可分**的定义：对于训练集 $\mathcal{D} = \left\{\mathbf{x}^{(n)}, y^{(n)}\right\}_{n=1}^{N}$，如果存在 C 个权重向量 ω_c^*，$1 \leq c \leq C$，对所有第 c 类样本都满足 $f_c(\mathbf{x}, \mathbf{w}_c) > f_{\tilde{c}}(\mathbf{x}, \mathbf{w}_{\tilde{c}}), \forall \tilde{c} \neq c$，那么训练集 $\mathcal{D}$ 是线性可分的。因此如果数据集可以多类线性可分，一定存在一个“argmax”方式的线性分类器将它们正确分开。

3.3 Logistic 回归 (Logistic Regression, LR)

3.3.1 模型

- **必备知识：Logistic 函数**

$$logistic(x) = \frac{L}{1 + exp(-k(x - x_0))} \tag{97}$$

当参数为 $(k=1, x_0=0, L=1)$ 时，logistic 函数称为标准 logistic 函数，记为 $\sigma(x)$：

$$\sigma(x) = \frac{1}{1+exp(-x)} \tag{98}$$

标准 logistic 函数经常用来将一个实数空间的数映射到 $(0,1)$ 之间。其导数为：

$$\sigma'(x) = \sigma(x)(1-\sigma(x)) \tag{99}$$

Logistic 回归是一种常用的处理两类分类的线性模型。目标标签设定为 $y \in \{0,1\}$ 以符合 Logistic 描述习惯。为了解决连续线性函数不适合分类的问题引入了非线性函数 $g: \mathbb{R}^d \longrightarrow (0,1)$ 来预测类别标签的后验概率 $p(y=1|\mathbf{x})$：

$$p(y=1|\mathbf{x}) = g(f(\mathbf{x}, \mathbf{w})) \tag{100}$$

其中 $g(\cdot)$ 即激活函数 (activate function)，用来把实数区间挤压到 $(0,1)$ 区间来表示概率，逆函数 $g^{-1}(\cdot)$ 也称为联系函数 (link function)。在 logistic 回归中，用标准 logistic 函数来作为激活函数，那么标签为 1 的后验概率为：

$$p(y=1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) \triangleq \frac{1}{1+exp(-\mathbf{w}^T\mathbf{x})} \tag{101}$$

那么标签为 0 的后验概率为：

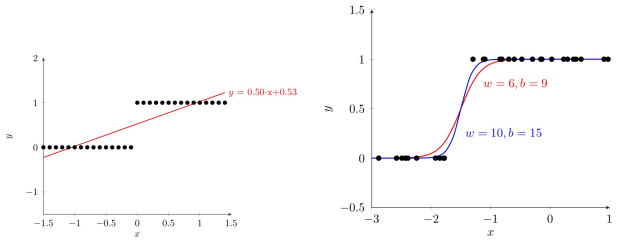
$$p(y=0|\mathbf{x}) = 1 - p(y=1|\mathbf{x}) = \frac{exp(-\mathbf{w}^T\mathbf{x})}{1+exp(-\mathbf{w}^T\mathbf{x})} \tag{102}$$

将 101进行变换得到：

$$\mathbf{w}^T\mathbf{x} = \log\frac{p(y=1|\mathbf{x})}{1-p(y=1|\mathbf{x})} = \log\frac{p(y=1|\mathbf{x})}{p(y=0|\mathbf{x})} \tag{103}$$

- $\mathbf{w}^T\mathbf{x}$：线性函数 (增广后)
- $\frac{p(y=1|\mathbf{x})}{p(y=0|\mathbf{x})}$：样本 $\mathbf{x}$ 的正反例后验概率的比值也就是几率 (odds)
- $\log\frac{p(y=1|\mathbf{x})}{p(y=0|\mathbf{x})}$：对数几率 (log odds or logit)

因此 logistic 回归可以看作是预测值为“标签的对数几率”的线性回归模型，也被称为对数几率回归 (Logit Regression)。由图 19可以看出，Logistic 回归比线性回归拟合地更到位。



(a) 线性回归　　(b) Logistic 回归

图 19: 一维数据的两类问题示例

3.3.2 参数学习

Logistic 回归用交叉熵作为损失函数 (公式见 20) 并用梯度下降的方法进行优化。

- 模型预测：对给定 N 个样本的训练集 $\mathcal{D}=\left\{(\mathbf{x}^{(n)},y^{(n)})\right\}_{n=1}^{N}$，用此模型对每个样本 $\mathbf{x}^{(n)}$ 进行预测，并用 $\hat{y}^{(n)}$ 表示 $\mathbf{x}^{(n)}$ 的输出标签为 1 的后验概率：

$$\hat{y}^{(n)}=\sigma(\mathbf{w}^T\mathbf{x}^{(n)}),1\leq n\leq N \tag{104}$$

- 真实：由于 $\hat{y}^{(n)}\in\{0,1\}$ 样本 $(\mathbf{x}^{(n)},y^{(n)})$ 真实的条件概率可以表示为：

$$p_r(y^{(n)}=1|\mathbf{x}^{(n)})=y^{(n)} \tag{105}$$

$$p_r(y^{(n)}=0|\mathbf{x}^{(n)})=1-y^{(n)} \tag{106}$$

因此，使用交叉熵损失函数，其风险函数 (为计算简便忽略了正则化项)：

$$\mathcal{R}(\mathbf{w})=-\frac{1}{N}\sum_{n=1}^{N}(p_r(y^{(n)}=1|\mathbf{x}^{(n)})\log\hat{y}^{(n)}+p_r(y^{(n)}=0|\mathbf{x}^{(n)})\log(1-\hat{y}^{(n)})) \tag{107}$$

即

$$\mathcal{R}(\mathbf{w})=-\frac{1}{N}\sum_{n=1}^{N}(y^{(n)}\log\hat{y}^{(n)}+(1-y^{(n)})\log(1-\hat{y}^{(n)})) \tag{108}$$

$\mathcal{R}(\mathbf{w})$ 对参数 $\mathbf{w}$ 求导得 (证–链式法则 + 公式 99)：

$$\frac{\partial\mathcal{R}(\mathbf{w})}{\partial\mathbf{w}}=-\frac{1}{N}\sum_{n=1}^{N}\mathbf{x}^{(n)}(y^{(n)}-\hat{y}^{(n)}) \tag{109}$$

进而采用梯度下降法对其进行优化：初始化 $w_0=0$，进而通过此公式进行迭代：

$$\mathbf{w}_{t+1}=\mathbf{w}_t+\alpha\frac{1}{N}\sum_{n=1}^{N}\mathbf{x}^{(n)}(y^{(n)}-\hat{y}_{\mathbf{w}_t}^{(n)}) \tag{110}$$

其中，$\hat{y}_{\mathbf{w}_t}^{(n)}$ 是当参数为 $\mathbf{w}_t$ 时，logistic 回归模型的输出。由公式看出，$\mathcal{R}(\mathbf{w})$ 是关于参数 $\mathbf{w}$ 的连续可导凸函数，因此还可用高阶的优化方法如牛顿法来进行优化。(以证–为什么不用平方损失函数来看梯度下降法的优良特性)

3.4 Softmax 回归 (Soft Regression)

3.4.1 模型

- **必备知识：softmax 函数**对于 K 个标量 $x_1,x_2,...,x_K$，softmax 函数：

$$z_k=softmax(x_k)=\frac{exp(x_k)}{\sum_{i=1}^{K}exp(x_i)} \tag{111}$$

这样，k 个变量 $x_1,x_2,...,x_K\longrightarrow$ 分布：$z_1,z_2,...,z_K$，满足：

$$\forall k,z_k\in[0,1],\sum_{i=1}^{K}z_k=1 \tag{112}$$

当输入变为 K 维向量 $\mathbf{x}$，softmax 函数：

$$\hat{\mathbf{z}} = softmax(\mathbf{x}) = \frac{exp(\mathbf{x})}{\sum_{k=1}^{K} exp(x_k)} = \frac{exp(\mathbf{x})}{\mathbf{1}_K^T exp(\mathbf{x})} \tag{113}$$

其中，$\mathbf{1}_K = [1, ...1]_{k\times 1}$ 是 K 维的全 1 向量。此时 softmax 导数为 (证)：

$$\frac{\partial softmax(\mathbf{x})}{\partial \mathbf{x}} = diag(softmax(\mathbf{x})) - softmax(\mathbf{x}) softmax(\mathbf{x})^T \tag{114}$$

Softmax 回归也称为多项 (multinomial) 或多类的 (multi-class) 的 logistic 回归，即 logistic 在多类问题的推广。对于多类分类问题，类别标签 $y \in \{1, 2, ..., C\}$ 可以有 C 个取值。那么对给定样本 $\mathbf{x}$，预测属于类别 c 的概率：

$$p(y = c|\mathbf{x}) = softmax(\mathbf{w}_c^T \mathbf{x}) = \frac{exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{c=1}^{C} exp(\mathbf{w}_c^T \mathbf{x})} \tag{115}$$

其中 $\mathbf{w}_c$ 为第 c 类的权重向量。那么根据 $argmax$ 分类方式即公式 95，Softmax 回归的决策函数：

$$\hat{y} = \underset{c=1}{\overset{C}{argmax}}\, p(y = c|\mathbf{x}) = \underset{c=1}{\overset{C}{argmax}}\, \mathbf{w}_c^T \mathbf{x} \tag{116}$$

与 logistic 回归的关系：当 $C = 2$ 时，softmax 决策函数：

$$\hat{y} = \underset{y\in\{0,1\}}{argmax}\, \mathbf{w}_y^T \mathbf{x} \tag{117}$$

即：

$$\hat{y} = I(\mathbf{w}_1^T \mathbf{x} - \mathbf{w}_0^T \mathbf{x} > 0) = I((\mathbf{w}_1 - \mathbf{w}_0)^T \mathbf{x} > 0) \tag{118}$$

可以发现两类分类的权重向量为 $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$。(决策平面：$f(\mathbf{x}, \mathbf{w}) = 0$ 也符合了 96)

向量表示：据公式 113：

$$\hat{\mathbf{y}} = softmax(W^T \mathbf{x}) = \frac{exp(W^T \mathbf{x})}{\mathbf{1}^T exp(W^T \mathbf{x})} \tag{119}$$

- $W = [\mathbf{w}_1, ..., \mathbf{w}_C]$：由 C 个类的权重向量组成的矩阵
- $\mathbf{1}$：全 1 向量
- $\hat{\mathbf{y}} \in \mathbb{R}^C$：所有类别的预测条件概率组成的向量，第 c 维的值即第 c 类的预测条件概率。

3.4.2 参数学习

同样地，对给定 N 个样本的训练集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)}, y^{(n)})\right\}_{n=1}^{N}$，softmax 回归用交叉熵作为损失函数 (公式见 20) 来学习最优参数矩阵 W^*，并用梯度下降的方法进行优化。

- 真实：用 C 维的 one-hot 向量 $\mathbf{y} \in \{0, 1\}^C$ 来表示类别标签 (第 c 维为 1，其余为 0)。那么对于类别 c，其向量表示为：

$$\mathbf{y} = [I(1 = c), (2 = c), ..., I(C = c)]^T \tag{120}$$

- 模型：$\hat{\mathbf{y}}^{(n)} = softmax(W^T \mathbf{x}^{(n)})$ 样本 $\mathbf{x}^{(n)}$ 在每个类别的后验概率。

那么采用交叉熵损失函数，softmax 回归模型的风险函数：

$$\mathcal{R}(W) = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C}\mathbf{y}_c^{(n)}\log\widehat{\mathbf{y}}_c^{(n)} = -\frac{1}{N}\sum_{n=1}^{N}(\mathbf{y}^{(n)})^T\log\widehat{\mathbf{y}}^{(n)} \tag{121}$$

那么风险函数关于 $\mathcal{R}(W)$ 关于 W 的梯度 (证–链式法则 + 公式 114)：

$$\frac{\partial\mathcal{R}(\mathbf{w})}{\partial\mathbf{w}} = -\frac{1}{N}\sum_{n=1}^{N}\mathbf{x}^{(n)}(\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)})^T \tag{122}$$

进而采用梯度下降法对其进行优化：初始化 $W_0 = 0$，进而通过此公式进行迭代：

$$W_{t+1} = W_t + \alpha(\frac{1}{N}\sum_{n=1}^{N}\mathbf{x}^{(n)}(\mathbf{y}^{(n)} - \hat{\mathbf{y}}_{W_t}^{(n)})^T) \tag{123}$$

其中，$\hat{y}_{W_t}^{(n)}$ 是当参数为 W_t 时，softmax 回归模型的输出。

- **注：** softmax 回归中使用的 C 个权重向量是冗余的，即对所有权重向量都减去一个同样的向量 $\mathbf{y}$ 不改变其输出结果。因此需要正则化来约束参数防止在数值计算出现溢出问题。

3.5 感知器 (perceptron)

感知器是最简单的人工神经网络，只有一个神经元。是对生物神经元的简单模拟：

- 权重–突触
- 偏置–阈值
- 激活函数–细胞体：输出为 +1 或 -1

感知器是一种简单的二分类线性分类模型，分类准则与公式 89类似

$$\hat{y} = sgn(\mathbf{w}^T\mathbf{x}) \tag{124}$$

3.5.1 参数学习

对给定 N 个样本的训练集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)}, y^{(n)})\right\}_{n=1}^{N}$，其中 $\mathbf{y}^{(n)} \in \{+1, -1\}$ 感知器学习的目标即试图找到一组参数 $\mathbf{w}^*$ 使得对每个样本都有：

$$\mathbf{y}^{(n)}\mathbf{w}^{*T}\mathbf{x}^{(n)} > 0, \forall n \in \{1, ..., N\} \tag{125}$$

即二分类定义公式 92。感知器的学习算法是一种**错误驱动**的**在线**学习算法。即先初始化一个权重为 0 (通常为全 0 向量)，每错一个样本 $(\mathbf{x}, y)$，即 $y\mathbf{w}^T\mathbf{x} < 0$，就用这个错误样本更新权重 (类似 SGD 每次更新一个参数)：

$$\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x} \tag{126}$$

(合理性推导)，算法如图 20所示：因此根据其学习策略，可以反推感知器的损失函数：

$$\mathcal{L} = max(0, -y\mathbf{w}^T\mathbf{x}) \tag{127}$$

算法 3.1　两类感知器的参数学习算法

输入: 训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，最大迭代次数 T

初始化：$\boldsymbol{w}_0 \leftarrow 0, k \leftarrow 0, t \leftarrow 0$；
repeat
  对训练集 $\mathcal{D}$ 中的样本随机排序;
  for $n = 1 \cdots N$ **do**
    选取一个样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$;
    if $\boldsymbol{w}_k^{\mathrm{T}}(y^{(n)}\boldsymbol{x}^{(n)}) \leq 0$ **then**
      $\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k + y^{(n)}\boldsymbol{x}^{(n)}$;
      $k \leftarrow k + 1$;
    end
    $t \leftarrow t + 1$;
    if $t = T$ **then** break;　// 达到最大迭代次数
  end
until $t = T$;

输出: $\boldsymbol{w}_k$

图 20: 两类感知器算法

(对–0；错 $-y\mathbf{w}^T\mathbf{x}$) 采用随机梯度下降，每次更新的梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \begin{cases} 0 & \text{if} \quad y\mathbf{w}^T\mathbf{x} > 0 \\ -y\mathbf{x} & \text{if} \quad y\mathbf{w}^T\mathbf{x} < 0 \end{cases} \tag{128}$$

图 21为感知器参数学习的更新过程。正例：红色实心点；负例：蓝色实心点；红虚线箭头：权重更新的方向 $(y\mathbf{x})$。

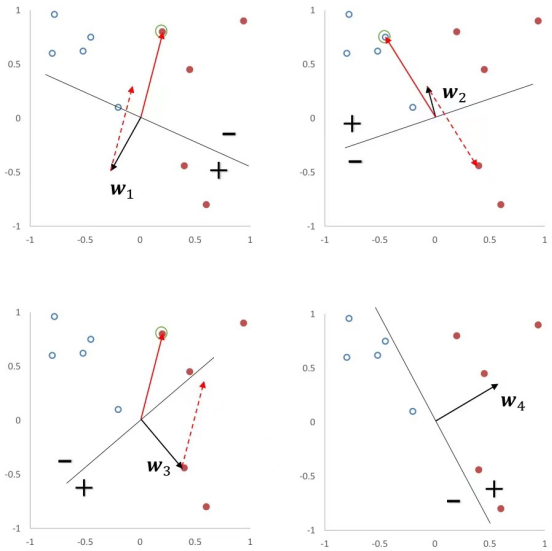


图 21: 感知器参数学习的过程

3.5.2 感知器的收敛性

1. 两类线性可分：当数据集是**两类线性可分**时（ 92），对于训练集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)}, y^{(n)})\right\}_{n=1}^{N}$，其中 $\mathbf{x}^{(n)}$ 为样本的增广权重向量，$y^{(n)} \in \{+1, -1\}$，存在一个正的常数 γ 和权重向量 $\mathbf{w}^*$，且 $\|\mathbf{w}^*\| = 1$ (由于是分类问题只关注 $\mathbf{w}^*$ 的符号，模的长度为方便约定 1)，对所有 n 都满足 $(\mathbf{w}^*)^T(y^{(n)}\mathbf{x}^{(n)}) \geq \gamma$。

2. 感知器收敛性定理：给定一个训练集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)}, y^{(n)})\right\}_{n=1}^{N}$，假设 R 是其中最大的特征向量的模：

$$R = \max_{n} \left\|x^{(n)}\right\| \tag{129}$$

   如果 $\mathcal{D}$ 线性可分，那么感知器算法的更新 (迭代) 次数不超过 $\frac{R^2}{\gamma^2}$ 次。可通过计算 $\|\mathbf{w}_K\|^2$ 上下界来证明 (证)。

但感知器存在以下不足之处：

1. 数据集线性可分时，感知器可以找到一个平面把两类数据分开但无法保证泛化能力。
2. 感知器对样本顺序敏感 (3.5.3 提及)，每次迭代顺序不一样时，找到的分割平面往往不一致。
3. 如果训练集不是线性可分的，感知器就永远不会收敛。

3.5.3 参数平均感知器

1. 必要性：由感知器收敛性定理可知，间隔 γ 越大，收敛越快，然而保证不了找到最优判别函数，可能导致过拟合。由上小节感知器第二个缺点知感知器学到的权重向量和样本训练顺序有关，因此在迭代过程中排在后面的错误样本比排在前面的对最终权重影响更大。例如：共有 1000 个训练样本，在前 100 个之后，感知器已经学习到到一个较好的参数，后面 899 个都预测正确，然而最后一个预测错误使得参数发生更新反而会降低其泛化程度使得学习到的参数变差。因此采取了**参数平均**的策略来提高感知器的鲁棒性和泛化能力也叫“投票感知器”。(补充：鲁棒性即 robust, 指异常和危险情况下系统生存的能力。这里即使得感知器对噪声数据的应对能力更强，泛化能力更强等。)

2. 投票感知器 (Voted Perceptron)：

   - 形式：

$$\hat{y} = sgn(\sum_{k=1}^{K} c_k sgn(\mathbf{w}_k^T \mathbf{x})) \tag{130}$$

   - 原理：将感知器学习过程的 K 个权重向量全部保存，并赋予每个 $\mathbf{w}_k$ 一个置信系数 $c_k (1 \leq k \leq K)$ 最终的结果由 K 个不同权重的感知器投票决定因此也叫投票感知器。
   - 置信系数 c_k：

$$c_k = t_{k+1} - t_k \tag{131}$$

     t_k 为第 k 次更新权重 $\mathbf{w}_k$ 时的迭代次数 (训练过的样本的数量)，则 t_{k+1} 为下次权重更新时的迭代次数，那么 c_k 即二者之间间隔的迭代次数，因此置信系数越大说明权重在 $\mathbf{w}_k$ 之后的训练过程正确分类样本的数量越多，越值得信赖。

3. 平均感知器 (Averaged Perceptron)：投票感知器虽然提高了泛化能力，但是需要保存 K 个权重向量，反而会带来额外开销，因此用“参数平均”的策略来减少投票感知器的参数数量即 **平均感知器**。

   - 形式：

$$\hat{y} = sgn(\frac{1}{T}\sum_{k=1}^{K} c_k(\mathbf{w}_k^T\mathbf{x})) = sgn(\frac{1}{T}\sum_{k=1}^{K}(c_k\mathbf{w}_k)^T\mathbf{x}) = sgn((\frac{1}{T}\sum_{t=1}^{T}\mathbf{w}_t)^T\mathbf{x}) = sgn(\bar{\mathbf{w}}^T\mathbf{x}) \tag{132}$$

   - 原理：T 即迭代回合总数，$\bar{\mathbf{w}}$ 即迭代总数的平均权重向量因此相当于在之前的“两类感知器算法” 20中增加一个 $\bar{\mathbf{w}}$ 并且在**每次**迭代时，都更新 $\bar{\mathbf{w}}$：

$$\bar{\mathbf{w}} \leftarrow \bar{\mathbf{w}} + \mathbf{w}_t \tag{133}$$

   - 改进：以上方法需要在处理每一个样本时都更新 $\bar{\mathbf{w}}$，然而因为 $\bar{\mathbf{w}}$ 和 $\mathbf{w}_t$ 都是稠密向量，更新操作比较耗费时间，因此为了提高迭代速度，要使用改进方法令其只在**错误**预测发生时才更新，图 22提供了一种可能的改进算法：最终得到的 $\bar{\mathbf{w}}$ 和公式 133是

算法 3.2 一种改进的平均感知器参数学习算法

输入: 训练集 $\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，最大迭代次数 T

初始化：$\boldsymbol{w} \leftarrow 0$, $\boldsymbol{u} \leftarrow 0$, $t \leftarrow 0$；
repeat
  对训练集 $\mathcal{D}$ 中的样本随机排序；
  for $n = 1 \cdots N$ **do**
    选取一个样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$；
    计算预测类别 $\hat{y}_t$；
    if $\hat{y}_t \neq y_t$ **then**
      $\boldsymbol{w} \leftarrow \boldsymbol{w} + y^{(n)}\boldsymbol{x}^{(n)}$；
      $\boldsymbol{u} \leftarrow \boldsymbol{u} + ty^{(n)}\boldsymbol{x}^{(n)}$；
    end
    $t \leftarrow t + 1$；
    if $t = T$ **then** break； // 达到最大迭代次数
  end
until $t = T$；
$\bar{\boldsymbol{w}} = \boldsymbol{w}_T - \frac{1}{T}\boldsymbol{u}$；
输出: $\bar{\boldsymbol{w}}$

图 22: 改进的平均感知器算法

   等价的。(证?)

3.5.4 扩展到多分类

两类感知器可以拓展到多分类问题，甚至是更一般的结构化学习问题。

1. 方法：输入 $\mathbf{x}$ 的特征空间 $\longrightarrow$ 输入输出联合空间的特征函数 $\phi(\mathbf{x}, \mathbf{y})$，即将样本对 $(\mathbf{x}, \mathbf{y})$ 映射到一个特征向量空间。

2. 模型：(参照 1+ 95)

$$\hat{\mathbf{y}} = \underset{y \in Gen(\mathbf{x})}{argmax}\, \mathbf{w}^T\phi(\mathbf{x}, \mathbf{y}) \tag{134}$$

   $Gen(\mathbf{x})$ 表示输入 $\mathbf{x}$ 所有输出目标的集合。当用广义感知器处理 C 分类问题时，用 C 维的 one-hot 向量 $\mathbf{y} \in \{0, 1\}^C$ 来表示类别标签 (第 c 类为 1，其余为 0，见 120)。在 C 分

类问题，一种常用的特征函数 $\phi(\mathbf{x},\mathbf{y})$ 是 $\mathbf{x}$ 和 $\mathbf{y}$ 的外积：

$$\phi(\mathbf{x},\mathbf{y}) = vec(\mathbf{x}\mathbf{y}^T) \in \mathbb{R}^{(D\times C)} \tag{135}$$

$vec(\cdot)$ 即向量化算子，$\phi(\mathbf{x},\mathbf{y})$ 即 $(D\times C)$ 维的向量。(补充知识:)

- Kronecker 积：$\mathbf{A}$ 是 $M\times N$ 矩阵，$\mathbf{B}$ 是 $S\times T$ 矩阵，它们的 Kronecter 是一个 $MS\times NT$ 矩阵：

$$[\mathbf{A}\otimes\mathbf{B}] = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} & ... & a_{1N}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} & ... & a_{2N}\mathbf{B} \\ ... & ... & ... & ... \\ a_{M1}\mathbf{B} & a_{M2}\mathbf{B} & ... & a_{MN}\mathbf{B} \end{bmatrix} \tag{136}$$

- 外积 (Outer product)：Kronecker 的一种特例，但二者不等价，$\otimes$ 二者都可以表示需要具体说明。$\mathbf{a}\in\mathbb{R}^M$ 和 $\mathbf{b}\in\mathbb{R}^N$ 的外积是一个 $M\times N$ 的矩阵，定义为：

$$\mathbf{a}\otimes\mathbf{b} = \begin{bmatrix} a_1b_1 & a_1b_2 & ... & a_1b_N \\ a_2b_1 & a_2b_2 & ... & a_2b_N \\ ... & ... & ... & ... \\ a_Mb_1 & a_Mb_2 & ... & a_Mb_N \end{bmatrix} = \mathbf{a}\mathbf{b}^T \tag{137}$$

  其中 $[\mathbf{a}\otimes\mathbf{b}]_{mn} = a_mb_n$

- 向量化 (Vectorization)：矩阵的向量化即将矩阵表示成一个列向量。令 $\mathbf{A} = [a_{ij}]_{M\times N}$，因此向量化算子定义为：

$$vec(\mathbf{A}) = [a_{11}, a_{21}, ..., a_{M1}, a_{12}, a_{22}, ..., a_{M2}, ..., a_{1N}, a_{2N}, ..., a_{MN}]^T \tag{138}$$

因此特征函数即：

$$\phi(\mathbf{x},\mathbf{y}) = \begin{bmatrix} x_1y_1 & x_1y_2 & ... & x_1y_C \\ x_2y_1 & x_2y_2 & ... & x_2y_C \\ ... & ... & ... & ... \\ x_Dy_1 & x_Dy_2 & ... & x_Dy_C \end{bmatrix} = vec(\mathbf{x}\mathbf{y}^T) \in \mathbb{R}^{D\times C} \tag{139}$$

因此给定样本 $(\mathbf{x},\mathbf{y})$ 若 $\mathbf{x}\in\mathbb{R}^D$，由于 $\mathbf{y}$ 为第 c 维为 1 的 one-hot 向量因此：

$$\phi(\mathbf{x},\mathbf{y}) = [...0, x_1, ..., x_D, 0, ...]^T \tag{140}$$

其中 x_1 即第 $c\times D - D + 1 = (c-1)\times D + 1$ 行；x_D 即第 $c\times D - D + D = cD$ 行。

3. 算法即图 23：

4. 收敛性：

- 广义线性可分 (是公式 96处多类线性可分的拓展 $0 \longrightarrow \gamma$)：对于训练集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)},\mathbf{y}^{(n)})\right\}_{n=1}^N$，如果存在一个正的常数 γ 和权重向量 $\mathbf{w}^*$，且 $\|\mathbf{w}^*\| = 1$，对所有 n 都满足 $\left\langle \mathbf{w}^*, \phi(\mathbf{x}^{(n)},\mathbf{y}^{(n)})\right\rangle - \left\langle \mathbf{w}^*, \phi(\mathbf{x}^{(n)},\mathbf{y})\right\rangle \geq \gamma$，其中 $\mathbf{y}\neq\mathbf{y}^{(n)}$，$\phi(\mathbf{x}^{(n)},\mathbf{y}^{(n)})$ 为样本的联合特征向量，那么训练集 $\mathcal{D}$ 在联合特征向量空间上线性可分。

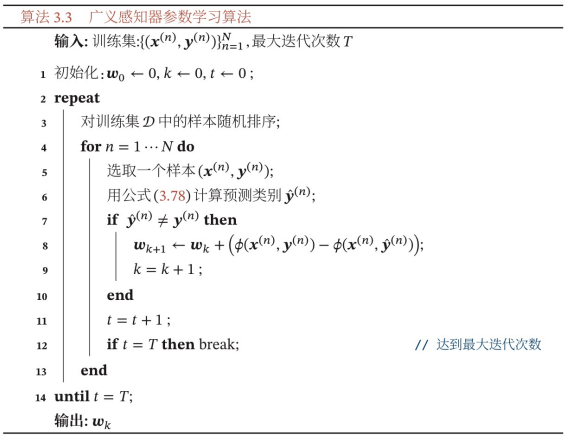


图 23: 广义感知器参数学习算法

- 广义感知器的收敛性：如果一个训练集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\right\}_{n=1}^{N}$ 是线性可分的，并令 R 是所有样本中**真实标签**和**错误标签**在特征空间 $\phi(\mathbf{x}, \mathbf{y})$ 的最远距离：

$$R = \max_{n} \max_{\mathbf{z} \neq \mathbf{y}^{(n)}} \left\| \phi(\mathbf{x}^{(n)}, \mathbf{y}^{(n)}) - \phi(\mathbf{x}^{(n)}, \mathbf{z}) \right\| \tag{141}$$

那么广义感知器参数学习算法权重更新的次数不超过 $\frac{R^2}{\gamma^2}$。(证–和两类感知器类似)

3.6 支持向量机 (Support Vector Machine, SVM)

支持向量机是一个经典的二分类算法，其找到的分割平面具有很好的鲁棒性，具有很强的优势。(**注：本节不使用增广特征和增广权重向量**)

3.6.1 寻找目标函数

给定一个二分类器数据集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)}, y^{(n)})\right\}_{n=1}^{N}$，其中 $y_n \in \{+1, -1\}$，根据定义，如果两类样本是线性可分的，即存在一个超平面：

$$\mathbf{w}^T \mathbf{x} + b = 0 \tag{142}$$

对每一个样本都有 $y^{(n)}(\mathbf{w}^T \mathbf{x}^{(n)} + b) > 0$。数据集 $\mathcal{D}$ 中每个样本 $\mathbf{x}^{(n)}$ 到分割超平面的距离 (此处指无向距离即公式 90加上绝对值)：

$$\gamma^{(n)} = \frac{\left| \mathbf{w}^T \mathbf{x}^{(n)} + b \right|}{\|\mathbf{w}\|} \tag{143}$$

为了计算方便且由于 $y^{(n)} \mathbf{w}^T \mathbf{x} + b > 0$，可去掉绝对值：

$$\gamma^{(n)} = \frac{y^{(n)}(\mathbf{w}^T \mathbf{x}^{(n)} + b)}{\|\mathbf{w}\|} \tag{144}$$

SVM 的目标即找一条线使得离该线最近的点能最远。定义间隔 (Margin) γ 为整个数据集 D 中所有样本到分割平面的**最短**距离：

$$\gamma = \min_{n} \gamma^{(n)} \tag{145}$$

如果间隔 γ 越大，说明分割超平面对两个数据集的划分越稳定，越不容易受到噪声因素的影响，因此 SVM 的目标即寻找一个最优超平面 $(\mathbf{w}^*, b^*)$ 使得 γ **最大**:

$$\begin{aligned} \underset{\mathbf{w},b}{max} \quad & \gamma \\ \text{s.t.} \quad & \frac{y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)}+b)}{\|\mathbf{w}\|} \geq \gamma, \qquad \forall n \in \{1,...,N\} \end{aligned} \tag{146}$$

我们想优化参数调整样本到分割超平面的距离，然而同时缩放 $\mathbf{w} \to k\mathbf{w}$ 和 $b \to kb$ 不会改变，因此限制 $\|\mathbf{w}\| \cdot \gamma = 1$ (间隔 $\gamma = \frac{1}{\|\mathbf{w}\|}$)，那么以上公式就等价于:

$$\begin{aligned} \underset{\mathbf{w},b}{max} \quad & \frac{1}{\|\mathbf{w}\|^2} \\ \text{s.t.} \quad & y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)}+b) \geq 1, \qquad \forall n \in \{1,...,N\} \end{aligned} \tag{147}$$

数据集中所有满足 $y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)}+b) = 1$ 的样本点都称为 **支持向量 (Support Vector)** (即到分割超平面最近的点，优化目标只用关注这些最近的点到分割超平面最远即可。) 对于一个线性可分的数据集，分割平面有很多个，然而间隔最大的超平面是唯一的。支持向量机的最大间隔超平面如图 24所示，轮廓线加粗的样本点为支持向量。

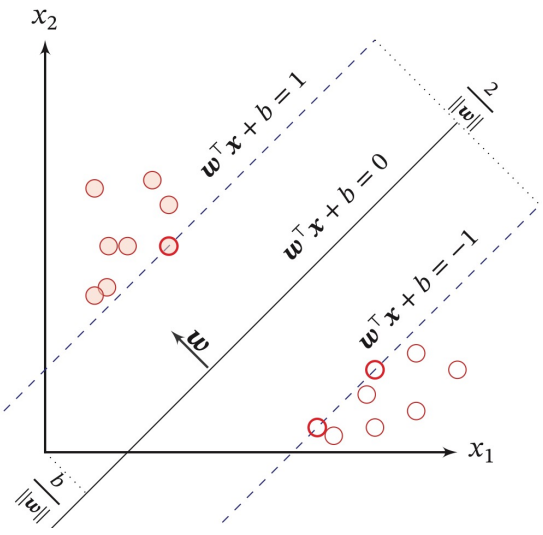


图 24: SVM 示例

3.6.2 参数学习

为了找到最大间隔分割超平面，将公式 147改写成凸优化 (浅补充：凸函数、凸优化、仿射函数) 问题:

$$\begin{aligned} \underset{\mathbf{w},b}{min} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{s.t.} \quad & 1 - y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)}+b) \leq 0, \qquad \forall n \in \{1,...,N\} \end{aligned} \tag{148}$$

($\frac{1}{2}\|\mathbf{w}\|^2$：优化的习惯，对 $\mathbf{w}$ 求偏导只剩 $\mathbf{w}$ 方便计算。)
(补充知识：拉格朗日乘数法 (Lagrange Multiplier) 与 KKT 条件)

约束优化问题可以表示为：

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & h_m(\mathbf{x}) = 0, \qquad m = 1, ..., M \\ & g_n(\mathbf{x}) \leq 0, \qquad n = 1, ..., N \end{aligned} \tag{149}$$

$h_m(\mathbf{x})$ 为等式约束函数，$g_n(\mathbf{x})$ 为不等式约束函数，$\mathbf{x}$ 的可行域：

$$\mathcal{D} = dom(f) \cap \bigcap_{m=1}^{M} dom(h_m) \cap \bigcap_{n=1}^{N} dom(g_n) \subseteq \mathbb{R}^D$$

其中 $dom(f)$ 即函数 f 的定义域。

1. 等式约束优化问题：如果只有等式约束那么构造拉格朗日函数

$$\Lambda(\mathbf{x}, \lambda) = f(\mathbf{x}) + \sum_{m=1}^{M} \lambda_m h_m(\mathbf{x}) \tag{150}$$

λ 为拉格朗日乘数，可正可负，如果 $f(\mathbf{x}^*)$ 是原始约束优化问题的局部最优值，那么存在一个 λ^* 使得 $(\mathbf{x}^*, \lambda^*)$ 为拉格朗日函数的驻点。因此令 $\frac{\partial \Lambda}{\partial \mathbf{x}} = 0$，$\frac{\partial \Lambda}{\partial \lambda} = 0$ 得到：

$$\nabla f(\mathbf{x}) + \sum_{m=1}^{M} \lambda_m \nabla h_m(\mathbf{x}) = 0 \tag{151}$$

$$h_m(\mathbf{x}) = 0, \qquad \forall m = 1, ..., M \tag{152}$$

拉格朗日乘数法即将一个有 D 个变量和 M 个等式约束条件的最优化问题转为一个有 $D+M$ 个变量的函数求驻点问题，所求驻点会包含原问题的最小解但不保证每一个驻点都是原问题的最小解，需要验证。

2. 不等式约束优化问题 *：对于 149定义的一般约束优化问题，其拉格朗日函数：

$$\Lambda(\mathbf{x}, \mathbf{a}, \mathbf{b}) = f(\mathbf{x}) + \sum_{m=1}^{M} a_m h_m(\mathbf{x}) + \sum_{n=1}^{N} b_n g_n(\mathbf{x}) \tag{153}$$

其中，$\mathbf{a} = [a_1, ..., a_M]^T$ 为等式约束的拉格朗日乘数；$\mathbf{b} = [b_1, ..., b_N]^T$ 为不等式约束的拉格朗日乘数。原始约束优化问题等价于：

$$\begin{aligned} \min_{\mathbf{x}} \max_{\mathbf{a},\mathbf{b}} \quad & \Lambda(\mathbf{x}, \mathbf{a}, \mathbf{b}) \\ \text{s.t.} \quad & \mathbf{b} \geq 0 \end{aligned} \tag{154}$$

- 主问题 (Primal Problem)：即这个 min-max 问题，先调整拉格朗日算子 $\mathbf{a}, \mathbf{b}$ 使 Λ 最大，变成主目标，再调整 x 使主目标 (Primal Object) $\min_{\mathbf{x}} \Lambda$ 最小。
- 主可行 (Primal feasible)：即约束条件满足时 $(g_n(\mathbf{x}) \leq 0)$，此时主目标等于原凸优化问题的目标函数，主目标的最小值等价于原凸优化问题的最小值；否则不是主可行的 $(g_n(\mathbf{x}) \geq 0)$，此时 $\max_{\mathbf{a},\mathbf{b}} \Lambda = \infty$ 那么 $f(\mathbf{x}) + \max_{\mathbf{a},\mathbf{b}} \Lambda = \infty$。
- 对偶问题：主问题的优化比较困难，可以通过交换 min-max 顺序来简化。定义拉格朗日对偶函数为：

$$\Gamma(\mathbf{a}, \mathbf{b}) = \inf_{\mathbf{x} \in \mathcal{D}} \Lambda(\mathbf{x}, \mathbf{a}, \mathbf{b}) \tag{155}$$

(inf-infimum 下确界：对于函数 $y=f(x)$，在使 $f(x)$ 大于等于 M 成立的所有常数 M 中，把 M 的最大值 $max(M)$ (即函数 $y=f(x)$ 的最小值）叫做函数 $y=f(x)$ 的下确界。) Γ 是一个凹函数，当 $\mathbf{b}\geq 0$ 时，对于任意的 $\tilde{\mathbf{x}}\in\mathcal{D}$ 有：

$$\Gamma(\mathbf{a},\mathbf{b})=\inf_{\mathbf{x}\in\mathcal{D}}\Lambda(\mathbf{x},\mathbf{a},\mathbf{b})\leq\Lambda(\mathbf{x},\mathbf{a},\mathbf{b})\leq f(\tilde{\mathbf{x}}) \tag{156}$$

令 $\mathbf{p}^*$ 是原问题的最优值，则有：

$$\Gamma(\mathbf{a},\mathbf{b})\leq\mathbf{p}^* \tag{157}$$

即拉格朗日对偶函数为原问题的最优下界。($max(M)$?)

- 拉格朗日对偶问题 (Lagrange Dual Problem)：优化拉格朗日对偶函数 $\Gamma(\mathbf{a},\mathbf{b})$ 并得到原问题的最优下界即拉格朗日对偶问题。

$$\begin{aligned}\max_{\mathbf{a},\mathbf{b}}\quad&\Gamma(\mathbf{a},\mathbf{b})\\ \text{s.t.}\quad&\mathbf{b}\geq 0\end{aligned} \tag{158}$$

拉格朗日对偶函数 $\Gamma(\mathbf{a},\mathbf{b})$ 为凹函数，因此拉格朗日对偶问题为凸优化问题。令 $\mathbf{d}^*$ 为拉格朗日对偶问题的最优值。

  - 弱对偶 (Weak Duality)：

$$\mathbf{d}^*\leq\mathbf{p}^* \tag{159}$$

  - 强对偶 * (Strong Duality)-KKT 条件 (Karush-Kuhn-Tucker Condition)：

$$\mathbf{d}^*=\mathbf{p}^* \tag{160}$$

当强对偶成立时，令 $\mathbf{x}^*$ 和 $\mathbf{a}^*$，$\mathbf{b}^*$ 分别为原问题和对偶问题的最优解，它们满足 KKT 条件：

$$\nabla f(\mathbf{x}^*)+\sum_{m=1}^{M}a_m^*\nabla h_m(\mathbf{x}^*)+\sum_{n=1}^{N}b_n^*\nabla g_n(\mathbf{x}^*)=0 \tag{161}$$

$$h_m(\mathbf{x}^*)=0,\qquad m=1,...,M \tag{162}$$

$$g_n(\mathbf{x}^*)\leq 0,\qquad n=1,...,N \tag{163}$$

$$b_n^*g_n(\mathbf{x}^*)=0,\qquad n=1,...,N \tag{164}$$

$$b_n^*\geq 0,\qquad n=1,...,N \tag{165}$$

KKT 条件是拉格朗日乘数法在不等式约束优化问题的泛化，当原问题为凸优化问题时，满足 KKT 条件的解也是原问题和对偶问题的最优解。在 KKT 条件中最需要关注的即公式 164，称为 **互补松弛 (Complementary Slackness)** 条件。如果最优解出现在不等式约束边界上即 $g_n(\mathbf{x})=0$，则 $b_n^*>0$；如果最优解在不等式的内部即 $g_n(\mathbf{x})<0$，那么 $b_n^*=0$，此时约束失效。

将以上知识运用到 SVM 寻找最优参数的问题中，那么使用拉格朗日乘数法，公式 148的拉格朗日函数为：

$$\Lambda(\mathbf{w},b,\lambda)=\frac{1}{2}\|\mathbf{w}\|^2+\sum_{n=1}^{N}\lambda_n(1-y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)}+b)) \tag{166}$$

其中，$\lambda_1, ..., \lambda_N \geq 0$ 为拉格朗日乘数。由于支持向量机的主问题为凸优化问题，满足强对偶性即 160，因此可以通过交换 min-max 顺序来求解：

$$\min_{\mathbf{w},b} \max_{\lambda} \Lambda(\mathbf{w}, b, \lambda) \longrightarrow \max_{\lambda} \min_{\mathbf{w},b} \Lambda(\mathbf{w}, b, \lambda) \tag{167}$$

因此第一步需要 $\min_{\mathbf{w},b} \Lambda(\mathbf{w}, b, \lambda)$：

$$\frac{\partial \Lambda}{\partial \mathbf{w}} = 0 \tag{168}$$

$$\frac{\partial \Lambda}{\partial b} = 0 \tag{169}$$

得到：

$$\begin{aligned} \mathbf{w} &= \sum_{n=1}^{N} \lambda_n y^{(n)} \mathbf{x}^{(n)} \\ 0 &= \sum_{n=1}^{N} \lambda_n y^{(n)} \end{aligned} \tag{170}$$

将上式带入拉格朗日函数 166得到拉格朗日对偶函数：

$$\Gamma(\lambda) = -\frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} y^{(m)} y^{(n)} (\mathbf{x}^{(m)})^T \mathbf{x}^{(n)} + \sum_{n=1}^{N} \lambda_n \tag{171}$$

第二步：

$$\begin{aligned} \max_{\lambda} \quad & \Gamma(\lambda) \\ \text{s.t.} \quad & \lambda \geq 0 \\ & \sum_{n=1}^{N} \lambda_n y^{(n)} = 0 \end{aligned} \tag{172}$$

由于最大化对偶函数是一个凸优化问题，可以通过多种凸优化方法来求解，得到最优值 λ^*，但由于其约束条件的数量为训练样本的数量，一般的优化方法代价比较高，因此在实践中采用如序列最小化算法 (Sequential Minimal Optimization, SMO) 等。根据 KKT 条件，令 $\mathbf{w}^*$, b^* 和 λ^* 分别为原问题和对偶问题的最优解：

$$\frac{\partial \Lambda}{\partial \mathbf{w}} = 0, \quad \frac{\partial \Lambda}{\partial b} = 0, \quad \frac{\partial \Gamma(\lambda)}{\partial \lambda} = 0 \tag{173}$$

$$1 - y^{(n)} (\mathbf{w}^{*T} \mathbf{x}^{(n)} + b^*) \leq 0 \tag{174}$$

$$\lambda_n^* (1 - y^{(n)} (\mathbf{w}^{*T} \mathbf{x}^{(n)} + b^*)) = 0 \tag{175}$$

$$\lambda_n^* \geq 0 \tag{176}$$

由于互补松弛条件即 175，由此看出：

- 如果样本 $\mathbf{x}$ 不在约束边界上那么 $\lambda_n^* = 0$，约束失效
- 如果样本在约束边界上，$\lambda_n^* \geq 0$，在边界上的点即支持向量 (Support Vector) 也就是离决策平面最近的点。

因此在计算出 λ_n^* 之后，根据 170得到最优权重 $\mathbf{w}^*$。最优偏置 b^* 的计算方式可以任选一个支持向量 $(\tilde{\mathbf{x}}, \tilde{y})$：

$$b^* = \tilde{y} - \mathbf{w}^{*T}\tilde{\mathbf{x}} \tag{177}$$

(详见例)
因此最优参数的决策函数为：

$$\begin{aligned} f(\mathbf{x}) &= sgn(\mathbf{w}^{*T}\mathbf{x} + b^*) \\ &= sgn(\sum_{n=1}^{N} \lambda_n^* y^{(n)} (\mathbf{x}^{(n)})^T \mathbf{x} + b^*) \end{aligned} \tag{178}$$

支持向量机的决策函数只依赖 $\lambda_n^* > 0$ 的样本点即支持向量 (由于此时 $1 - y^{(n)}(\mathbf{w}^{*T}\mathbf{x}^{(n)} + b^*) = 0$)，与训练样本总数无关，因此分类速度比较快。另外如上文所提，支持向量机的目标函数可以通过 SMO 等优化方法得到全局最优解，因此比其他分类器的学习效率更高。

3.6.3 软间隔 (Soft Margin)

在支持向量机的优化过程中要求最近的样本的点到分割超平面的距离至少是 1，这一约束条件比较严格，如果训练集中的样本在特征空间中不是线性可分的，无法找到最优解，为了容忍部分不满足约束的样本，引入了**松弛变量 (Slack Variable)** ξ，引入松弛变量的间隔称为软间隔。因此优化问题由 148变成：

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{n=1}^{N}\xi_n \\ \text{s.t.} \quad & 1 - y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)} + b) - \xi_n \le 0, \qquad \forall n \in \{1, ..., N\} \\ & \xi_n \ge 0, \qquad \forall n \in \{1, ..., N\} \end{aligned} \tag{179}$$

- C：参数 $C > 0$ 用来控制间隔和松弛变量惩罚的平衡 (C 越小，说明容忍度高，ξ_n 可以较大；反之容忍程度低，松弛变量应该较小)。
- ξ_n：如果分类正确且支持向量到分割超平面的距离为 1，那么 $\xi_n = 0$；如果间隔小于 1 那么至少要保证分类正确即 $y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)} + b) > 0$，因此 $\xi_n \in [0, 1)$。

公式 179也可以表示成“经验风险 + 正则化”的形式：

$$\min_{\mathbf{w},b} \quad \sum_{n=1}^{N} max(0, 1 - y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)} + b)) + \frac{1}{2C}\|\mathbf{w}\|^2 \tag{180}$$

- $max(0, 1 - y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)} + b))$：即 Hinge 损失函数 (Hinge Loss Function) 见 26
- $\frac{1}{2C}\|\mathbf{w}\|^2$：正则化项；$\frac{1}{2C}$ 是正则化系数。

软间隔支持向量机的参数学习和原始支持向量机类似，其最终决策函数也只和支持向量有关，即在约束边界上，满足 $1 - y^{(n)}(\mathbf{w}^T\mathbf{x}^{(n)} + b) - \xi_n = 0$ 的样本。

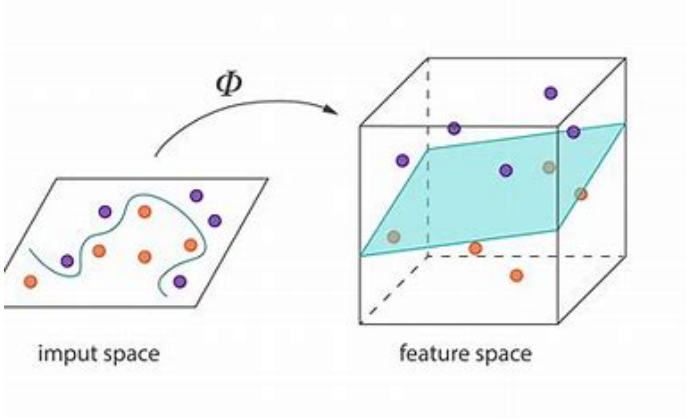


图 25: 核函数

3.6.4 核函数 (Kernel Function)

支持向量机还有一个重要的优点即可以利用核函数隐式地将样本从原始特征空间映射到更高维的空间，并解决原始空间中线性不可分的问题，图 25直观地展示了这一过程：比如在一个特征空间 ϕ 中，支持向量机的决策函数为：

$$\begin{aligned} f(\mathbf{x}) &= sgn(\mathbf{w}^{*T}\phi(\mathbf{x}) + b^*) \\ &= sgn(\sum_{n=1}^{N} \lambda_n^* y^{(n)} k(\mathbf{x}^{(n)}, \mathbf{x}) + b^*) \end{aligned} \tag{181}$$

其中 $k(\mathbf{x}^{(n)}, \mathbf{z}) = \phi(\mathbf{x})^T\phi(\mathbf{z})$ 即核函数，通常不需要给出 $\phi(\mathbf{x})$ 的具体形式，可以通过核技巧 (Kernel Trick) 来构造，例如以 $\mathbf{x}, \mathbf{z} \in \mathbb{R}^2$ 为例，可以构造一个核函数：

$$k(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^T\mathbf{z})^2 = \phi(\mathbf{x})^T\phi(\mathbf{z}) \tag{182}$$

以此来隐式地计算 $\mathbf{x}, \mathbf{z}$ 在特征空间中的内积，其中根据 $\phi(\mathbf{x})$ 的具体形式 183，可以推导出公式 182 ：

$$\phi(\mathbf{x}) = [1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2]^T \tag{183}$$

也就是说，为了计算简便，相当于先在低维特征空间内积再映射到高维，而非先映射到高维再内积，二者之间需要运用核技巧来等价。公式 182是多项式核函数的一种，其他核函数如图 26所示：

各种核函数与它们的计算公式

核函数	计算公式
线性核	$K(\mathrm{x}_i,\mathrm{x}_j) = \mathrm{x}_i^{\mathrm{T}}\mathrm{x}_j$
多项式核	$K(\mathrm{x}_i,\mathrm{x}_j) = (\gamma\mathrm{x}_i^{\mathrm{T}}\mathrm{x}_j + b)^d$
径向基函数核/高斯核	$K(\mathrm{x}_i,\mathrm{x}_j) = \exp(-\gamma\left\|\mathrm{x}_i - \mathrm{x}_j\right\|^2)$
sigmoid 核	$K(\mathrm{x}_i,\mathrm{x}_j) = \tanh(\gamma\mathrm{x}_i^{\mathrm{T}}\mathrm{x}_j + b)$

图 26: 核函数种类

3.7 三种二分类模型损失函数对比

3.7.1 公式改写

本章节介绍了三种二分类模型：Logistic 回归、感知器和支持向量机，虽然它们的决策函数相同 (89) 但由于使用了不同的损失函数以及相应的优化方法，导致它们在实际任务上的表现存在差异。为了比较这些损失函数，将它们统一定义：

- 类别标签：$y \in \{+1, -1\}$
- 线性判别函数：$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T\mathbf{x} + b$，根据两类线性可分的定义 92，$yf(\mathbf{x}; \mathbf{w}) > 0$ 分类正确，反之分类错误。
- 损失函数：都表述为定义在 $yf(\mathbf{x}; \mathbf{w})$ 上的函数

重新整理损失函数：

1. Logistic 损失函数：为适合 logistic 的描述习惯，原本的真实类别标签代表样本的真实条件概率，用 one-hot 向量来表示。

$$\mathcal{L}_{LR} = -(y^{(n)} \log \hat{y}^{(n)} + (1 - y^{(n)}) \log (1 - \hat{y}^{(n)})) \tag{184}$$

   由于统一定义，此处用指示函数 93进行替换：

$$\mathcal{L}_{LR} = -I(y = 1) \log \sigma(f(\mathbf{x}; \mathbf{w})) - I(y = -1) \log (1 - \sigma(f(\mathbf{x}; \mathbf{w}))) \tag{185}$$

   由于 sigmoid 函数的性质：$1 - \sigma(x) = \sigma(-x)$，上式进一步化简：

$$\mathcal{L}_{LR} = -I(y = 1) \log \sigma(f(\mathbf{x}; \mathbf{w})) - I(y = -1) \log \sigma(-f(\mathbf{x}; \mathbf{w})) \tag{186}$$

   由于统一定义 $y \in \{+1, -1\}$，上式可以表示为：

$$\mathcal{L}_{LR} = -\log \sigma(yf(\mathbf{x}; \mathbf{w})) = \log (1 + exp(-yf(\mathbf{x}; \mathbf{w}))) \tag{187}$$

2. 将平方损失函数 (来自 17) 重写：

$$\begin{aligned} \mathcal{L}_{squared} =& (y - f(\mathbf{x}; \mathbf{w}))^2 \\ =& 1 - 2yf(\mathbf{x}; \mathbf{w}) + (yf(\mathbf{x}; \mathbf{w}))^2 \\ =& (1 - yf(\mathbf{x}; \mathbf{w}))^2 \end{aligned} \tag{188}$$

3. 感知器损失函数 (来自 127)：

$$\mathcal{L}_p = max(0, -yf(\mathbf{x}; \mathbf{w})) \tag{189}$$

4. 软间隔支持向量机的损失函数 (来自 179)：

$$\mathcal{L}_{hinge} = max(0, 1 - yf(\mathbf{x}; \mathbf{w})) \tag{190}$$

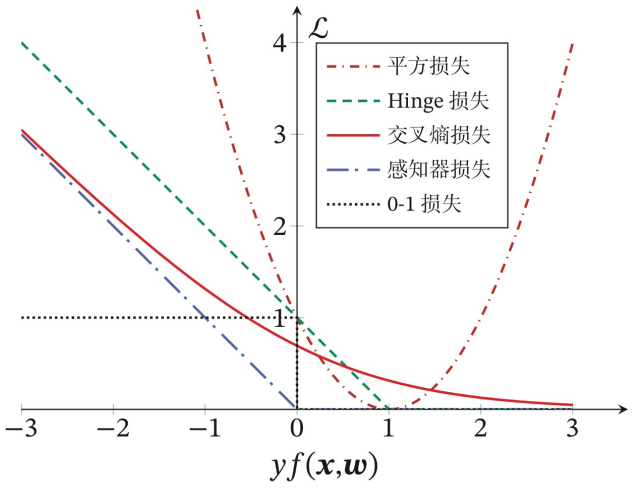


图 27: 不同损失函数的对比

3.7.2 图像分析

改写完毕后，这几类损失函数的对比如图 27所示。首先需要明确对于定义在 $yf(\mathbf{x};\mathbf{w})$ 上的损失函数，$yf(\mathbf{x};\mathbf{w})$ 和分类正确程度成正比。

- 平方损失：结合 188可知，在对称轴 1 的左侧，随着 $yf(\mathbf{x};\mathbf{w})$ 的增大，$\mathcal{L}$ 在减小说明优化效果越来越好；然而在对称轴的右侧确相反，这与分类准则相悖，因此也说明了平方损失函数不适合分类问题。
- 交叉熵损失：随着 $yf(\mathbf{x};\mathbf{w})$ 的增大，$\mathcal{L}$ 在减小，然而对于 $yf(\mathbf{x};\mathbf{w}) > 0$ 依然有损失惩罚，可以优化让损失函数最小化，然而这对只想完成二分类的问题是没有必要的。
- 0-1 损失：如果仅仅只想达成二分类的目标，0-1 损失函数是最直接的方式，$yf(\mathbf{x};\mathbf{w}) > 0$ 则有惩罚，反之则无。然而其不连续无法优化。
- 感知器损失：在 $yf(\mathbf{x};\mathbf{w}) < 0$ 时有惩罚但随着 $yf(\mathbf{x};\mathbf{w})$ 的增大，$\mathcal{L}$ 在减小，说明参数在逐渐优化，当全部分类正确即 $yf(\mathbf{x};\mathbf{w}) > 0$ 时无惩罚。
- Hinge 损失：Hinge 损失函数可以看作是感知器损失函数向右平移了一个单位，当分类错误时，与感知器模式相同；然而当 $yf(\mathbf{x};\mathbf{w}) > 0$ 时，对于感知器来说只需要分类正确就没有损失；然而 Hinge 损失函数更严格即使分类正确，如果 $yf(\mathbf{x};\mathbf{w}) < 1$ 即得分不超过 1 意味着支持向量 (离分割超平面最近的点) 离分割超平面的距离小于 1，此时依然有损失。因此软间隔支持向量机分类更稳定，鲁棒性更强。

3.8 总结

本章主要介绍了分类问题，即与第二章回归问题不同，分类问题的目标标签 y 是离散的类别标签，因此分类问题的鞠策函数需要输出离散值或者标签的后验概率。线性分类模型一般是一个广义的线性函数即一个或多个 **线性判别函数**加上**一个非线性激活函数**。所谓“线性”即决策边界由一个或多个超平面组成。图 28给出了几种常见的线性模型的比较。需要注意的是类别标签的表示形式不同：

- Logistic 回归和 Softmax 回归：$\mathbf{y}$ 为类别的 one-hot 向量
- 感知器和支持向量机：$y \in \{+1, -1\}$

线性模型	激活函数	损失函数	优化方法
线性回归	-	$(y - \boldsymbol{w}^\top \boldsymbol{x})^2$	最小二乘、梯度下降
Logistic 回归	$\sigma(\boldsymbol{w}^\top \boldsymbol{x})$	$\boldsymbol{y} \log \sigma(\boldsymbol{w}^\top \boldsymbol{x})$	梯度下降
Softmax 回归	$\text{softmax}(\boldsymbol{W}^\top \boldsymbol{x})$	$\boldsymbol{y} \log \text{softmax}(\boldsymbol{W}^\top \boldsymbol{x})$	梯度下降
感知器	$\text{sgn}(\boldsymbol{w}^\top \boldsymbol{x})$	$\max(0, -y\boldsymbol{w}^\top \boldsymbol{x})$	随机梯度下降
支持向量机	$\text{sgn}(\boldsymbol{w}^\top \boldsymbol{x})$	$\max(0, 1 - y\boldsymbol{w}^\top \boldsymbol{x})$	二次规划、SMO 等

图 28: 常见的线性模型对比

其中，

- Logistic 回归是一种概率模型，可以通过 Logistic 函数将实数值映射到 [0,1] 之间，还有其他函数可以达到此目的比如正态分布的累积还有概率密度函数，即 probit 函数。
- 感知器作为一种最简单的神经网络，其学习算法 (3.5.1) 也非常直观有效，核技巧 (3.6.4) 可以改进算法，解决线性不可分的问题，投票感知器可以提高其泛化能力，参数平均化更加有效且实用 (3.5.3)。同时感知器算法还可拓展到结构化学习 (3.5.4) 且同样也是收敛的。

4 前馈神经网络 (Feedforward Neural Network, FNN)

4.1 神经元 (Artificial Neuron)

- 生物结构：一个生物神经元具有多个树突和一条轴突。树突接受信息；轴突发送信息。当神经元所获得的输入信号超过某个阈值，它就处于兴奋状态，产生电脉冲；轴突末端的末梢给其他神经元的树突产生突触连接，将电脉冲信号传递给其他神经元。如图 29 所示。人工神经元的对应见 3.5。

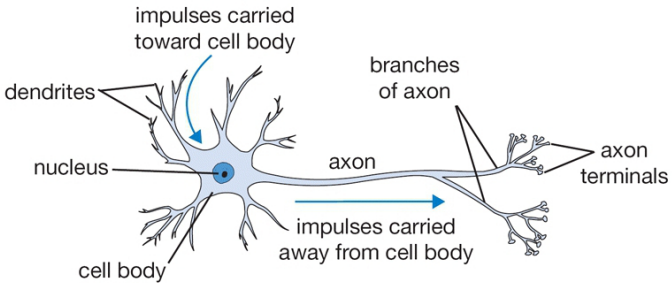


图 29: 神经元生物结构

- 神经元结构–MP 神经元：现代神经元结构和 MP 神经元的结构基本一致，区别是：MP 神经元中的激活函数 f 为 0 或 1 的阶跃函数，而现代神经元的激活函数通常是连续可

导的函数。

一个神经元的净输入：

$$z = \sum_{d=1}^{D} w_d x_d + b = \mathbf{w}^T \mathbf{x} + b \tag{191}$$

- 净输入 (Net Input)：$z \in \mathbb{R}$ 一个神经元所获得的输入信号的加权和。
- 权重：$\mathbf{w} = [w_1; w_2; ...; w_D] \in \mathbb{R}^D$
- 偏置：$b \in \mathbb{R}$

净输入 z 在经过一个非线性函数 $f(\cdot)$ 后，得到神经元的活性值 (Activation) a：

$$a = f(z) \tag{192}$$

非线性函数 $f(\cdot)$ 即激活函数 (Activation Function)。典型结构如图 30 所示，机制如图 31 所示。一个神经元是一个简单的线性模型。

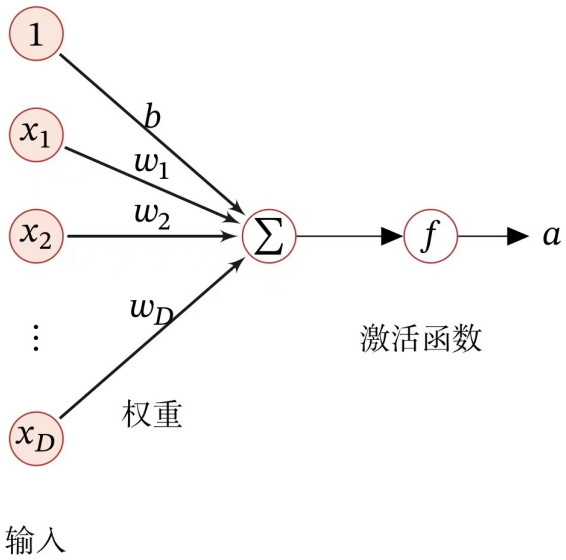


图 30: 典型神经元结构



图 31: 神经元运行机制

- 激活函数需要具备的性质：
  1. 连续可导 (允许少数点不可导) 的非线性函数，可导 $\longrightarrow$ 利用数值优化的方法来学习网络参数

2. 激活函数及其导数：尽可能简单 $\longrightarrow$ 提高网络计算效率
3. 激活函数的导函数的值域要在一个合适的区间，不能太大或太小 $\longrightarrow$ 影响训练的效率和稳定性。(结合后误差反向传播 4.4))

几种激活函数：

4.1.1 Sigmoid 型函数

Sigmoid 型函数即一类 S 型曲线函数，为两端饱和函数。常用的有 Logistic 函数和 Tanh 函数。
(补充：对于函数 $f(x)$:)

- 若 $x \longrightarrow -\infty$，$f'(x) \longrightarrow 0$ 即左饱和；
- 若 $x \longrightarrow \infty$，$f'(x) \longrightarrow 0$ 即右饱和；
- 当同时满足左右饱和则称为两端饱和。

1. Logistic 函数 (见公式 98)：即把实数值域“挤压”到 (0, 1) 之间。输入在 0 附近时，近似线性函数；输入值靠近两端：对输入进行抑制，输入越小越接近 0，反之越接近 1，即和生物神经元类似，输出 1 时即对一些输入产生兴奋；反之输出 0 即对另一些输出产生抑制。Logistic 函数性质：
   (a) 连续且可导 (导数见公式 99)，比阶跃函数的数学性质更好。
   (b) 输出可以看作是概率分布，使得神经网络可以很好地和统计学习模型结合。
   (c) 可以看作是一个软性门 (Soft gate)，用来控制其他神经元输出信息的数量。
2. Tanh 函数：也是一种 Sigmoid 型函数，定义：
$$tanh(x) = \frac{exp(x) - exp(-x)}{exp(x) + exp(-x)} \tag{193}$$
   可看做是 Logistic 函数放大平移的结果，值域是 (-1, 1)：
$$tanh(x) = 2\sigma(2x) - 1 \tag{194}$$
   Logistic 函数和 Tanh 函数形状如图 32 所示。由图可知，Tanh 是零中心化的 (Zero-Centered)，而 Logistic 是非零中心化的即输出恒大于 0，这会使得其后一层的神经元的输入发生偏置偏移 (Bias Shift)，进一步使得梯度下降速度变慢 (呈“之字型”模式下降)，具体结合后反向传播算法 4.4解释图 34。(可通过 normalization 规划以及加入偏置图 33来解决。)
3. Hard-Logistic 函数和 Hard-Tanh 函数：Sigmoid 型的函数具有饱和性，但是计算开销较大 (0 附近近似线性而两端饱和)，因此这两个函数可以用分段函数来近似。以 Logistic 函数 $\sigma(x)$ 为例，其导数即公式 99，在 0 附近的一阶泰勒展开 (Taylor expansion)：
$$g_l(x) \approx \sigma(0) + x \times \sigma'(0) = 0.25x + 0.5 \tag{195}$$

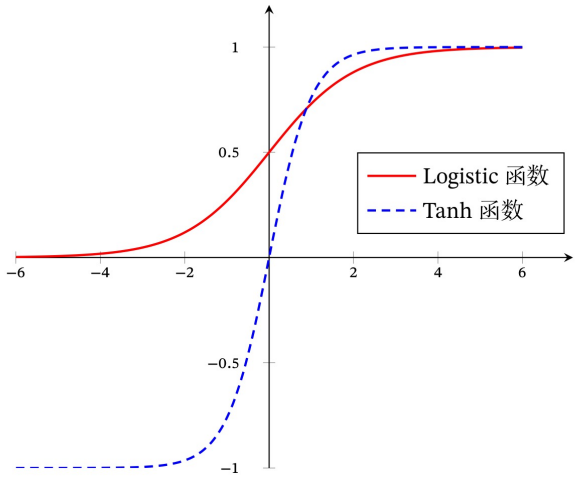


图 32: Logistic 函数和 Tanh 函数

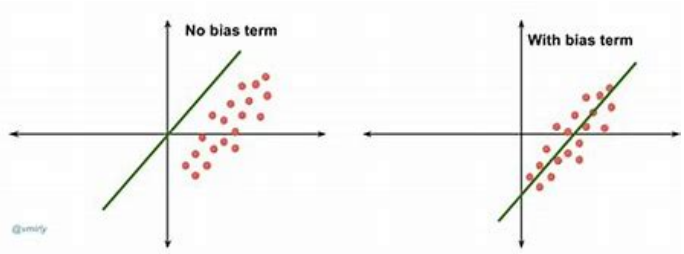


图 33: 加入偏置

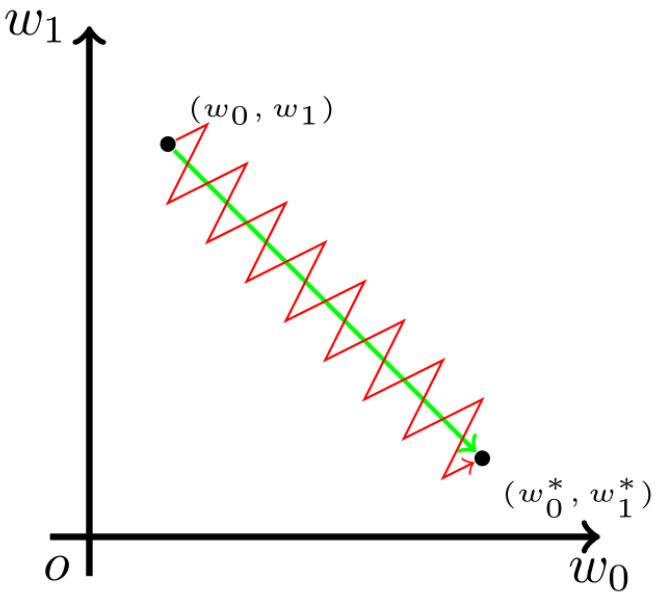


图 34: 非零中心化的影响

(Maclaurin series:)

$$\sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!} x^{(n)} = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 ... \tag{196}$$

这样的 Logistic 函数可以用分段函数 hard-logistic(x) 来近似。

$$\begin{aligned}\text{hard-logistic}(x) &= \begin{cases} 1 & g_l(x) \geq 1 \\ g_l & 0 < g_l(x) < 1 \\ 0 & g_l(x) \leq 0 \end{cases} \\ &= max(min(g_l(x), 1), 0) \\ &= max(min(0.25x + 0.5, 1), 0)\end{aligned} \tag{197}$$

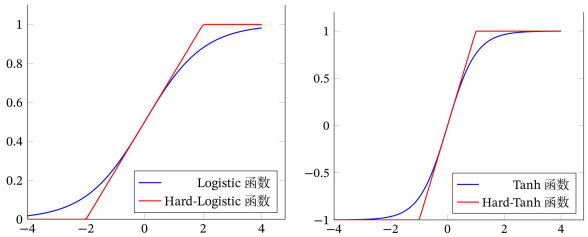
同样，Tanh 函数在 0 附近的一阶泰勒展开：

$$\begin{aligned}g_l(x) \approx & tanh(0) + x \times tanh'(0) \\ = & x\end{aligned} \tag{198}$$

同理，Tanh 函数也可用 hard-tanh(x) 近似：

$$\begin{aligned}\text{hard-tanh}(x) &= max(min(g_t(x), 1), -1) \\ &= max(min(x, 1), -1)\end{aligned} \tag{199}$$

Hard-Logistic 函数和 Hard-Tanh 函数如图 35 所示。



(a) Hard Logistic 函数　　(b) Hard Tanh 函数

图 35: Hard Sigmoid 型激活函数

4.1.2 ReLU 函数 (Rectified Linear Unit，修正线性单元)

这是目前深度神经网络中常用的激活函数，实际上是一个斜坡函数 (Ramp Function)，定义为：

$$\begin{aligned}\text{ReLU}(x) &= \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \\ &= max(0, x)\end{aligned} \tag{200}$$

优点：

1. 计算更高效：只需要进行加、乘和比较操作
2. 生物学合理性 (Biological Plausibility)：单侧抑制 ($x < 0$)，宽兴奋边界 ($ReLU(x) = x$ when $x \geq 0$–兴奋程度可以非常高)。神经网络中，同时处于兴奋状态的很稀疏 (同一时刻可能只有 1% ∼ 4% 处于活跃状态)，Sigmoid 型激活函数可能会导致一个非稀疏的神经网络，而 ReLU 有很好的稀疏性，约 50% 的神经元会处于激活状态。

3. 优化：相比 Sigmoid 两端饱和，ReLU 函数为左饱和函数，且在 $x > 0$ 时导数为 1，在一定程度上缓解了神经网络梯度消失的问题 (Sigmoid 型函数有梯度消失的问题见 4.6.2)，加速梯度下降的收敛速度。

缺点：

1. 偏置偏移：输出是非零中心化的，给后一层的神经网络引入偏置偏移，影响梯度下降的效率。

2. 死亡 ReLU 问题 (Dying ReLU Problem)：在训练时，如果参数在一次不恰当的更新后，第一个隐藏层的某个 ReLU 神经元会在所有的训练数据上都不能被激活，那么这个神经元自身参数的梯度将永远是 0，在以后的训练过程中永远不能被激活，且这种现象也可能发生在其他隐藏层。(详解：) 参数的更新方式即

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \tag{201}$$

当学习率 η 不小心设置过大时，$\frac{\partial L}{\partial w}$ 一般为正，会导致 $\eta \frac{\partial L}{\partial w}$ 过大，因此可能超过 w 此时，被更新的新参数即 $w' < 0$，因此对于输入进来的激活值 (一般为正)，如果偏置设置较小，即有 $wa + b < 0$ 的可能，而此项经过 ReLU 激活之后为 0，后续导数也为 0，就会出现 $a = max\{z, 0\} = 0, \quad \forall x_i, z = \mathbf{w} \cdot x_i < 0$ 的情况，即神经元死亡。

实际应用为了避免上述的问题，有几种 ReLU 函数的变种被广泛应用：

1. 带泄露的 ReLU (Leaky ReLU)：解决上述死亡 ReLU 问题，在输入 $x < 0$ 时，保持一个很小的梯度 γ ，这样当神经元处于非激活状态时也有一个较小的梯度保持更新，就避免了永远不能被激活的问题。定义如下：

$$\begin{aligned} \text{LeakyReLU}(x) &= \begin{cases} x & x > 0 \\ \gamma x & x \leq 0 \end{cases} \\ &= max(0, x) + \gamma min(0, x) \end{aligned} \tag{202}$$

其中，γ 是一个很小的常数，当 $\gamma < 1$，带泄露的 ReLu 也可以表示为：

$$\text{LeakyReLU}(x) = max(x, \gamma x) \tag{203}$$

相当于一个较为简单的 maxout 单元。

2. 带参数的 ReLU (Parametric ReLU, PReLU)：引入一个可学习的参数，不同神经元可以有不同的参数，对于第 i 个神经元，其 PReLU 定义为：

$$\begin{aligned} \text{PReLU}_i(x) &= \begin{cases} x & x > 0 \\ \gamma_i x & x \leq 0 \end{cases} \\ &= max(0, x) + \gamma_i min(0, x) \end{aligned} \tag{204}$$

其中，γ_i 为 $x \leq 0$ 时的斜率，因此 PReLU 是非饱和函数。

- $\gamma_i = 0$：PReLU 退化为 ReLU。
- γ_i 是一个很小的常数，PReLU 可视作 LeakyReLU。

- PReLU 可以允许不同神经元有不同的参数，也可以一组神经元共享一个参数。

3. ELU 函数 (Exponential Linear Unit，指数线性单元)：是一个近似的零中心化的非线性函数，定义为：

$$\begin{aligned} \text{ELU}(x) &= \begin{cases} x & \text{if} \quad x > 0 \\ \gamma(exp(x) - 1) & \text{if} \quad x \leq 0 \end{cases} \\ &= max(0, x) + min(0, \gamma(exp(x) - 1)) \end{aligned} \tag{205}$$

4. Softplus 函数：可看作是 Rectifier 函数的平滑版本，其定义为：

$$\text{Softplus}(x) = \log\left(1 + exp(x)\right) \tag{206}$$

Softplus 函数其导数刚好是 Logistic 函数。虽然也有单侧抑制，宽兴奋边界的特性却没有稀疏激活性。

上述 ReLU、Leaky ReLU、ELU 以及 Softplus 函数如图 36所示。

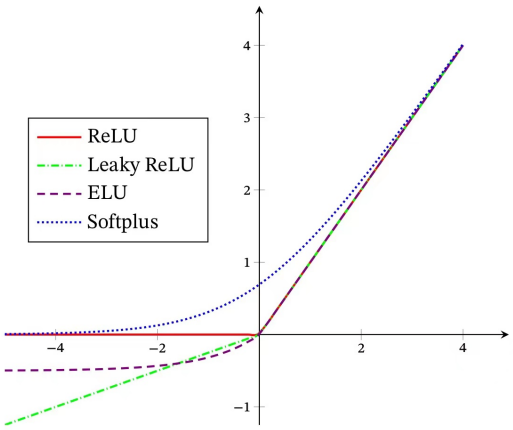


图 36: Relu、Leaky ReLU、ELU 以及 Softmax 函数

4.1.3 Swish 函数

Swish 函数是一种复合函数 (Compound Function–在设计了不同函数的组合方式并通过强化学习搜索合适的激活函数之后，发现 Swish 具有很好的性能)，是一种自门控 (Self-Gated) 激活函数，定义为：

$$swish(x) = x\sigma(\beta x) \tag{207}$$

其中，$\sigma(\cdot)$ 为 Logistic 函数，β 为可学习的参数或固定超参数。$\sigma(\cdot) \in (0, 1)$ 可看作是一种软性的门控机制：

- $\sigma(\beta x)$ 接近 1，门“开”，激活函数近似于 x 本身
- $\sigma(\beta x)$ 接近 0，门“关”，激活函数输出近似 0

Swish 函数如图 37所示。

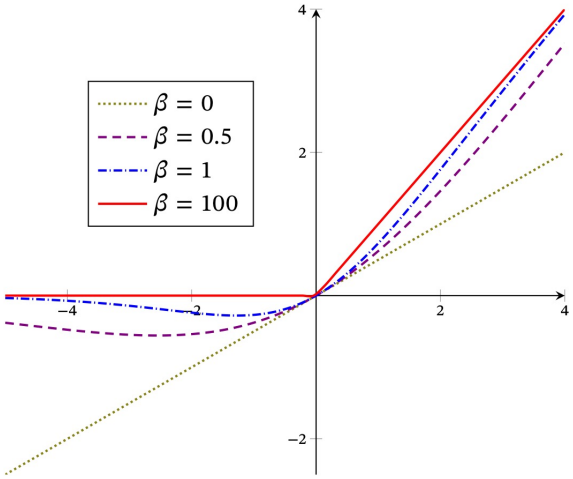


图 37: Swish 函数

- $\beta = 0$: Swish 函数变成线性函数 $x/2$
- $\beta = 1$: Swish 函数在 $x > 0$ 的时候近似线性，在 $x < 0$ 时近似饱和，同时具有一定的非单调性
- $\beta \to +\infty$: $\sigma(\beta x)$ 趋向 0-1 函数，Swish 函数近似 ReLU 函数。

因此 Swish 函数可以视为线性函数和 ReLU 函数之间的非线性插值函数，其程度由参数 β 控制。

4.1.4 GELU 函数 (Gaussian Error Linear Unit, 高斯误差线性单元)

也是通过门控机制来调整其输出的激活函数，和 Swish 函数类似:

$$\text{GELU}(x) = xP(X \leq x) \tag{208}$$

$P(X \leq x)$ 是高斯分布 $\mathcal{N}(\mu, \sigma^2)$ 的累积分布函数，其中一般设置 $\mu = 0, \sigma = 1$ ，二者为超参数。(补充: 累积分布函数–Cumulative Distribution Function, CDF) 对于一个随机变量 X，其累积分布函数是随机变量 X 的取值小于等于 x 的概率:

$$cdf(x) = P(X \leq x) \tag{209}$$

以连续随机变量 X 为例，累积分布函数定义为:

$$cdf(x) = \int_{-\infty}^{x} p(t)dt \tag{210}$$

其中，$p(x)$ 为概率密度函数见 4，标准正态分布的概率密度函数和累积分布函数如图 38所示。

由于其为 S 型函数，因此 GELU 函数可以用 Tanh 函数或者 Logistic 函数来近似:

$$\text{GELU}(x) \approx 0.5x(1 + tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3))) \tag{211}$$

$$\text{或GELU}(x) \approx x\sigma(1.702x) \tag{212}$$

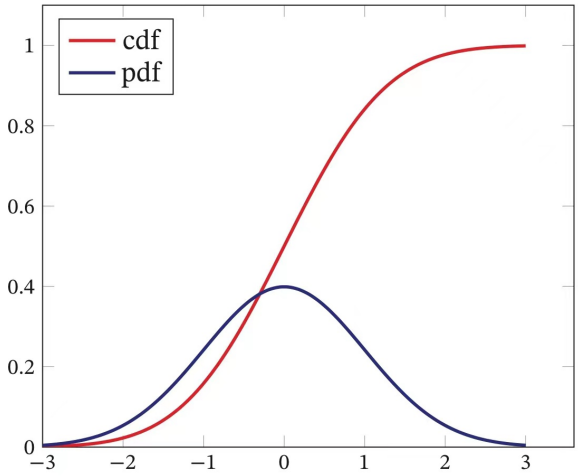


图 38: 标准正态分布的概率密度函数和累积分布函数

当用 Logistic 函数近似，GELU 函数相当于一种特殊的 Swish 函数。($\beta = 1.702$)

4.1.5 Maxout 单元

Maxout 单元也是一种分段线性函数。Sigmoid 型函数、ReLU 等激活函数的输入是一个**标量**; Maxout 单元的输入是上一层神经元的全部原始输出，是一个**向量** $\mathbf{x} = [x_1; x_2; ...; x_D]$。每个 Maxout 单元有 K 个权重向量 $w_k \in \mathbb{R}^D$ 和偏置 $b_k (1 \leq k \leq K)$。对于输入 $\mathbf{x}$，可以得到 K 个净输入 z_k:

$$z_k = \mathbf{w}_k^T \mathbf{x} + b_k \tag{213}$$

- 第 k 个权重向量: $\mathbf{w}_k = [w_{k,1}, ..., w_{k,D}]^T$

Maxout 单元的非线性函数定义为:

$$\text{maxout}(\mathbf{x}) = \underset{k \in [1,K]}{max} (z_k) \tag{214}$$

Maxout 单元并非简单的从**净输入**到输出的非线性映射，而是整体学习**输入**到输出之间的非线性映射关系。Maxout 激活函数可看作是任意凸函数的分段线性近似，并且在有限的点不可微。
常见的激活函数及其导数如图 39所示。

4.2 网络结构

通过一定的连接方式或信息传递方式进行协作的神经元可看作一个网络即神经网络。目前常用的结构有以下三种 (实际情况一个神经网络可能是多种结构的复合):

1. 前馈网络:

   - 传递方式: 各个神经元按接收信息的先后分为不同的组，每一组看成是一个神经层，每一层的神经元接受前一层神经元的输出并输出到下一层神经元。整个网络的信息

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)\big(1 - f(x)\big)$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min\big(0, \gamma(\exp(x) - 1)\big)$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log\big(1 + \exp(x)\big)$	$f'(x) = \frac{1}{1+\exp(-x)}$

图 39: 常见激活函数及其导数

朝一个方向，无反向信息传播，可用一个有向无环路图表示。前馈网络包括全连接前馈网络（ 4.3) 和卷积神经网络（ 5)

- 形式：可看作一个函数，即通过简单非线性函数的多次复合实现从输入空间到输出空间的复杂映射，这种结构简单，便于实现。

2. 记忆网络 (反馈网络)：

   - 传递方式：不但可以接收其他神经元的信息，也可以接收自己的**历史信息**。和前馈神经网络相比，记忆网络的神经元有记忆功能，在不同的时刻有不同的状态。信息传播可以是单向或双向的，可用一个有向循环图或无向图表示。记忆网络包括循环神经网络，Hopfield 网络，玻尔兹曼机，受限玻尔兹曼机等 (后面章节详细介绍)。
   - 形式：可看作是一个程序，有更强的计算和记忆能力。为了增强其记忆容量，可引入外部记忆单元和读写机制，用来保存一些网络的中间状态，称为记忆增强网络 (Memory Augmented Neural Network, MANN)，如神经图灵机和记忆网络。

3. 图网络：

   - 传递方式：定义在图结构数据上的神经网络，每个节点都由一个或一组神经元构成，节点之间的连接可以是有向或无向的，每个节点可以收到来自相邻节点或自身的信息。
   - 应用：前馈和记忆网络的输入都可以是向量或向量序列，但实际应用很多数据是图结构的数据，图网络是前馈网络和记忆网络的泛化，包含很多不同实现形式，如图卷积网络 (Graph Convolutional Network, GCN)、图注意网络 (Graph Attention Network, GAT) 、消息传递神经网络 (Message Passing Neural Network, MPNN) 等。

上述三种结构如图 40所示，其中圆形节点表示一个神经元，方形节点表示一组神经元。

4.3 前馈神经网络

前馈神经网络是最早发明的前馈神经网络，也称为全连接神经网络 (Fully Connected Neural Network, FCNN) 或多层感知器 (Multi-layer Perceptron, MLP)，但实际上，前馈神经网络是由多层 Logistic 回归模型 (**连续**的非线性函数) 组成而非多层感知器 (**不连续**的非线性函数) 组成。

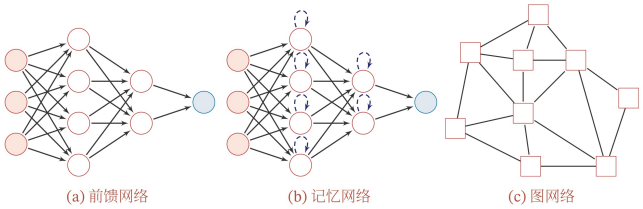


图 40: 三种不同的网络结构示例

4.3.1 结构表示以及信息传递方式

在前馈神经网络中，各神经元属于不同层，每一层接受前一层的信号继而产生信号输出到下一层。第 0 层为输入层，最后一层为输出层，中间为隐藏层，整个网络中无反馈，信号从输入到输出层单向传播，可用有向无环图表示，如图 41所示。描述前馈神经网络的记号如

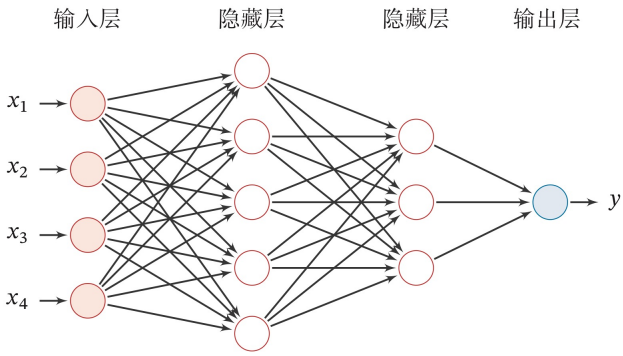


图 41: 前馈神经网络

图 42所示。注：层数 L 一般只考虑隐藏层和输入层。令 $\mathbf{a}^{(0)} = \mathbf{x}$ ，前馈神经网络通过不断迭

记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\boldsymbol{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\boldsymbol{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\boldsymbol{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$\boldsymbol{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

图 42: 前馈神经网络的记号表示

代下式进行信息传播：

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \tag{215}$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}) \tag{216}$$

即每个神经层要经过一个仿射变换 215 (Affine Transformation) 和一个非线性变换 216。

- 仿射变换：根据 $l-1$ 的**活性值 (Activation)** $\longrightarrow$ 第 l 层的**净活性值 (Net activation)** $\mathbf{z}^{(l)}$。
- 非线性变换：把**净活性值** $\mathbf{z}^{(l)}$ 放入一个激活函数 $\longrightarrow$ 第 l 层的**活性值** $\mathbf{a}^{(l)}$。

(补充知识：)

- 仿射变换即通过一个**线性变换**和**平移**，将一个向量空间转变成另一个向量空间的过程：

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b} \tag{217}$$

其中, $\mathbf{A} \in \mathbb{R}^{N\times N}$ 即 $N \times N$ 的实数矩阵，$\mathbf{x} \in \mathbb{R}^N$ 是 N 维向量空间中的点，$\mathbf{b} \in \mathbb{R}^N$ 为平移项，当 $\mathbf{b} = 0$，仿射变换退化为线性变换。公式 215 和 216也可合并写成：

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} f_{l-1}(\mathbf{z}^{(l-1)}) + \mathbf{b}^{(l)} \tag{218}$$

或者：

$$\mathbf{a}^{(l)} = f_l(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) \tag{219}$$

因此整个过程可以通过逐层信息传播得到最后的输出 $\mathbf{a}^{(L)}$，可将网络看作是一个复合函数 $\phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$：

$$\mathbf{x} = \mathbf{a}^{(0)} \to \mathbf{z}^{(1)} \to \mathbf{a}^{(1)} \to \mathbf{z}^{(2)} \to ... \to \mathbf{a}^{(L-1)} \to \mathbf{z}^{(L)} \to \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b}) \tag{220}$$

其中, 将向量 $\mathbf{x}$ 看作第一层的输入 $\mathbf{a}^{(0)}$, 将最后一层 L 的输出 $\mathbf{a}^{(L)}$ 作为整个函数的输出, $\mathbf{W}, \mathbf{b}$ 表示网络中所有层的连接权重和偏置。

4.3.2 通用近似定理 (Universal Approximation Theorem)

通用近似定理如图 43所示。通用近似定理在实数空间 $\mathbb{R}^D$ 的有界闭集依然成立，此定理说明，前馈神经网络具有很强的拟合能力，常见的非连续线性函数都可用前馈神经网络去近似。也就是说，对于具有线性输入层和至少一个使用“挤压”性质的激活函数 (Sigmoid 型，但也被证明对于 ReLU 也适用) 的隐藏层组成的前馈神经网络，只要隐藏层的数量足够，就可以以任意的精度逼近任何一个定义在实数空间 $\mathbb{R}^D$ 的有界闭集函数 (Borel 可测函数)。然而通用近似定理只是说明了神经网络的计算能力可以去近似一个给定的**连续**函数，但并未说明如何找到一个最优的网络以及如何判定是最优的。另外在应用到机器学习的过程中，不知道真实映射函数，一般通过经验风险最小化和正则化来学习参数，由于神经网络强大的拟合能力反而容易在训练集上**过拟合**。

定理 4.1 – 通用近似定理（Universal Approximation Theorem）[Cybenko, 1989; Hornik et al., 1989]：令 $\phi(\cdot)$ 是一个非常数、有界、单调递增的连续函数，$\mathcal{I}_D$ 是一个 D 维的单位超立方体 $[0,1]^D$，$C(\mathcal{I}_D)$ 是定义在 $\mathcal{I}_D$ 上的连续函数集合. 对于任意给定的一个函数 $f \in C(\mathcal{I}_D)$，存在一个整数 M，和一组实数 $v_m, b_m \in \mathbb{R}$ 以及实数向量 $\boldsymbol{w}_m \in \mathbb{R}^D$，$m = 1, \cdots, M$，以至于我们可以定义函数

$$F(\boldsymbol{x}) = \sum_{m=1}^{M} v_m \phi(\boldsymbol{w}_m^{\mathrm{T}} \boldsymbol{x} + b_m), \tag{4.37}$$

作为函数 f 的近似实现，即

$$|F(\boldsymbol{x}) - f(\boldsymbol{x})| < \varepsilon, \forall \boldsymbol{x} \in \mathcal{I}_D, \tag{4.38}$$

其中 $\varepsilon > 0$ 是一个很小的正数.

图 43: 通用近似定理

4.3.3 机器学习应用

1. 神经网络的特征抽取应用：根据通用近似定理，神经网络从某种程度上可以作为一个“万能”函数来使用，可用来进行复杂的特征转换或逼近一个复杂的条件分布。在机器学习中，输入样本的特征对分类器的影响很大，以监督学习为例，好的特征可以提高分类器的性能，因此需要将样本的原始特征向量 $\mathbf{x}$ 转换到更有效的特征向量 $\phi(\mathbf{x})$ 即**特征抽取**。(1.3.2) 多层前馈神经网络可看作是一个非线性复合函数 ϕ：$\mathbb{R}^D \to \mathbb{R}^{D'}$，即将输入 $\mathbf{x} \in \mathbb{R}^D \to \phi(\mathbf{x}) \in \mathbb{R}^{D'}$ 因此多层前馈神经网络可以看作是一种特征转换的方法，给定一个训练样本 $(\mathbf{x}, y)$，先利用多层神经网络将 $\mathbf{x}$ 映射到 $\phi(\mathbf{x})$，然后再将输出 $\phi(\mathbf{x})$ 作为分类器的输入，输入到线性或者非线性分类器进行分类。

2. 公式表示：

$$\hat{y} = g(\phi(\mathbf{x}); \theta) \tag{221}$$

3. 特殊实例：如果 $g(\cdot)$ 为 Logistic 回归或者 Softmax 回归分类器那么 $g(\cdot)$ 可以看作是网络最后一层，直接输出不同类别的条件概率 $p(y|\mathbf{x})$。

   - Logistic 回归分类器：面对二分类问题 $y \in \{0, 1\}$，Logistic 回归分类器即神经网络的最后一层的唯一一个神经元的激活函数，输出即类别 $y = 1$ 的条件概率:

   $$p(y = 1|\mathbf{x}) = a^{(L)} \tag{222}$$

   $a^{(L)} \in \mathbb{R}$ 即第 L 层神经元的活性值。

   - Softmax 回归分类器：面对多分类问题 $y \in \{1, ..., C\}$ 如果使用 Softmax 回归分类器，相当于最后一层设置 C 个神经元，激活函数都为 Softmax 函数，网络最后一层 L 的输出可作为每个类的条件概率：

   $$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)}) \tag{223}$$

     – $\mathbf{z}^{(L)} \in \mathbb{R}^C$ ： 第 L 层神经元的净输入

     – $\hat{\mathbf{y}} \in \mathbb{R}^C$ ： 第 L 层神经元的活性值，每一维分别表示不同类别标签的条件预测概率。

4.3.4 参数学习

1. 学习准则：如果采用交叉熵损失函数，对于样本 $(\mathbf{x}, y)$，$\mathbf{y} \in \{0,1\}^C$ 为标签 y 对应的 one-hot 向量表示，损失函数：

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}} \tag{224}$$

给定训练集 $\mathcal{D} = \left\{(\mathbf{x}^{(n)}, y^{(n)})\right\}_{n=1}^N$，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络输出为 $\hat{\mathbf{y}}^{(n)}$，在数据集 $\mathcal{D}$ 上的结构化风险函数：

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2}\lambda \|\mathbf{W}\|_F^2 \tag{225}$$

- $\mathbf{W}, \mathbf{b}$：所有权重和偏置向量
- $\frac{1}{2}\lambda \|\mathbf{W}\|_F^2$：正则化项，防止过拟合；$\lambda > 0$ 为超参数，越大 $\mathbf{W}$ 越接近 0，这里的 $\|\mathbf{W}\|_F^2$ 一般使用 Frobenius 范数：

$$\|\mathbf{W}\|_F^2 = \sum_{l=1}^{L}\sum_{i=1}^{M_l}\sum_{j=1}^{M_{l-1}}(w_{ij}^{(l)})^2 \tag{226}$$

2. 优化算法：网络参数可通过梯度下降法进行学习。每次迭代，第 l 层的参数 $\mathbf{W}^{(l)}$ 和 $\mathbf{b}^{(l)}$ 更新方式：

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{W}^{(l)}} \tag{227}$$

$$= \mathbf{W}^{(l)} - \alpha(\frac{1}{N}\sum_{n=1}^{N}(\frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{W}^{(l)}}) + \lambda \mathbf{W}^{(l)}) \tag{228}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{b}^{(l)}} \tag{229}$$

$$= \mathbf{W}^{(l)} - \alpha(\frac{1}{N}\sum_{n=1}^{N}(\frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{b}^{(l)}}) \tag{230}$$

梯度下降需要计算损失函数对参数的偏导数，如果通过链式法则逐一求导较为低效，经常使用反向传播算法来高效计算梯度。

4.4 反向传播算法 (Backpropagation, BP)

假设采用随机梯度下降进行参数学习，对于样本 $(\mathbf{x}, \mathbf{y})$ 的输出为 $\hat{\mathbf{y}}$，需要计算损失函数对于每个参数的导数。由于计算损失函数对于第 l 层的参数 $\mathbf{W}^{(l)}$ 设计矩阵微积分很繁琐因此先计算损失函数关于矩阵中**每个元素**的偏导，则根据链式法则：

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}}\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \tag{231}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \tag{232}$$

上述公式的第二项都是目标函数关于第 l 层神经元的偏导数，称为 **误差项**，因此共需计算三个导数：

1. $\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}}$：根据公式 215，偏导 (采用分母布局：一个列向量关于标量的偏导为行向量)：

$$\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} = [\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, ..., \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, ..., \frac{\partial z_{M_l}^{(l)}}{\partial w_{ij}^{(l)}}] \tag{233}$$

由于 $\mathbf{z}_i^{(l)} = \mathbf{w}_i^{(l)} a_j^{(l-1)} + \mathbf{b}^{(l)}$，其中 $\mathbf{w}_i^{(l)}$ 为权重矩阵 $\mathbf{W}^{(l)}$ 的第 i 行，因此：

$$\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} = [0, ..., a_j^{(l-1)}, ..., 0] \triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{1 \times M_l} \tag{234}$$

$\mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{1 \times M_l}$：第 i 个元素为 $a_j^{(l-1)}$，其余为 0 的行向量。

2. $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}$：仍然依据公式 215，偏导：

$$\frac{\mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{M_l} \in \mathbb{R}^{M_l \times M_l} \tag{235}$$

即 $M_l \times M_l$ 的单位矩阵。

3. $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$：表示第 l 层神经元对最终损失的影响，或者最终损失对第 l 层神经元的敏感程度，因此一般称为第 l 层神经元的误差项：

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{M_l} \tag{236}$$

误差项 $\delta^{(l)}$ 也间接反应了不同神经元对网络能力的贡献程度，从而较好地解决了贡献度分配问题 (Credit Assignment Problem, CAP– 1.5)。
根据 $\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$，有：

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (\mathbf{W}^{(l+1)})^T \in \mathbb{R}^{M_l \times M_{l+1}} \tag{237}$$

根据 $\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$，其中 $f_l(\cdot)$ 为按位计算的函数，因此有：(激活函数的导数)

$$\begin{aligned} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} =& \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \\ =& diag(f_l'(\mathbf{z}^{(l)})) \in \mathbb{R}^{M_l \times M_l} \end{aligned} \tag{238}$$

因此根据链式法则，第 l 层的误差：

$$\begin{aligned} \delta^{(l)} \triangleq& \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \\ =& \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ =& diag(f_l'(\mathbf{z}^{(l)})) \cdot (\mathbf{W}^{(l+1)})^T \cdot \delta^{(l+1)} \\ =& f_l'(\mathbf{z}^{(l)}) \odot ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}) \in \mathbb{R}^{M_l} \end{aligned} \tag{239}$$

其中，$\odot$ 是向量的 Hadamard 积 (逐点乘积，对应元素相乘) 运算符，表示每个元素相乘。由公式 239得到，第 l 项的误差由后一项第 $l+1$ 项得到，这就是误差的 **反向传播**，由此得出反向传播算法的含义：第 l 层的一个神经元的误差项 (或对最终损失函数的敏感度) 是所有与该神经元相连的第 $l+1$ 层误差项的权重和 $((\mathbf{W}^{(l+1)})^T \delta^{(l+1)})$ 再乘上该神经元激活函数的梯度 $f_l'(\mathbf{z}^{(l)})$。

由此得到了三个导数，因此可以得到损失函数关于某个参数的偏导，即公式 231变为：

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} =& \mathbb{I}_i(a_j^{(l-1)})\delta^{(l)} \\ =& [0, ..., a_j^{(l-1)}, ..., 0][\delta_1^{(l)}, ..., \delta_i^{(l)}, ..., \delta_{M_l}^{(l)}]^T \\ =& \delta_i^{(l)} a_j^{(l-1)}\end{aligned} \tag{240}$$

其中，$\delta_i^{(l)} a_j^{(l-1)}$ 相当于向量 $\delta^{(l)}$ 和向量 $\mathbf{a}^{(l-1)}$ 外积 (见公式 2) 的第 i, j 个元素，因此可以进一步写为：

$$[\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}}]_{ij} = [\delta^{(l)}(\mathbf{a}^{(l-1)})^T]_{ij} \tag{241}$$

因此，损失函数对于第 l 层权重 $\mathbf{W}^{(l)}$ 的梯度：

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)}(\mathbf{a}^{(l-1)})^T \in \mathbb{R}^{M_l \times M_{l-1}} \tag{242}$$

同理，损失函数对于第 l 层偏置 $\mathbf{b}$ 的梯度：

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)} \in \mathbb{R}^{M_l} \tag{243}$$

由此可知，在计算出每一层的误差项之后，就可以得到每层参数的梯度，因此用误差反向传播的前馈神经网络的训练过程分成三步：

1. **前馈计算**每一层的净输入 $\mathbf{z}^{(l)}$ 和激活值 $\mathbf{a}^{(l)}$，直到最后一层。
2. **反向传播**每一层的误差项 $\delta^{(l)}$。
3. 计算每一层参数的梯度并更新参数。

算法如图 44所示。(解释 Sigmoid 型激活函数非零中心化的影响)

算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α, 正则化系数 λ, 网络层数 L, 神经元数量 $M_l, 1 \le l \le L$.

随机初始化 $\boldsymbol{W}, \boldsymbol{b}$;
repeat
  对训练集 $\mathcal{D}$ 中的样本随机重排序;
  for $n = 1 \cdots N$ **do**
    从训练集 $\mathcal{D}$ 中选取样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$;
    前馈计算每一层的净输入 $\boldsymbol{z}^{(l)}$ 和激活值 $\boldsymbol{a}^{(l)}$, 直到最后一层;
    反向传播计算每一层的误差 $\delta^{(l)}$; // 公式 (4.63)
    // 计算每一层参数的导数
    $\forall l, \quad \frac{\partial \mathcal{L}(y^{(n)}, \hat{y}^{(n)})}{\partial \boldsymbol{W}^{(l)}} = \delta^{(l)}(\boldsymbol{a}^{(l-1)})^{\top}$; // 公式 (4.68)
    $\forall l, \quad \frac{\partial \mathcal{L}(y^{(n)}, \hat{y}^{(n)})}{\partial \boldsymbol{b}^{(l)}} = \delta^{(l)}$; // 公式 (4.69)
    // 更新参数
    $\boldsymbol{W}^{(l)} \leftarrow \boldsymbol{W}^{(l)} - \alpha(\delta^{(l)}(\boldsymbol{a}^{(l-1)})^{\top} + \lambda \boldsymbol{W}^{(l)})$;
    $\boldsymbol{b}^{(l)} \leftarrow \boldsymbol{b}^{(l)} - \alpha \delta^{(l)}$;
  end
until 神经网络模型在验证集 $\mathcal{V}$ 上的错误率不再下降;
输出: $\boldsymbol{W}, \boldsymbol{b}$

图 44: 反向传播算法

4.5 自动梯度计算

神经网络的参数主要通过梯度下降进行优化，可以手动通过链式法则计算梯度进而转换为代码但过程繁琐容易出错，因此考虑用计算机自动计算梯度无须人工干预，自动计算的方法可分为三类：数值微分、符号微分和自动微分。

4.5.1 数值微分 (Numerical Differentiation)

数值微分即用数值方法来计算函数 $f(x)$ 的导数。函数 $f(x)$ 在点 x 的导数定义为：

$$f'(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x} \tag{244}$$

即给 x 加上一个很小的非零扰动 Δx 通过上述定义来计算函数 $f(x)$ 的梯度。数值微分方法容易实现，但是对于合适扰动 Δx 的确定十分困难：

- Δx 过小：引起数值计算问题，比如**舍入误差 (Round-off Error)**。(舍入误差即数值计算过程中由于数字舍入造成**近似值和精确值**之间的差异，比如用浮点表示实数。直观解释即分子 $f(x + \Delta x) - f(x)$ 会随着 Δx 减小而减小，因此计算机**有限的精度** (字长有限) 产生的误差的影响会越来越大。)
- Δx 过大：会增加**截断误差 (Truncation Error)**，使得导数计算不准确。(截断误差即数学模型**理论解**和数值计算问题的**精确解**之间的误差，因为截断误差是数值计算方法固有的，因此又称方法误差。)
- 补充：误差分为固有误差 (模型误差–数学模型与实际问题的误差、观测误差–观测的物理量和实际值的误差) 和计算误差 (截断误差、舍入误差)。对于上述提到的计算误差，以 $f(x) = exp(5x)$ 在 $x = 1$ 处导数为例，截断误差和舍入误差如图 45所示。

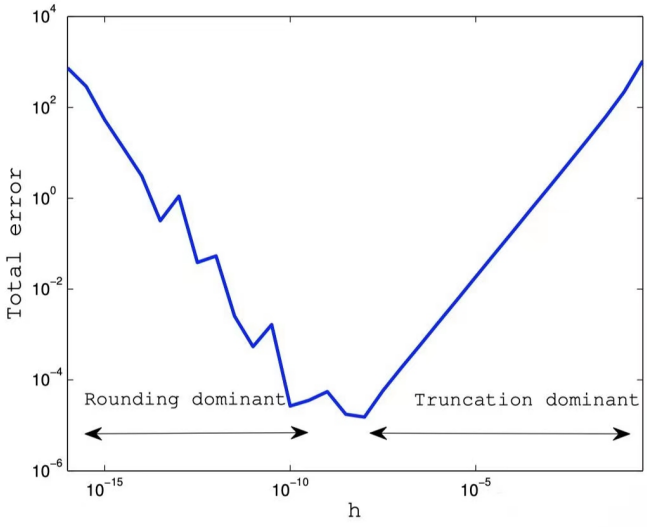


图 45: 舍入误差，截断误差示例

因此数值微分的实用性差，实际应用以下公式来减少截断误差：

$$f'(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x - \Delta x)}{2\Delta x} \tag{245}$$

另外一个问题是计算的复杂度。假设参数数量为 N，则每个参数都需要单独施加扰动，计算梯度。假设每次正向传播的计算复杂度为 $O(N)$，则计算数值微分的总体时间为 $O(N^2)$。(反向传播？)

4.5.2 符号微分 (Symbolic Differentiation)

符号微分是一种基于 **符号计算**的自动求导方法，符号计算也可以看作是代数计算，指用计算机来处理带有变量的数学表达式，此处的变量即符号 (Symbols)，一般不需要代入具体值，符号计算的输入输出都是数学表达式，一般是对输入表达式通过迭代或递归使用一些实现定义的规则进行运算，当转换结果不能再继续使用变换规则就停止计算。计算规则一般包括化简、因式分解、微分、积分、解代数方程、求解常微分方程等等。化简例：

$$\begin{aligned} \textbf{Input} \quad & 3x - x + 2x + 1 \\ \textbf{Output} \quad & 4x + 1 \end{aligned}$$

符号微分在编译时就计算梯度的数学表示，并进一步用符号计算进行优化，与计算平台无关 (可在 CPU 和 GPU 运行)。然而也有缺点：

1. 编译时间长尤其对于循环。
2. 为了符号微分需要设计一种专门的语言来表示数学表达式，且要对符号进行预先声明。
3. 很难对程序进行调试。

4.5.3 自动微分 (Automatic Differentiation, AD)

1. 概念：自动微分即一种可以对一个程序 (或函数) 进行导数计算的方法，符号微分的处理对象是数学表达式，而自动微分的处理对象是一个函数或者程序。
2. 原理：将所有的数值计算分解为一些基本操作，包含加减乘除和一些初等函数 exp, log, sin, cos 等，然后利用**链式法则**来自动计算一个复合函数的梯度。
3. 按计算顺序分：
   - 前向模式：按计算图中计算方向来递归地计算梯度。
   - 反向模式：按计算图中计算方向的相反方向来递归地计算梯度，**反向模式和反向传播的计算梯度的方式相同**。
   - 对比：对于一般的函数形式 $f\colon \mathbb{R}^N \to \mathbb{R}^M$，前向模式需要对每一个**输入**对象进行遍历，共需要 N 遍；反向模式需要对每一个**输出**进行一个遍历，共需要 M 遍。当 $N > M$ 时，反向更高效。在前馈神经网络中，风险函数 $f\colon \mathbb{R}^N \to \mathbb{R}$，输出为**标量** ($\mathbf{a}^L$)，因此反向模式为最有效的计算方式，只需要计算一遍。
4. 按构建方式分：
   - 静态计算图 (Static Computational Graph)：编译时构建计算图，计算图构建好后在程序运行时不能改变。(Theano, Tensorflow)
   - 动态计算图 (Dynamic Computational Graph)：程序运行时动态构建。

- 对比：两种模式各有优缺点。静态计算图在构建时可以优化，并行能力强，但灵活性差；动态计算图不容易优化，当不同输入网络结构不一致时，难以并行计算，但是灵活性较高。(DyNet、Chainer、Tensorflow2.0 和 PyTorch)

5. 符号微分和自动微分的对比：符号图和自动微分都利用计算图和链式法则自动求导。符号微分在编译阶段先构造一个复合函数的计算图，通过符号计算得到导数的表达式还可对导数表达式进行优化，在程序运行阶段才代入具体的值计算导数；自动微分无须实现编译，在运行阶段边计算边记录计算图，计算图上的局部梯度直接带入数值进行计算，继而用前向或者反向模式计算最终梯度。二者对比如图 46所示。

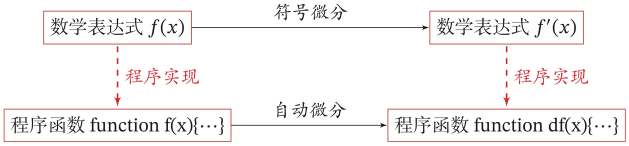


图 46: 符号微分自动微分对比

实例展示：以一个神经网络常见的复合函数例子展示自动微分的过程，令复合函数 $f(x;w,b)$：

$$f(x;w,b)=\frac{1}{exp(-(wx+b))+1} \tag{246}$$

其中 x 为输入标量。

首先将复合函数分解为一系列操作并构建计算图 (Computational Graph) 即数学运算的图形化表示。每个非叶子点表示一个基本操作，叶子节点为输入变量或常量，如图 47 所示即 $x=1$，$w=0$，$b=0$ 时复合函数的计算图。连边上的红色数字表示前向计算时复合函数每个变量的实际取值。由计算图得，$f(x;w,b)$ 由 6 个基本函数 h_i $(1\le i\le 6)$ 组成，每个基本函数的局

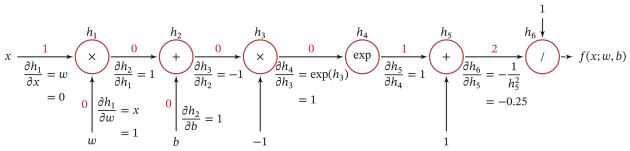


图 47: 计算图示例

部梯度都能很简单地被计算，如图 48 所示。

$f(x;w,b)$ 关于参数 w 和 b 的导数可以通过 $f(x;w,b)$ 与参数 w 和 b 之间路径上的所有导数的连乘得到：

$$\frac{\partial f(x;w,b)}{\partial w}=\frac{\partial f(x;w,b)}{\partial h_6}\frac{\partial h_6}{\partial h_5}\frac{\partial h_5}{\partial h_4}\frac{\partial h_4}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial w} \tag{247}$$

$$\frac{\partial f(x;w,b)}{\partial b}=\frac{\partial f(x;w,b)}{\partial h_6}\frac{\partial h_6}{\partial h_5}\frac{\partial h_5}{\partial h_4}\frac{\partial h_4}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial b} \tag{248}$$

以 $\frac{\partial f(x;w,b)}{\partial w}$ 为例，当 $x=1,w=0,b=0$ 时，可以得到：

$$\begin{aligned}\frac{\partial f(x;w,b)}{\partial w}\Big|_{x=1,w=0,b=0}=&\frac{\partial f(x;w,b)}{\partial h_6}\frac{\partial h_6}{\partial h_5}\frac{\partial h_5}{\partial h_4}\frac{\partial h_4}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial w}\\ =&1\times(-0.25)\times 1\times 1\times(-1)\times 1\times 1\\ =&0.25\end{aligned} \tag{249}$$

函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

图 48: 复合函数的 6 个基本函数及其导数

如果函数和参数有多条路径，可将这多条路径的导数再进行相加，得到最终的梯度。按照计算导数的顺序，自动微分可以分为两种模式：前向模式和反向模式。

1. 前向模式：仍然以 $\frac{\partial f(x;w,b)}{\partial w}$ 为例，当 $x = 1, w = 0, b = 0$ 时，可以得到:

$$\frac{\partial h_1}{\partial w} = x = 1 \tag{250}$$

$$\frac{\partial h_2}{\partial w} = \frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial w} = 1 \times 1 = 1 \tag{251}$$

$$\frac{\partial h_3}{\partial w} = \frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial w} = -1 \times 1 \tag{252}$$

$$... \tag{253}$$

$$\frac{\partial h_6}{\partial w} = \frac{\partial h_6}{\partial 5}\frac{\partial h_5}{\partial w} = -0.25 \times (-1) = 0.25 \tag{254}$$

$$\frac{\partial f(x;w,b)}{\partial w} = \frac{\partial f(x;w,b)}{\partial h_6}\frac{\partial h_6}{\partial w} = 1 \times 0.25 = 0.25 \tag{255}$$

2. 反向模式 (图 49)：条件同上：

$$\frac{\partial f(x;w,b)}{\partial h_6} = 1 \tag{256}$$

$$\frac{\partial f(x;w,b)}{\partial h_5} = \frac{\partial f(x;w,b)}{\partial h_6}\frac{\partial h_6}{\partial h_5} = 1 \times (-0.25) = -0.25 \tag{257}$$

$$\frac{\partial f(x;w,b)}{\partial h_4} = \frac{\partial f(x;w,b)}{\partial h_5}\frac{\partial h_5}{\partial h_4} = -0.25 \times 1 = -0.25 \tag{258}$$

$$... \tag{259}$$

$$\frac{\partial f(x;w,b)}{\partial w} = \frac{\partial f(x;w,b)}{\partial h_1}\frac{\partial h_1}{\partial w} = 0.25 \times 1 = 0.25 \tag{260}$$

4.6 优化问题

神经网络的参数学习比线性模型更困难，主要是非凸优化和梯度消失问题。

4.6.1 非凸优化

以最简单的 $1-1-1$ 结构的两层神经网络为例：

$$y = \sigma(w_2\sigma(w_1 x)) \tag{261}$$

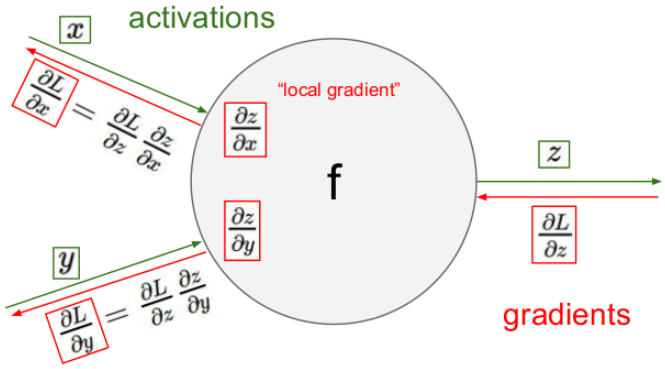
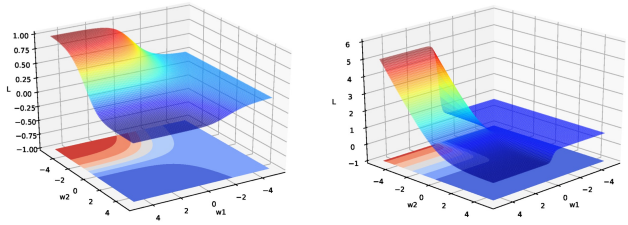


图 49: 反向传播模式

其中，w_1 和 w_2 为网络参数，$\sigma(\cdot)$ 为 Logistic 函数。给定一个输入样本 (1,1)，其平方损失函数和交叉熵损失函数分别为：$\mathcal{L}(w_1, w_2) = (1-y)^2$ 和 $\mathcal{L}(w_1, w_2) = \log y$。损失函数图像如图 50所示，都是非凸函数，非凸函数就会面临难以优化的问题，不像凸优化问题是局部最优即全局最优解。



(a) 平方误差损失　　(b) 交叉熵损失

图 50: $y = \sigma(w_2\sigma(w_1 x))$ 的损失函数

4.6.2 梯度消失问题 (Vanishing Gradient)

神经网络的误差反向传播公式：

$$\delta(l) = f_l'(\mathbf{z}^{(l)}) \odot (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \tag{262}$$

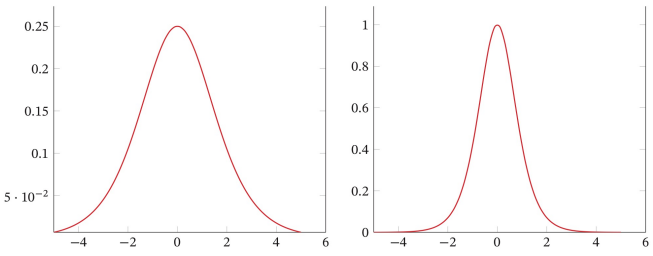
由此看出，误差从反向输出传播时，每一层都要乘该层激活函数的导数，如果我们使用 Sigmoid 型函数即 Logistic 函数或 Tanh 函数时，其导数为：

$$\sigma'(x) = \sigma(x)(1-\sigma(x)) \in [0, 0.25] \tag{263}$$

$$tanh'(x) = 1 - (tanh(x))^2 \in [0, 1] \tag{264}$$

Sigmoid 型函数的导数的值域小于或等于 1，如图 51所示。

由于 Sigmoid 函数两端饱和，因此饱和区的导数更是接近 0，这样误差在反向传播的过程中不断衰减，当误差很深，梯度会不停衰减直到消失影响网络的训练，即梯度消失或梯度弥散问题。(补充：[例] 与之相对应的即梯度爆炸：激活函数导函数的值域超过 1。) 因此需要激活的函数的导函数的值域在一个相对合理的范围内，比如 ReLU 激活函数，当输入 $x > 0$，ReLU 函数的导函数为 1，可以减轻梯度消失的问题。



(a) Logistic 函数的导数　　(b) Tanh 函数的导数

图 51: Sigmoid 型导数值域

5 卷积神经网络

5.1 引入

卷积神经网络主要用来处理图像信息 (图像分类、人脸识别、物体识别、图像分割等) 准确率较高，近年也被应用到自然语言处理、推荐系统等。若用全连接前馈网络会存在问题：

1. 参数过多：如果输入图像是 $100 \times 100 \times 3$，那么在第一层就会有 30000 个互相独立的连接以及对应的权重参数，同时随着隐藏层层数的增多，参数规模也急剧增加，导致效率低下和过拟合。

2. 局部不变性：对图像进行尺度缩放、平移、旋转、操作等不改变语义信息，而全连接前馈神经网络很难提取，一般需要数据增强来提高性能。

卷积神经网络的一些概念：

1. 启发点：生物学的感受野 (Receptive Field) 机制：神经元只接受其所支配区域的刺激区域 (主要在听觉、视觉上)，生物学上，在视觉神经系统中，视觉皮层中的神经细胞的输出依赖于视网膜上的光感受器受刺激兴奋时，将神经冲动信号传到视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。

2. 结构：由卷积层、汇聚层、全连接层交叉堆叠而成。

3. 特性：**局部连接**、**权重共享**以及**汇聚**。这些特性使得卷积神经网络具有一定程度上的平移、缩放和旋转不变性，且参数更少。

5.2 卷积 (Convolution)

卷积，也叫褶积，是分析数学中的一种重要运算，在信号或图像处理中，经常用一维或二维卷积。

5.2.1 一维卷积–信号处理

1. 信号的**延迟累积**：假设一个信号发生器：

   - t 时刻产生一个信号 x_t

- **信息衰减率**：w_k 指在 $k-1$ 个时间步长后，信息为原来的 w_k 倍。

假设 $w_1=1$, $w_2=\frac{1}{2}$, $w_3=\frac{1}{4}$ ($w_4=0$, 信息衰减为 0)，那么在 t 时刻收到的信号 y_t 为**当前时刻**产生的信息和**以前时刻**延迟信息的**叠加**:

$$\begin{aligned} y_t =& 1 \times x_t + \frac{1}{2} \times x_{t-1} + \frac{1}{4} \times x_{t-2} \\ =& w_1 \times x_t + w_2 \times x_{t-1} + w_3 \times x_{t-2} \\ =& \sum_{k=1}^{3} w_k x_{t-k+1} \end{aligned} \tag{265}$$

把 w_1，w_2 称为滤波器 (Filter) 或卷积核 (Convolution Kernel)，假设滤波器长度为 K，它和一个信号序列的卷积:

$$y_t = \sum_{k=1}^{K} w_k x_{t-k+1} \tag{266}$$

为了简单起见，假设卷积的输出 y_t 的下标 t 从 K 开始 (x 的下标也会从 K 开始，和 w 的下标是逆序)。

2. 卷积的定义：信号序列 $\mathbf{x}$ 和滤波器 $\mathbf{w}$ 的定义：

$$\mathbf{y} = \mathbf{w} * \mathbf{x} \tag{267}$$

$*$ 即卷积运算，一般情况下滤波器的长度 K 远小于信号序列 $\mathbf{x}$ 的长度。

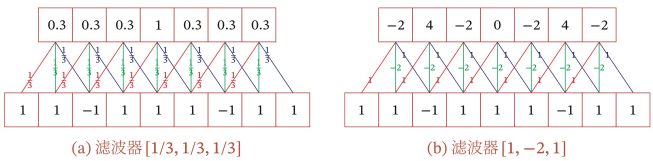
3. 滤波器的应用：可以设计不同的滤波器来提取信号序列的不同特征。比如当令滤波器 $\mathbf{w}=[1/K,...,1/K]$ 时，卷积相当于信号序列的简单移动平均 (窗口大小为 K)。(补充：移动平均–Moving Average 是在分析时间序列数据时的一种简单平滑技术，能有效消除数据的随机波动。)

   - 近似实现二阶微分：当令滤波器 $\mathbf{w}=[1,-2,1]$ 时可以近似实现对信号序列的二阶微分 (证)：

   $$x''(t) = x(t+1) + x(t-1) - 2x(t) \tag{268}$$

   注：$x(t)=x_t$ 即将输入信号序列看作是关于时间的函数。

   - 提取高频和低频信息：如图 52所示，滤波器 $\mathbf{w}=[1/3,1/3,1/3]$ 可检测信号序列的低频信息，滤波器 $\mathbf{w}=[1,-2,1]$ 可以检测信号序列的高频信息，其中，高低频指信号变化的剧烈程度，下层为输入序列，上层为卷积结果，连接边上的数字为滤波器中的权重，图 (a) 的输出结果为近似值。



(a) 滤波器 $[1/3,1/3,1/3]$　　(b) 滤波器 $[1,-2,1]$

图 52: 一维卷积示例

5.2.2 二维卷积–图像处理

1. 定义：二维卷积是一维卷积的拓展，给定一个图像 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和一个滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$ 一般 $U << M$，$V << N$，则二维卷积为：

$$y_{ij} = \sum_{u=1}^{U} \sum_{v=1}^{V} w_{uv} x_{i-u+1, j-v+1} \tag{269}$$

此处假设卷积的输出 y_{ij} 的下标 (i, j) 从 (U, V) 开始，那么可以定义输入信息和滤波器的二维卷积：

$$\mathbf{Y} = \mathbf{W} * \mathbf{X} \tag{270}$$

2. 图示：如图 53所示。

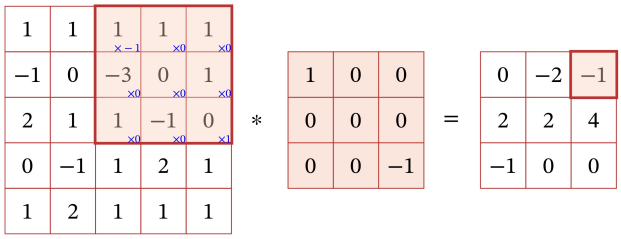


图 53: 二维卷积示例

3. 几种滤波器：在图像处理中，卷积经常作为特征提取的有效方法，一副图像在经过卷积操作之后得到的结果称为特征映射 (Feature Map)。

   - 均值滤波 (Mean Filter)：一种二维卷积，将当前位置的像素值设为滤波器窗口中所有像素的均值即 $w_{uv} = \frac{1}{UV}$
   - 高斯滤波器：用来对图像进行平滑去噪。

   同时可以设计不同的滤波器来提取想要的特征，如图所示，中间 (二维二阶微分) 和下面的滤波器可用来提取边缘特征。

5.2.3 互相关 (Cross-Correlation)

由于互相关是不翻转卷积，因此在具体操作上会以互相关来代替卷积来减少不必要的开销。互相关用来衡量两个序列的相关性，通常使用滑动窗口的点积来实现，给定一个图像 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和一个滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$，它们的互相关：

$$y_{ij} = \sum_{u=1}^{U} \sum_{v=1}^{V} w_{uv} x_{i+u-1, j+v-1} \tag{271}$$

即不翻转卷积，可由此公式看出和真正的卷积之间的关系：

$$\mathbf{Y} = \mathbf{W} \otimes \mathbf{X} = \text{rot180}(\mathbf{W}) * \mathbf{X} \tag{272}$$

其中，输出 $\mathbf{Y} \in \mathbb{R}^{M-U+1, N-V+1}$，$\otimes$ 即互相关运算，之后的描述中除非特殊声明，卷积一般指“互相关”，在神经网络中，卷积的主要目的是特征提取，因此是否翻转即下标是否变化和提取能力无关，因此互相关和真正的卷积等价，用互相关来代替卷积。

5.2.4 卷积变种

在卷积的标准定义的基础上可以引入滑动步长和零填充来增加卷积的多样性。

1. 步长 (Stride)：即卷积核在滑动时的时间间隔，可以通过调整步长来调整输出特征映射的个数。

2. 零填充 (Zero Padding)：即在输入向量的**两端**进行补 0，使得图像的边缘特征被提取到，或者增加其对输出影响的重要性，如图 54 所示。

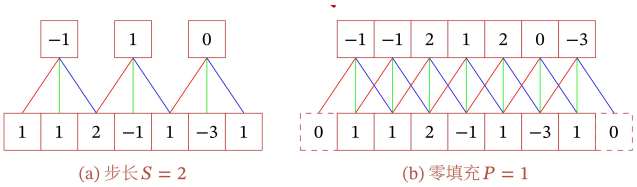


图 54: 卷积的步长和零填充

3. 几种卷积：根据不同的步长和填充以及输入输出信号序列的关系，可以将卷积进行分类。假设卷积层的输入神经元个数为 M，卷积大小为 K，步长为 S，在输入两端各补 P 个 0，则该卷积层**输出**的神经元的数量为 $(M - K + 2P)/S + 1$ (选择合适的卷积大小和步长使其为整数)。

   - 窄卷积 (输出长度 $<$ 输入长度–Narrow Convolution)：$S = 1$，$P = 0$，卷积后的输出长度为 $M - K + 1$。
   - 宽卷积 (输出长度 $>$ 输入长度–Wide Convolution)：$S = 1$，$P = K - 1$，卷积后的输出长度为 $M + K - 1$。
   - 等宽卷积 (输出长度 $=$ 输入长度–Equal Width Convolution)：$S = 1, P = (K-1)/2$，卷积后的输出长度为 M。

注：早期文献卷积一般指窄卷积；目前文献，卷积一般默认为等宽卷积。

5.2.5 卷积数学性质

1. 交换性：

   - 真正的卷积：如果不限制两个卷积信号的长度，真正的卷积是有交换性的即 $\mathbf{x} * \mathbf{y} = \mathbf{y} * \mathbf{x}$。
   - 二维宽卷积 (Wide Convolution)：给定一个图像 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和一个滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$，对图像 $\mathbf{X}$ 进行 0 填充，两端各补 $U - 1$ 和 $V - 1$ 个零，得到全填充 (Full Padding) 图像 $\tilde{\mathbf{X}} \in \mathbb{R}^{(M+2U-2) \times (N+2V-2)}$，图像和卷积核的宽卷积的定义：

$$\mathbf{W} \tilde{\otimes} \mathbf{X} \triangleq \mathbf{W} \otimes \tilde{\mathbf{X}} \tag{273}$$

   - 输入信息和卷积核有固定宽度的宽卷积–仍然具有交换性。

$$\text{rot180}(\mathbf{W}) \tilde{\otimes} \mathbf{X} = \text{rot180}(\mathbf{X}) \tilde{\otimes} \mathbf{W} \tag{274}$$

2. 导数：假设 $\mathbf{Y}=\mathbf{W}\otimes\mathbf{X}$，其中 $\mathbf{X}\in\mathbb{R}^{M\times N},\mathbf{W}\in\mathbb{R}^{U\times V},\mathbf{Y}\in\mathbb{R}^{(M-U+1)\times(N-V+1)}$，$f(\mathbf{Y})\in\mathbb{R}$ 为一个标量函数，那么，

$$\frac{\partial f(\mathbf{Y})}{\partial w_{uv}}=\sum_{i=1}^{M-U+1}\sum_{j=1}^{N-V+1}\frac{\partial y_{ij}}{\partial w_{uv}}\frac{\partial f(\mathbf{Y})}{\partial y_{ij}} \tag{275}$$

根据公式 271，并交换点积顺序，得到

$$\frac{\partial f(\mathbf{Y})}{\partial w_{uv}}=\sum_{i=1}^{M-U+1}\sum_{j=1}^{N-V+1}x_{i+u-1,j+v-1}\frac{\partial f(\mathbf{Y})}{\partial y_{ij}}=\sum_{i=1}^{M-U+1}\sum_{j=1}^{N-V+1}\frac{\partial f(\mathbf{Y})}{\partial y_{ij}}x_{u+i-1,v+j-1} \tag{276}$$

因此 $f(\mathbf{Y})$ 关于 $\mathbf{W}$ 的偏导即 $\mathbf{X}$ 和 $\frac{\partial f(\mathbf{Y})}{\partial \mathbf{Y}}$ 的卷积：

$$\frac{\partial f(\mathbf{Y})}{\partial \mathbf{W}}=\frac{\partial f(\mathbf{Y})}{\partial \mathbf{Y}}\otimes\mathbf{X} \tag{277}$$

令 $s=u+i-1,t=v+j-1$，得到 $u=s+1-i,v=t+1-j$。同理可得，

$$\frac{\partial f(\mathbf{Y})}{\partial x_{st}}=\sum_{i=1}^{M-U+1}\sum_{j=1}^{N-V+1}\frac{\partial y_{ij}}{x_{st}}\frac{\partial f(\mathbf{Y})}{y_{ij}} \tag{278}$$

$$=\sum_{i=1}^{M-U+1}\sum_{j=1}^{N-V+1}\omega_{s-i+1,t-j+1}\frac{\partial f(\mathbf{Y})}{y_{ij}} \tag{279}$$

其中当 $(s-i+1)<1$, 或 $(s-i+1)>U$, 或 $(s+i-1)<1$, 或 $(s+i-1)>V$ 时，$\omega_{s-i+1,t-j+1}=0$, 和 W 进行了 $P=(M-U,N-V)$ 的零填充等效。

依据公式279，可看出 $f(\mathbf{Y})$ 关于 $\mathbf{X}$ 的偏导为 $\mathbf{W}$ 和 $\frac{\partial f(\mathbf{Y})}{\partial \mathbf{Y}}$ 的宽卷积（非互相关），因此统一为互相关格式：

$$\frac{\partial f(\mathbf{Y})}{\partial \mathbf{X}}=rot180(\frac{\partial f(\mathbf{Y})}{\partial \mathbf{Y}})\tilde{\otimes}\mathbf{W} \tag{280}$$

$$=rot180(\mathbf{W})\tilde{\otimes}\frac{\partial f(\mathbf{Y})}{\partial \mathbf{Y}} \tag{281}$$

5.3 卷积神经网络

卷积神经网络一般由卷积层、汇聚层和全连接层构成。

5.3.1 用卷积代替全连接

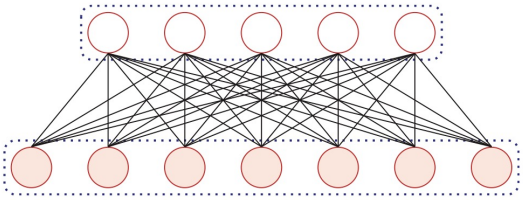
1. 全连接缺点：如果用全连接前馈神经网络，第 l 层有 M_l 个神经元，第 $l-1$ 层有 M_{l-1} 个神经元，则连接边或权重矩阵就有 $M_l\times M_{l-1}$ 个参数，当二者都很大时，权重矩阵的参数会很多，导致训练效率降低。

2. 卷积的两个特性：用卷积代替全连接，那么

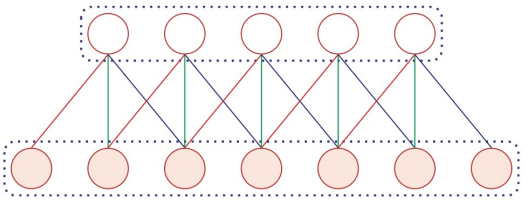
$$\mathbf{z}^{(l)}=\mathbf{w}^{(l)}\otimes\mathbf{a}^{(l-1)}+b^{(l)} \tag{282}$$

$\mathbf{z}^{(l)}$ 为第 l 层的净输入，$\mathbf{a}^{(l-1)}$ 即前一层的激活值，卷积核 $\mathbf{w}^{(l)}\in\mathbb{R}^K$ 和 $b^{(l)}\in\mathbb{R}$ 为可学习的参数。因此卷积有两个重要特性：

- 局部连接：在卷积层 (假设第 l 层) 中的每个神经元都只和前一层即第 $l-1$ 层中某个局部窗口中的神经元相连，构成一个局部的神经网络，因此卷积层和前一层的连接数大大减少，由 $M_l \times M_{l-1}$ 个连接变为 $M_l \times K$ 个连接，K 即卷积核大小，如图所示。
- 权重共享：由公式 282 得，卷积核 $\mathbf{w}^{(l)}$ 对于第 l 层所有的神经元都是相同的；同时由图 55得，所有同颜色连接上的权重都是相同的，可以理解为一个卷积核只捕捉输入数据中的**一种**特定的局部特征，因此如果需要提取**多种**特征就要使用**多个**不同的卷积核 (后文的 P)，一种卷积核对应一种输出。



(a) 全连接层



(b) 卷积层

图 55: 全连接层和卷积层对比

因此由于局部连接和权重共享，卷积层的参数只有一个 K 维的权重和 1 维的偏置共 $K+1$ 个参数，参数的个数与神经元的数量无关 (神经元的个数是和连接数有关)，此外第 l 层的神经元个数不是任意选择的，而是满足 $M_l = M_{l-1} - K + 1$，默认步长为 1，无零填充。

5.3.2 卷积层 (Convolution layer–图像提取)

1. 作用：卷积层的作用是提取一个局部区域的特征，不同的卷积核相当于不同的特征提取器，在图像处理上为了更充分地利用局部信息，将神经元组织为三维结构的神经层，由 D 个 $M \times N$ 大小的特征映射构成即 $M \times N \times D$。

2. 特征映射：即卷积提取的结果。每个特征映射可以作为一类抽取的图像特征。为了提高卷积网络的表示能力，可以在每一层使用多个不同的特征映射，以更好地表示图像的特征。

3. 一般结构：如图 56 所示

   (a) 输入特征映射组：$\chi \in \mathbb{R}^{M \times N \times D}$ 为三维张量，其中每个切片矩阵 $\mathbf{X}^d \in \mathbb{R}^{M \times N}$ 为一个输入特征映射，$1 \leqslant d \leqslant D$。

(b) 输出特征映射组：$\mathbf{y} \in \mathbb{R}^{M' \times N' \times P}$ 为三维张量，其中每个切片矩阵 $\mathbf{Y}^p \in \mathbb{R}^{M' \times N'}$ 为一个输出特征映射，$1 \leqslant p \leqslant P$。

(c) 卷积核：$W \in \mathbb{R}^{U \times V \times P \times D}$ 四维张量，每个切片矩阵 $\mathbf{W}^{p,d} \in \mathbb{R}^{U \times V}$ 。

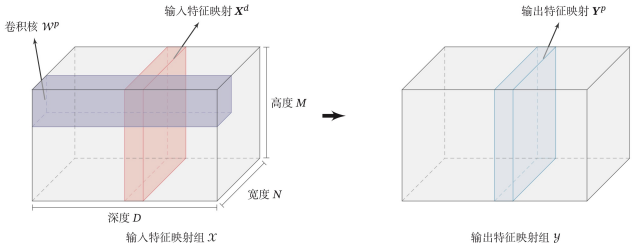


图 56: 卷积层的三维结构

4. 计算卷积过程：

- 内部拆解: 图 57 所示

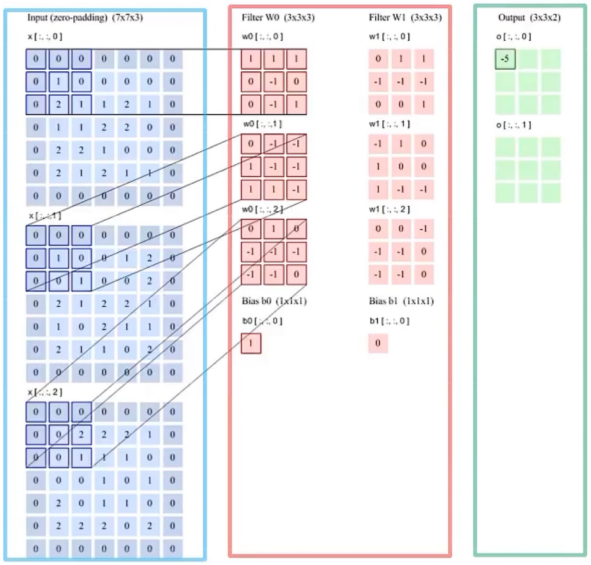


图 57: 三维计算输出特征映射

- 完整流程: 图 58 所示
- 文字描述：用卷积核 $W^{p,1}, W^{p,2}, ...W^{p,D}$ 分别对输入特征映射 $X^1, X^2, ..., X^D$ 进行卷积，然后将卷积结果相加，并加上一个标量偏置 b^p 得到卷积层的净输入 Z^p，（是指没有经过非线性激活函数的净活性值（Net Activation），再经过非线性激活函数后得到输出特征映射 Y^p。

$$\mathbf{Z}^p = \mathbf{W}^p \otimes X + b^p = \sum_{d=1}^{D} \mathbf{W}^{(p,d)} \otimes \mathbf{X}^d + b^p \tag{283}$$

$$\mathbf{Y}^p = f(\mathbf{Z}^p) \tag{284}$$

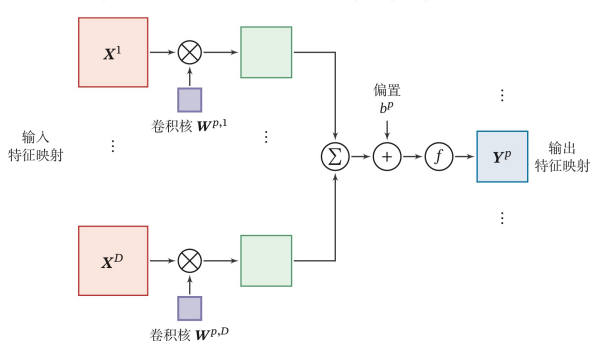


图 58: 三维计算输出特征映射完整示例

$\mathbf{W}^p \in \mathbb{R}^{U\times V\times D}$ 为三维卷积核，$f(\cdot)$ 为非线性激活函数，一般为 ReLU 函数。用卷积核 $W^{p,1}, W^{p,2}, ...W^{p,D}$ 分别对输入特征映射 $X^1, X^2, ..., X^D$ 进行卷积，然后将卷积结果相加，并加上一个标量偏置 b^p 得到卷积层的净输入 Z^p，（是指没有经过非线性激活函数的净活性值（Net Activation），再经过非线性激活函数后得到输出特征映射 Y^p。

$$\mathbf{Z}^p = \mathbf{W}^p \otimes X + b^p = \sum_{d=1}^{D} \mathbf{W}^{(p,d)} \otimes \mathbf{X}^d + b^p \tag{285}$$

$$\mathbf{Y}^p = f(\mathbf{Z}^p) \tag{286}$$

5.3.3 汇聚层 (Pooling layer–图像压缩)

汇聚层也叫子采样层 (Subsampling Layer) 。

1. 作用：图像压缩，进行特征选择，降低特征数量，最终降低了参数的数量。
2. 原因：卷积层只进行了对特征的提取，虽然减少了权重数量，但并未减少神经元个数，容易出现过拟合，而加在卷积层之后的汇聚层可以通过降低特征维数来避免过拟合。
3. 汇聚：也指下采样 (Down Sampling) 即将特征映射划分为很多区域，可以重叠或不重叠，常用的汇聚函数：
   - 最大汇聚 (Max Pooling)：对一个区域 $R^d_{m,n}$ ，用这个区域内所有神经元最大活性值来表示这个区域：

     $$y^d_{m,n} = \max_{i\in R^d_{m,n}} x_i \tag{287}$$

     如图 59所示，如果用最大汇聚层的方式对 2×2 大小的不重叠区域进行下采样即构成了一个典型的汇聚层。注意采样不能过大否则会减小神经元的数量，造成信息的损失。
   - 平均汇聚 (Mean Pooling)：取此区域所有神经元活性的平均值来表示这个区域：

     $$y^d_{m,n} = \frac{1}{|R^d_{m,n}|} \sum_{i\in R^d_{m,n}} x_i \tag{288}$$

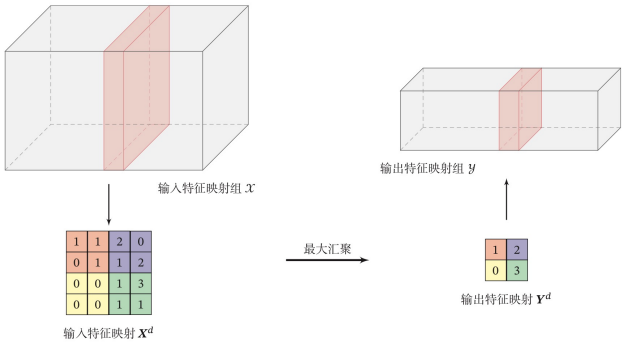


图 59: 最大汇聚示例

5.3.4 整体结构

卷积神经网络是由卷积层、汇聚层、全联接层交叉堆叠形成的，目前常用的卷积网络的整体结构如图 60所示。

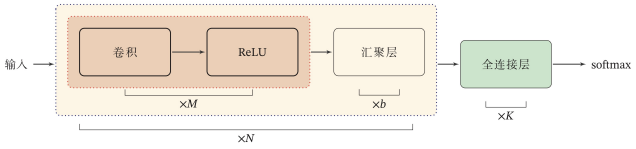


图 60: 卷积神经网络的结构

- 卷积块：由连续的 M (2 到 5) 个卷积层和 b (0 或 1) 个汇聚层组成。一个卷积神经网络可以堆叠 N (1 到 100) 个连续的卷积块。
- 全联接层：在卷积块后面连接的 K (0 到 2) 个全连接层
- 趋势：倾向于选择更小的卷积核 (1× 1 和 3×3) 和更深的结构 (大于 50)；由于卷积的操作逐渐灵活，汇聚层的作用越来越少，趋向于全卷积网络。

5.4 参数学习

类似前馈神经网络，梯度主要通过前一层的误差反向传播，由于参数为卷积核及其偏置，因此只需要计算卷积层中参数的梯度。因此对于第 l−1 层的输入特征映射为 $\chi^{(l-1)} \in \mathbb{R}^{M \times N \times D}$，通过卷积计算得到第 l 层的特征映射净输入 $Z^{(l)} \in R^{M' \times N' \times P}$，第 l 层的第 $p(1 \leqslant p \leqslant P)$ 个特征映射净输入为

$$\mathbf{Z}^{(l,p)} = \sum_{d=1}^{D} \mathbf{W}^{(l,p,d)} \otimes \mathbf{X}^{(l-1,d)} + b^{(l,p)} \tag{289}$$

其中第 l 层中共有 $P \times D$ 个卷积核和 P 个偏置，可参照 4.4 用链根据公式289，损失函数 $\mathcal{L}$ 关于第 l 层的卷积核 $\mathbf{W}^{(l,p,d)}$ 的偏导为：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l,p,d)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(l,p)}} \otimes \mathbf{X}^{(l-1,d)} \tag{290}$$

$$= \delta^{(l,p)} \otimes \mathbf{X}^{(l-1,d)} \tag{291}$$

同理，损失函数 $\mathcal{L}$ 关于第 l 层的第 p 个偏置 $b^{(l,p)}$ 的偏导为：

$$\frac{\partial \mathcal{L}}{\partial b^{(l,p)}} = \sum_{i,j} [\delta^{(l,p)}]_{i,j} \tag{292}$$

每层参数的梯度依赖误差项 $\delta^{(l,p)}$ 。

5.5 卷积网络反向传播算法

在卷积神经网络中需要分开计算卷积层和汇聚层的误差。

1. 汇聚层：当第 $(l+1)$ 层为汇聚层时，由于汇聚是下采样因此需要先对第 $l+1$ 层的误差项进行上采样使其和第 l 层的大小一样再和该层特征映射的激活值的偏导数逐元素相乘，进而得到 $\delta^{(l,p)}$。

   (a) 最大汇聚：误差项 $\delta^{(l+1,p)}$ 中每个值会直接传递到前一层对应区域中的最大值所对应的神经元，该区域中其他神经元的误差项都设为 0。

   (b) 平均汇聚：误差项 $\delta^{(l+1,p)}$ 中每个值会被平均分配到前一层对应区域中的所有神经元上。

   第 l 层第 p 个特征映射的误差项 $\delta^{(l,p)}$ 为：

$$\begin{aligned}
\delta^{(l,p)} &= \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{l,p}} && (293)\\
&= \frac{\partial \mathbf{X}^{(l,p)}}{\partial \mathbf{Z}^{(l,p)}} \frac{\partial \mathbf{Z}^{(l+1,p)}}{\partial \mathbf{X}^{(l,p)}} \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(l+1,p)}} && (294)\\
&= f_l'(\mathbf{Z}^{(l,p)}) \odot up(\delta^{(l+1,p)}) && (295)
\end{aligned}$$

2. 卷积层：当第 $(l+1)$ 层为卷积层时，第 l 层第 d 个特征映射的误差项 $\delta^{(l,d)}$ 为，

$$\begin{aligned}
\delta^{(l,d)} &= \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(l,d)}} && (296)\\
&= \frac{\partial \mathbf{X}^{(l,d)}}{\partial \mathbf{Z}^{(l,d)}} \frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(l,d)}} && (297)\\
&= f_l'(\mathbf{Z}^{(l,d)}) \odot \sum_{P=1}^{P} (rot180(\mathbf{W}^{(l+1,p,d)} \tilde{\otimes} \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(l+1,p)}}) && (298)\\
&= f_l'(\mathbf{Z}^{(l,d)}) \odot \sum_{P=1}^{P} (rot180(\mathbf{W}^{(l+1,p,d)} \tilde{\otimes} \delta^{(l+1,p)}) && (299)
\end{aligned}$$

5.6 几种典型的卷积

本节是对以上内容的应用。

5.6.1 LeNet-5

1. 特点：LeNet-5 提出时间较早，在 20 世纪 90 年代被美国很多银行使用，用来识别支票上面的手写数字。

2. 结构图：如图 61 所示，LeNet-5 共 7 层，接受输入图像的大小为 $32 \times 32 = 1024$ ，输出对应 10 个类别的得分。

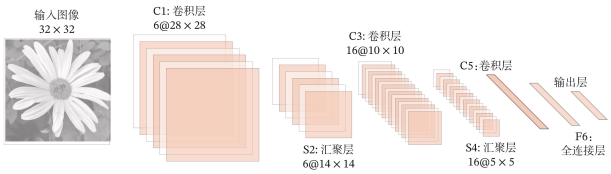


图 61: LeNet-5

3. 连接表 (Link Table): 由于卷积层的输入和输出特征映射之间之间的全连接的关系不是必须的，可以让每一个输出特征映射都依赖与少数几个输入特征映射，可定义一个连接表来描述输入和输出特征映射之间的连接关系。如果第 p 个输出特征映射依赖于第 d 个

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X

图 62: 连接表

输入特征映射, 则 $T_{p,d} = 1$, 否则为 0。输出特征映射为

$$\mathbf{Y}^p = f\left(\sum_{\substack{d,\\ T_{p,d}=1}} \mathbf{W}^{p,d} \otimes \mathbf{X}^d + b^p\right) \tag{300}$$

其中 T 为 $P \times D$ 大小的连接表，如果连接表 T 的非零个数为 K, 每个卷积核的大小为 $U \times V$, 那么共需要 $K \times U \times V + P$ 参数。

5.6.2 AlexNet

1. 特点：是第一个现代深度卷积网络模型，其首次使用很多现代深度卷积网络的技术方法，采用 ReLU 作为非线性激活函数，可以使用 Dropout 防止过拟合，使用数据增强来提高模型准确率。

2. 网络结构：输入为 $224 \times 224 \times 3$ 输出为 1000 个类别的条件概率。如图 63所示，包括 5 个卷积层，3 个汇聚层和 3 个全连接层。其将网络拆半放在两个 GPU 上，GPU 只在某些层进行通信。

5.6.3 Inception 网络

1. 特点：为了解决梯度消失问题，*GoogLeNet* 在网络中间层引入两个辅助分类器来加强监督信息。Inception 网络由多个 Inception 模块和少量的汇聚层堆叠而成。

2. Inception 模块结构：在 Inception 网络中，一个卷积层包含多个不同大小的卷积操作即 Inception 模块。如图 64所示，采用了四组平行的特征抽取方式。另外，为了提高效率，减少参数数量，模块进行 1×1 的卷积的目的：

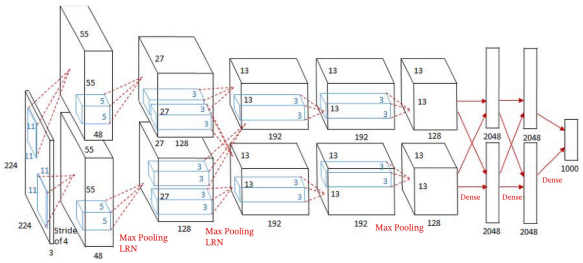


图 63: AlexNet 网络结构

- 减少特征映射的深度。
- 如果输入特征映射之间存在冗余信息，相当于先进行一次特征抽取。

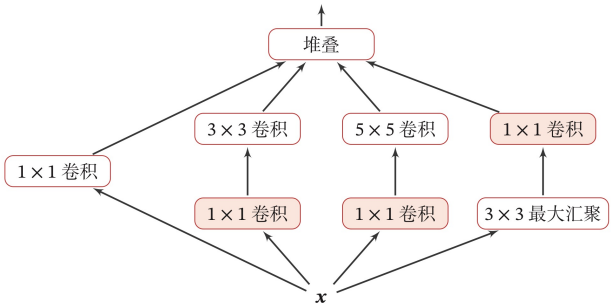


图 64: Inception v1 的模块结构

3. 网络结构：GoogLeNet 由 9 个 Inception v1 模块和 5 个汇聚层以及其他一些卷积层和全连接层构成，共 22 层网络，如图 65 所示。

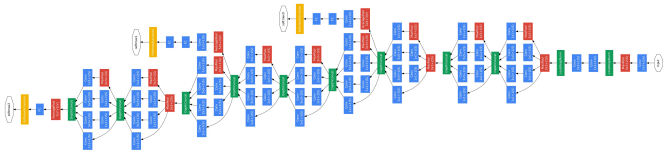


图 65: GoogLeNet 网络结构

5.6.4 残差网络 (Residual Network)

1. 特点：通过给非线性的卷积层增加直连边 (Shortcut Connection) 的方式来提高信息的传播效率。

2. 原理：在一个深度网络中，期望一个非线性单元 $f(\mathbf{x};\theta)$ 去逼近一个目标函数 $h(\mathbf{x})$。

$$h(\mathbf{x}) = \mathbf{x} + (h(\mathbf{x}) - \mathbf{x}) \tag{301}$$

- 恒等函数 (Identity Function)：$\mathbf{x}$
- 残差函数 (Residue Function)：$h(\mathbf{x}) - \mathbf{x}$

由于神经网络能够逼近原始目标函数或残差函数，因此优化问题可以转换为用非线性单元近似残差函数，并且用 $f(\mathbf{x};\theta)+\mathbf{x}$ 去逼近 $h(\mathbf{x})$。

- $f(\mathbf{x};\theta) \longrightarrow h(\mathbf{x})-\mathbf{x}$
- $f(\mathbf{x};\theta)+\mathbf{x} \longrightarrow h(\mathbf{x})$

3. 结构图：如图 66所示，残差单元由多个等宽卷积层和跨层的直连边组成，再经过 ReLU 激活输出，很多残差单元串联起来深度神经网络，类似的还有 Highway Network。

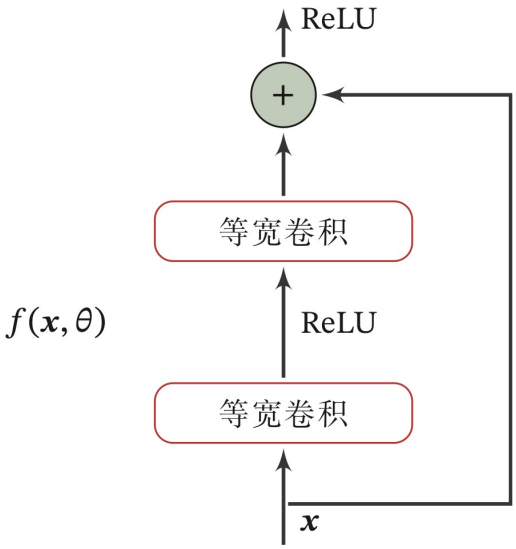


图 66: 一个简单的残差单元结构